

VIETNAMESE-GERMAN UNIVERSITY

Faculty of Engineering

Computer Science Program

**Multi-Agent Reinforcement Learning Framework for Graph-Aware
Traffic Signal Control for German Traffic Scenarios**

Full name: Tran Hai Duong

Student ID: 10422021

First supervisor: Dr. Nguyen Tuan Cuong

Second supervisor: Prof. Dr. Javad Ghofrani

BACHELOR THESIS

*Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science
(B.Sc.) in the study program Computer Science Program, Vietnamese-German University*

Ho Chi Minh City, Vietnam

30. June 2026

Declaration

Hereby I assure that I have written the presented thesis independently and without any third party help and that I have not used any other tools or resources than the ones mentioned/referred to in the thesis.

Any parts of the thesis that have been taken from other published or yet unpublished works in terms of their wording or meaning are thoroughly marked with an indication of the source.

All figures in this thesis have been drawn by myself or are clearly attributed with a reference.

This work has not been published or presented to any other examination authority before. I am aware of the importance of the affidavit and the consequences under examination law as well as the criminal consequences of an incorrect or incomplete affidavit.

Ho Chi Minh City, 30. June 2026

Tran Hai Duong

Acknowledgements

This thesis would not have been possible without the guidance, support, and encouragement of many people throughout my journey.

First and foremost, I would like to express my deepest gratitude to my first supervisor, Dr. Nguyen Tuan Cuong. Over the past years, through both my thesis work and my experience as his teaching assistant, I have learned a great deal from his guidance, patience, and dedication. His regular meetings, thoughtful advice, and timely support helped me keep this thesis on the right path and overcome many challenges along the way.

I am also sincerely grateful to Prof. Dr. Javad Ghofrani for his valuable supervision and insightful feedback throughout this research. His perspectives helped me reflect more deeply on the direction of the work and improve the quality of this thesis.

I would like to thank Mr. Ton That Nhat Minh, my friend and fellow researcher, for accompanying me through this research journey. I truly appreciate the long discussions, the late nights, and the shared effort we put into developing ideas, baselines, methods, and analyses. Working with him made this challenging process more meaningful, and I am grateful for both his dedication and his companionship.

My sincere thanks also go to Asst. Prof. Pham Duc Thinh and Dr. Le Trung Hoang at the Center for AI Research, VinUniversity. Although their guidance was not directly tied to this thesis, their mentorship in another traffic control project has greatly shaped the way I approach research. Their insights have helped me take important steps toward becoming a more independent researcher.

I would also like to thank my friends, relatives, and family for their constant emotional support throughout this journey. In particular, I am deeply grateful to Mr. Dang Tran Long and Ms. Dang Phuong Mai for being with me through the ups and downs, especially during the moments when I doubted myself. Their encouraging words, care, and presence gave me strength when I needed it most.

Finally, I would like to quietly thank myself for not giving up. This journey came with pain, pressure, and many challenges, but I am grateful that I kept going, one step at a time.

Abstract

Traffic signal control at the network level constitutes a canonical multi-agent decision problem in which inter-agent coordination can substantially reduce urban congestion. Graph-based multi-agent reinforcement learning (MARL) methods, which treat signalized intersections as nodes in a communication graph, have shown promise in synthetic environments but remain underexplored on real-world German road networks characterized by irregular topologies and heterogeneous phase structures.

This thesis proposes FGS (Framework for Graph-aware Signal control), a modular three-module pipeline for MARL-based traffic signal control that decouples the local encoder, graph communication layer, and RL algorithm as independently configurable design choices. The FGS pipeline is developed through three generations of iterative diagnosis-and-repair; the final FGS design combines demand-aware GATv2 communication with a factored neighborhood critic ($O(d_c)$ independent of N), enabling both SAC and PPO to complete training at the largest evaluated scenario. The framework is implemented on SUMO, SUMO-RL, and RLlib under the CTDE paradigm.

All six German RESCO benchmark scenarios spanning three scale tiers — Single-intersection (cologne1, ingolstadt1), Corridor (cologne3, ingolstadt7), and Regional (cologne8, ingolstadt21) — are evaluated against seven baselines: fixed-time control, max-pressure, IDQN, IPPO, independent SAC, FRAP, and CoLight. Performance is reported on mean waiting time, mean delay, mean trip duration, and throughput.

A component-level ablation across all eight design-space combinations establishes three design guidelines. First, MLP encoders outperform FRAP-based encoders on German intersection heterogeneity by up to $2.6\times$ on mean delay at Regional scale ($N = 21$); FRAP’s phase-competition priors are structurally mismatched with German variable-phase layouts. Second, GATv2 dynamic attention reduces mean delay by 52% over GAT at Regional scale ($N = 21$; 234.9 s vs. 492.0 s) while providing negligible benefit at smaller scale. Third, SAC achieves 8% lower mean delay than PPO at moderate scale on cologne8 (22.2 s vs. 24.0 s) but requires the FGSv3 factored critic to complete training at $N = 21$.

FGSv3-PPO achieves 68.4 s mean delay on ingolstadt21, competitive with standalone PPO (67.2 s) and strongly outperforming CoLight (216.8 s, -68%) and DQN (78.7 s, -13%). FGS(MLP+GATv2+SAC) achieves 22.2 s mean delay on the Regional cologne8 scenario, outperforming CoLight (51.8 s) by 57%. These results demonstrate that the FGS pipeline is an effective and analytically informative framework for characterizing MARL-TSC design on German road networks. All reported figures are best-validation estimates from single-seed runs; multi-seed statistical validation is identified as a priority for future work.

Keywords: multi-agent reinforcement learning, traffic signal control, graph attention network, soft actor-critic, FRAP encoder, RESCO benchmark, decentralized execution, German traffic scenarios

Kurzfassung

Die Verkehrssignalsteuerung auf Netzwerkebene stellt ein kanonisches Mehrfach-Agenten-Entscheidungsproblem dar, bei dem eine koordinierte Abstimmung zwischen Agenten den städtischen Verkehrsstau erheblich verringern kann. Graphbasierte Methoden des Multi-Agenten Reinforcement Learning (MARL), die Lichtsignalanlagen als Knoten in einem Kommunikationsgraphen modellieren, haben in synthetischen Umgebungen vielversprechende Ergebnisse gezeigt, wurden jedoch für reale deutsche Straßennetze mit unregelmäßigen Topologien und heterogenen Phasenstrukturen kaum untersucht.

Diese Arbeit schlägt FGS (Framework for Graph-aware Signal control) vor, eine modulare Drei-Modul-Pipeline für MARL-basierte Verkehrssignalsteuerung, die lokalen Kodierer, Graphkommunikationsschicht und RL-Algorithmus als unabhängig konfigurierbare Entwurfsentscheidungen entkoppelt. Die FGS-Pipeline wird durch drei Generationen iterativer Diagnose-und-Reparatur entwickelt; das finale FGS-Design kombiniert nachfrageorientierte GATv2-Kommunikation mit einem faktorierten Nachbarschaftskritiker ($O(d_c)$ unabhängig von N), der SAC- und PPO-Training im größten evaluierten Szenario ermöglicht. Das Framework ist auf Basis von SUMO, SUMO-RL und RLlib nach dem CTDE-Paradigma implementiert.

Alle sechs deutschen RESCO-Benchmark-Szenarien in drei Skalierungsstufen – Einzelkreuzung (cologne1, ingolstadt1), Korridor (cologne3, ingolstadt7) und Regional (cologne8, ingolstadt21) – werden gegen sieben Basisverfahren evaluiert: Festzeitsteuerung, Maximaldruck, IDQN, IPPO, unabhängiges SAC, FRAP und CoLight.

Eine komponentenweise Ablation über alle acht Entwurfsraumkombinationen liefert drei Entwurfsrichtlinien. Erstens übertreffen MLP-Kodierer FRAP-basierte Kodierer auf deutschen Kreuzungen um bis zu $2,6\times$ in mittlerer Verzögerung im Regionalmaßstab ($N = 21$), da FRAPs Phasenwettbewerbs-Annahmen strukturell mit deutschen variablen Phasenstrukturen inkompatibel sind. Zweitens reduziert GATv2-Aufmerksamkeit die mittlere Verzögerung im Regionalmaßstab ($N = 21$) um 52 % gegenüber GAT (234,9 s vs. 492,0 s), bietet jedoch bei kleinerem Maßstab keinen messbaren Vorteil. Drittens erzielt SAC eine 8 % niedrigere mittlere Verzögerung als PPO bei moderatem Maßstab auf cologne8 (22,2 s vs. 24,0 s), erfordert jedoch den faktorierten Kritiker von FGSv3, um das Training bei $N = 21$ abzuschließen.

FGSv3-PPO erreicht 68,4 s mittlere Verzögerung auf ingolstadt21, vergleichbar mit dem unabhängigen PPO-Basisverfahren (67,2 s), und übertrifft CoLight (216,8 s, -68%) sowie DQN (78,7 s, -13%) deutlich. FGS(MLP+GATv2+SAC) erzielt 22,2 s mittlere Verzögerung auf cologne8 und übertrifft CoLight (51,8 s) um 57 %. Diese Ergebnisse belegen, dass die FGS-Pipeline ein effektives und analytisch aufschlussreiches Framework für die Charakterisierung von MARL-TSC-Entwurfsentscheidungen auf deutschen Straßennetzen darstellt. Alle berichteten Werte basieren auf Einzelläufen (Seed 0) mit bester Validierungsprüfung; eine statistische Mehrfachsamens-Validierung ist als vorrangige Folgearbeit identifiziert.

Schlüsselwörter: Multi-Agenten Reinforcement Learning, Verkehrssignalsteuerung, Graphaufmerksamkeitsnetzwerk, Soft Actor-Critic, FRAP-Kodierer, RESCO-Benchmark, dezentralisierte Ausführung, deutsche Verkehrsszenarien

Contents

Declaration	1
Acknowledgements	2
Abstract	3
Kurzfassung	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Research Questions and Hypotheses	2
1.5 Scope and Limitations	3
1.5.1 Scope	3
1.5.2 Limitations	3
1.6 Thesis Structure	4
2 Literature Review	5
2.1 Traffic Signal Control Approaches	5
2.1.1 Classical Methods	5
2.1.2 Reinforcement Learning for Single-Agent TSC	5
2.1.3 Multi-Agent Reinforcement Learning for TSC	6
2.2 Algorithmic Design Choices in RL for TSC	7
2.2.1 Model-Free and Model-Based Reinforcement Learning	7
2.2.2 Action Space Formulation	7
2.2.3 On-Policy and Off-Policy Learning	7
2.3 Graph Machine Learning for Traffic Networks	7
2.4 Representation Learning and Fairness in TSC	8
2.5 Research Gap and Positioning of This Thesis	8
3 Background and Preliminaries	10
3.1 Markov Decision Processes	10
3.1.1 The Sequential Decision-Making Problem	10
3.1.2 Partial Observability and Multi-Agent Extensions	11
3.2 From Dynamic Programming to Deep Reinforcement Learning	12

3.2.1	Dynamic Programming	12
3.2.2	Temporal-Difference Learning and Q-Learning	12
3.2.3	Policy Gradient Methods	12
3.2.4	Actor-Critic Methods and the Advantage Function	13
3.2.5	Entropy-Regularized Reinforcement Learning	13
3.3	Graph Neural Networks	14
3.3.1	Why Graphs for Multi-Agent Traffic Control	14
3.3.2	Message Passing Framework	14
3.3.3	Graph Convolutional Networks	14
3.3.4	Graph Attention Networks	15
3.3.5	GATv2: Dynamic Attention	15
3.4	Traffic Signal Control Theory	15
3.4.1	Movements and Phases	15
3.4.2	Phase Timing and Transitions	16
3.4.3	Classical vs. Adaptive Control	17
4	Methodology	18
4.1	Problem Formulation	18
4.2	The Three-Module Framework	18
4.3	Module I: Local Encoder	19
4.3.1	Observation Space	19
4.3.2	MLP Encoder	20
4.3.3	FRAP Encoder	20
4.4	Module II: Graph Communication	22
4.4.1	Agent Communication Graph	22
4.4.2	GATv2 Attention	22
4.5	Module III: RL Optimizer	22
4.5.1	Action Space	23
4.5.2	Reward Function	23
4.5.3	DQN and PPO Baselines	24
4.5.4	Soft Actor-Critic	25
4.6	FGS Architecture	25
4.6.1	Pipeline Architecture	25
4.6.2	Training Algorithm	27
5	Experimental Design	30
5.1	Simulation Scenarios	30
5.1.1	TapasCologne and InTAS Demand Models	30
5.1.2	SUMO and TraCI	30
5.2	Evaluation Setup	31
5.3	Evaluation Metrics	31
5.4	Hyperparameter Configuration	32
5.5	System Architecture	33

6	Results and Discussion	34
6.1	Main Results	34
6.2	Scale-Dependent Performance	34
6.3	Baseline Comparison	37
6.4	Component Analysis	38
6.4.1	Encoder type: FRAP versus MLP	38
6.4.2	Graph-attention mechanism: GATv2 versus GAT	39
6.4.3	RL algorithm: SAC versus PPO	41
6.5	Training Convergence	43
6.6	Design Evolution	43
6.7	Answering the Research Questions	44
6.8	Limitations	46
7	Conclusion and Future Work	48
7.1	Summary	48
7.2	Contributions	49
7.2.1	Framework Contribution	49
7.2.2	Algorithmic Contribution	49
7.2.3	Empirical Contribution	50
7.2.4	Analytical Contribution	50
7.3	Future Research Directions	50
A	Mathematical Objectives, Gradients, and Stability Analysis	52
A.1	Notation and CTDE Objective	52
A.2	Differential Waiting-Time Reward: Telescoping Property	52
A.3	Masked Discrete Action Distribution	53
A.4	Discrete SAC: Losses and Gradients	53
A.4.1	Critic Loss and Gradient	53
A.4.2	Actor Loss and Softmax-Logit Gradient	54
A.4.3	Temperature Objective and Log-Parameterization	54
A.5	PPO: Generalized Advantage Estimation and Joint Loss	54
A.6	GATv2: Edge Weights and Permutation Equivariance	55
A.6.1	Edge-Weight Integration	55
A.6.2	Node-Permutation Equivariance	55
A.7	FRAP / FGSv2: Phase-Permutation Equivariance	55
A.8	Centralized Critic: Input-Dimension Scaling	55
A.9	FGSv3: Factored Critic Input-Dimension Bound	56
A.10	FGSv3: Demand Communication Branch Equivariance	57
A.11	Soft Bellman Contraction and Policy Improvement	57
A.11.1	Soft Bellman Contraction	57
A.11.2	Soft Policy Improvement	58
A.11.3	Convergence Disclaimer	58
A.12	Agent Communication Graph Extraction	58
A.13	Statistical Notes on Single-Seed Results	59

A.14 First-Generation FGS and FGSv2 Forward Passes	59
B Full Experimental Results	61
B.1 Training Convergence Curves	61
List of Figures	67
List of Tables	68
Bibliography	69

Chapter 1

Introduction

1.1 Background and Motivation

Urban traffic congestion is among the most persistent inefficiencies in modern infrastructure. In cities worldwide, the time lost to congestion increases vehicle emissions, raises logistics costs, and degrades quality of life for millions of daily commuters. Germany, with one of the highest vehicle densities in Europe, experiences significant peak-hour delays in its major urban centers; Cologne and Ingolstadt — the two cities whose road networks are evaluated in this thesis — are representative of the medium-density European urban environment where signal timing directly determines the throughput and fairness of the road network.

Traffic signal control (TSC) is the primary lever through which urban traffic is managed at the intersection level. A well-timed signal plan reduces average waiting time, balances queue lengths across directions, and enables coordinated flow along arterials. Conversely, poorly timed signals generate spillback — queues that extend upstream and block intersections they do not serve — which can cascade into network-wide congestion. The challenge is that optimal signal timing depends on real-time demand conditions that are neither stationary nor predictable from historical data alone. Fixed-time plans, designed for average demand, leave substantial throughput gains on the table during off-peak periods and fail under demand spikes caused by incidents or events.

The response to these limitations has followed two broad tracks. The first track, represented by adaptive control systems such as SCOOT [16] and SCATS [23], augments traditional signal engineering with real-time feedback from loop detectors. These systems have demonstrated meaningful improvements in arterial delay but require dense sensing infrastructure, expert parameter calibration, and remain constrained by pre-defined phase sequences. The second track, initiated by early reinforcement learning (RL) work [38] and greatly expanded by deep RL [35, 37], treats phase selection as a sequential decision problem that an agent can learn to solve from simulation experience. This approach requires no explicit traffic model and, in principle, adapts to any demand pattern within the distribution encountered during training.

The extension from single-intersection RL to multi-agent RL (MARL) for network-level control introduces a further challenge: agents must not only learn individually effective phase policies, but also produce collectively coherent behavior that reduces network-wide delay. Independent agents, each optimizing its own local reward, ignore the upstream and downstream consequences of their decisions [7, 2]. Graph-based communication — where each intersection aggregates information from its neighbors over the road graph — provides a mechanism for coupling agent decisions without requiring centralized coordination at execution time [36].

1.2 Problem Statement

Despite significant progress in RL-based TSC, three practical gaps remain unaddressed in the published literature.

First, the dominant evaluation environments in MARL-TSC research are synthetic grid networks or Chinese urban scenarios (Hangzhou, Jinan). The German RESCO benchmark [2] provides real-world demand models for Cologne and Ingolstadt and has been adopted as a standard evaluation protocol, but few MARL-TSC methods — particularly those using graph-based communication — have been evaluated on it. Whether coordination mechanisms that work on regular Chinese grids transfer to the irregular topologies and heterogeneous phase structures of German networks is an open empirical question.

Second, the RL algorithms used in MARL-TSC are almost exclusively value-based (DQN and its variants) or on-policy (PPO). Soft Actor-Critic [13], which offers off-policy sample efficiency and entropy-regularized exploration, has not been systematically evaluated in a graph-aware multi-agent setting for traffic control. SAC’s entropy bonus may be particularly beneficial in multi-agent environments, where diverse policies reduce the risk that agents converge to coordination patterns that perform well in training scenarios but fail under distribution shift.

Third, the design space of MARL-TSC architectures — encoder type, communication mechanism, and RL algorithm — has not been systematically mapped or ablated on German road networks. Whether phase-aware encoding complements graph attention, whether entropy regularization helps or hinders convergence in multi-agent German scenarios, and at what network scales each component provides consistent benefit are open empirical questions. No published method treats these three dimensions as independently configurable choices and evaluates their interactions on the German RESCO benchmark.

This thesis addresses all three gaps through the FGS (Framework for Graph-aware Signal control): a modular three-module pipeline that decouples encoder, graph communication, and RL algorithm as independent design choices. The FGS architecture is developed across three successive generations and evaluated on six German RESCO scenarios spanning three scale tiers: Single-intersection (cologne1, ingolstadt1), Corridor (cologne3, ingolstadt7), and Regional (cologne8, ingolstadt21).

1.3 Research Objectives

The work reported in this thesis pursues five interconnected objectives:

1. **Survey existing TSC approaches.** Review classical, RL-based, and MARL-based methods to identify the state of the art and the limitations that motivate graph-aware, entropy-regularized control.
2. **Study local feature representation.** Investigate the effect of local encoder design — specifically MLP encoders versus the FRAP phase-competition encoder — on agent performance and learning dynamics.
3. **Study graph-based inter-agent communication.** Analyse whether GATv2-based message passing between neighboring intersections improves coordination, and whether the benefit depends on network size and topology.
4. **Study RL optimization schemes.** Compare discrete-action SAC, DQN, and PPO as RL algorithms within the same MARL-TSC framework, examining sample efficiency, stability, and final performance.
5. **Evaluate across German RESCO scenarios.** Assess the FGS pipeline (and first-generation ablation variants) against classical and deep RL baselines on the German RESCO benchmark, characterizing performance, scalability, and the contribution of individual architectural choices. The FGS architecture is developed iteratively across three generations, with each generation targeting a specific interface failure identified in evaluation of the preceding one.

1.4 Research Questions and Hypotheses

The empirical investigation is organized around one overarching main research question and three focused sub-questions that operationalize it.

Main Research Question (MRQ):

Can a modular three-module MARL pipeline — combining a local encoder, graph-attention communication, and an RL algorithm as independently configurable choices — provide competitive and scalable traffic signal control on real-world German road networks, and which design dimensions are the primary determinants of performance across network scales from single intersections to 21-agent regional clusters?

The MRQ is operationalized through three sub-questions. Each sub-question addresses a distinct epistemic dimension: external validity (SQ1), internal design analysis (SQ2), and boundary conditions across scale (SQ3).

SQ1 (Baseline comparison): *How do FGS pipeline instantiations compare in performance with classical traffic-control methods and deep RL baselines across the three German RESCO scale tiers?*

SQ2 (*Design-space analysis*): How do encoder choice (MLP vs. FRAP) and graph-attention mechanism (GAT vs. GATv2) affect traffic signal control performance and training stability within the FGS pipeline?

SQ3 (*Scale and topology dependency*): How does the benefit of each design component vary with network scale and topology, from Single-intersection to Regional ($N = 21$) scenarios?

Four hypotheses guide the experimental design; they are stated as testable propositions, not guarantees. H1 and H4 inform SQ3; H2 and H3 inform SQ2; all four bear collectively on the MRQ.

H1: *Graph-aware communication improves coordination between neighboring intersections compared with independent-agent methods, as reflected in lower network-level mean travel time and waiting time.*

H2: *Entropy-regularized SAC produces more stable training curves than purely value-based methods (DQN) in selected multi-agent German RESCO scenarios.* This hypothesis is framed as an investigation: SAC may not outperform DQN in all scenarios, and the conditions under which entropy regularization is beneficial are themselves of interest.

H3: *Phase-aware local encoders such as FRAP do not necessarily improve performance in all German RESCO scenarios.* FRAP’s phase-competition structure encodes assumptions about movement representation and phase pairing that may not align uniformly with every network topology, making its relative contribution an empirical rather than a guaranteed finding.

H4: *Larger and more complex networks reveal stronger performance differences between independent-agent and graph-aware methods than small single-intersection scenarios.*

1.5 Scope and Limitations

1.5.1 Scope

This thesis focuses on the design, implementation, and empirical evaluation of a graph-aware MARL framework for signalized intersection control. The scope encompasses:

- Control algorithm design: the FGS architecture and its first-generation ablation variants described in Chapter 4.
- Algorithm analysis: ablation across encoder types, communication mechanisms, and RL algorithms.
- Empirical comparison: six German RESCO scenarios across three scale tiers — **Single-intersection** (cologne1, ingolstadt1), **Corridor** (cologne3, ingolstadt7), and **Regional** (cologne8, ingolstadt21) — against a standardized set of classical and deep RL baselines. This tier classification is used consistently throughout the thesis. FGSv3 was not evaluated on Corridor-tier scenarios; first-generation FGS_p results are available for ingolstadt7 and included in the comparison tables.

1.5.2 Limitations

The following are explicitly outside the scope of this work:

- **Pedestrian and cyclist modeling.** The SUMO environments used do not simulate pedestrian or cyclist movements; the control policy addresses vehicle phase selection only.
- **Simulation environment.** All experiments are conducted in SUMO [21] via the TraCI interface. Real-world deployment, with its sensor noise, communication latency, and physical actuator constraints, is not addressed.
- **Scenario construction.** Traffic network files, demand routes, and signal programmes are taken from the RESCO benchmark release without modification. No new traffic scenarios are constructed for this thesis.
- **RL library development.** The thesis does not implement an RL framework from scratch. The implementation builds on RLlib for distributed training, PyTorch Geometric for graph operations, and the SUMO-RL library [1] for environment wrapping, each extended with custom modules where required.¹ Earlier exploratory frameworks considered during development include the original RESCO evaluation scripts, CityFlow, LibSignal, and PyTSC; these were not adopted in the final implementation.

¹The full implementation is available at <https://github.com/fuisl/sumo-rl>.

- **Reward design.** The reward function is fixed as the differential waiting time (Section 4.5.2.1). Systematic reward shaping or fairness-constrained reward design is not investigated.
- **Continuous action spaces.** The framework operates in discrete phase-selection action spaces throughout; continuous duration assignment is not explored.

1.6 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 surveys existing TSC approaches from fixed-time control through single-agent and multi-agent RL, reviews graph neural network variants relevant to inter-agent communication, and identifies the research gap that this thesis addresses.

Chapter 3 establishes the mathematical and conceptual foundations: Markov Decision Processes, partially observable and multi-agent extensions, the progression from dynamic programming to deep RL and entropy-regularized actor-critic methods, graph neural network message passing, and the traffic engineering concepts required to interpret the experiments.

Chapter 4 describes the FGS framework in detail: observation space, action space, graph construction, baseline algorithms, reward function, and the full FGS pipeline architecture.

Chapter 5 specifies the experimental setup: the TapasCologne and InTAS demand models, SUMO simulation parameters, evaluation metrics and protocols, training hyperparameters, and the end-to-end software pipeline.

Chapter 6 presents and interprets the experimental results: main performance comparisons, ablation across architectural variants, cross-scenario analysis, and answers to the three sub-questions and the main research question.

Chapter 7 summarizes the contributions, reflects on the hypotheses in light of the experimental findings, and identifies directions for future work.

Chapter 2

Literature Review

Traffic signal control sits at the intersection of transportation engineering, optimization theory, and machine learning. This chapter traces the development of control methods from classical analytical approaches through single-agent reinforcement learning to modern multi-agent graph-based systems, identifying at each stage the limitations that motivated the next generation of work. Section 2.1 surveys TSC methods directly; Section 2.2 discusses the algorithmic design axes that differentiate them; Section 2.3 examines graph-based learning as an architectural building block; Section 2.4 reviews representation learning and fairness in the TSC context; and Section 2.5 states the research gap addressed by this thesis.

2.1 Traffic Signal Control Approaches

2.1.1 Classical Methods

The earliest principled approach to signal timing is Webster’s analytical formula [34], which derives the optimal cycle length that minimizes average vehicle delay under the assumption of stationary, Poisson-distributed arrival rates. Webster’s method remains the foundation of fixed-time signal plans deployed worldwide: the cycle length, phase splits, and offsets are computed offline from historical counts and applied without feedback. Under the demand conditions for which they were calibrated, fixed-time plans perform well; the limitation is that real traffic demand is neither stationary nor fully predictable [16, 23]. Incidents, events, and daily variability all cause the actual demand to deviate from the design scenario, and a fixed-time plan has no mechanism to adapt.

Vehicle-actuated control partially addresses this by extending green phases when loop detectors sense arriving vehicles and terminating phases early when queues are absent. More sophisticated adaptive systems such as SCOOT [16] and SCATS [23] estimate real-time occupancy from detector data and continuously adjust timing plans across a network. Both systems have been deployed at city scale and have demonstrated meaningful reductions in average delay compared to fixed-time control. However, they rely on dense detector infrastructure, require expert parameter tuning for each deployment, and optimize within a fixed phase sequence — they cannot discover phase orderings or combinations outside the hand-crafted programme.

Max pressure control [31] takes a different approach: rather than optimizing timing within a fixed plan, it selects at each step the phase that maximizes the difference between vehicle counts on incoming and outgoing lanes, a quantity called *pressure*. This policy is provably throughput-optimal under mild assumptions — it maximizes the set of demand rates for which the network remains stable, and requires no pre-calibration. However, max pressure minimizes global queue imbalance rather than individual vehicle waiting time, and its greedy single-step decisions can accumulate unnecessary delay when a suboptimal phase is forced by the pressure criterion. Both of these limitations are relevant to the German RESCO scenarios evaluated in this thesis.

2.1.2 Reinforcement Learning for Single-Agent TSC

The application of reinforcement learning to traffic signal control was first explored systematically by Wiering [38], who showed that tabular Q-learning agents can discover effective phase policies in small synthetic grids without any prior model of traffic dynamics. With the emergence of deep neural networks as function approximators, DQN-based controllers became the dominant approach. Wei et al. [35] demonstrated that a DQN agent trained on real-world demand data from Hangzhou could match or exceed fixed-time control in terms of average travel time, establishing RL as a viable

alternative rather than a theoretical curiosity. A key insight from this and subsequent work is that the choice of reward signal and state representation has a larger effect on performance than the specific RL algorithm: poorly designed states with irrelevant features or rewards that fail to capture network-level objectives consistently underperform even simple adaptive controllers.

Wei et al. [37] made this insight explicit by deriving both the state and reward from the max pressure theory. PressLight represents the intersection state as the pressure differential between incoming and outgoing lanes — the same quantity that max pressure control maximizes — and uses this as the reward signal. By grounding the RL problem in a quantity with known theoretical properties (throughput optimality), PressLight achieves more stable training and better generalization than heuristic state/reward designs. The result also illustrates a broader principle in the TSC literature: RL methods that incorporate domain knowledge, either in the reward function or in the state representation, consistently outperform those that treat the problem as a generic control task.

Single-agent RL is, however, limited to isolated intersection control [36]. Applying independent single-agent policies to each intersection in a network ignores the spatial dependencies between adjacent signals: the queue at one intersection is a direct consequence of the phase decisions at its upstream neighbors. Attempting to capture these dependencies through a larger observation window (including neighbor states in the input) inflates the observation space without providing a mechanism for agents to explicitly coordinate their decisions.

2.1.3 Multi-Agent Reinforcement Learning for TSC

The natural extension to network-level control is to treat each intersection as an independent RL agent and allow agents to share information. Naive independent learning — where each agent optimizes its own reward while ignoring other agents — suffers from the non-stationarity problem: from any single agent’s perspective, the environment appears to change as other agents update their policies during training, violating the Markov assumption and making convergence difficult to guarantee. Despite this, independent DQN and PPO baselines evaluated in the RESCO benchmark [2] often perform competitively, suggesting that the local reward signal provides sufficient learning signal even without explicit coordination.

The most influential graph-based approach to MARL-TSC is CoLight [36], which uses graph attention networks to aggregate information from neighboring intersections before the agent selects its next phase. Each intersection encodes its local state into an embedding, and GAT attention coefficients [32] weight the contributions of neighbors according to the similarity of their embeddings to the query node. Experiments on both synthetic grids and real-world Hangzhou road networks demonstrate that graph-attention communication consistently reduces average travel time compared to independent agents, with larger improvements on denser networks where coordination pressure is higher. CoLight established the pattern — local encoder, graph communication, shared parameters — that subsequent methods including FGS follow.

The FRAP encoder [41] contributes a different dimension: instead of improving coordination, it improves the local feature representation by making the phase competition structure explicit. Rather than encoding the intersection state as a flat vector, FRAP models pairwise competition between candidate phases, embedding the demand on each movement and computing how much each phase is favoured relative to its competitors. This encoder is permutation-equivariant with respect to phase labels and generalizes across intersections with different phase configurations. Applied to DQN, FRAP achieves faster convergence and better final performance than MLP encoders, particularly in scenarios where phase counts vary across the network.

MPLight [7] scales multi-agent pressure-based control to networks of up to one thousand intersections by combining the PressLight state/reward design with decentralized multi-agent training. The shared-parameter architecture keeps the per-intersection parameter count constant regardless of network size, and the pressure-based reward ensures that local objectives align with global throughput. MPLight demonstrates that decentralized MARL with a well-designed local objective can coordinate large networks without explicit inter-agent communication, though at the cost of the coordination advantages that communication-enabled methods such as CoLight provide.

The RESCO benchmark [2] standardizes comparison across these methods by providing a common set of evaluation scenarios, including the German traffic networks from TapasCologne [10] and InTAS [20] used in this thesis, along with fixed experimental protocols for episode length, seed averaging, and metric computation. RESCO reveals that no single method dominates across all scenarios: max pressure performs strongly on large Ingolstadt networks where global throughput is the primary objective, while RL methods with rich local representations perform better on smaller, more congestion-prone Cologne networks. This scenario-dependence motivates studying how architectural choices — encoder design, communication mechanism, RL algorithm — interact with network topology and size.

2.2 Algorithmic Design Choices in RL for TSC

2.2.1 Model-Free and Model-Based Reinforcement Learning

Model-based RL methods learn an explicit model of the environment transition dynamics and use it to plan or generate synthetic experience. In principle, a good model enables far more sample-efficient learning because the agent can reason about consequences without executing actions in the real environment. In practice, traffic networks present severe challenges for model learning: the dynamics depend on tens of thousands of vehicles interacting through complex car-following and lane-changing behaviors, the state space is effectively continuous and very high-dimensional, and the demand is non-stationary across the day and week. No tractable dynamics model has yet been demonstrated to generalize across the German RESCO scenarios at the level of fidelity required for planning-based control.

Model-free methods, which learn directly from environment interactions without building an explicit dynamics model, have therefore dominated the TSC literature. Their primary disadvantage is sample efficiency: each gradient update requires real environment transitions from the SUMO simulator, which are computationally expensive to generate. Off-policy methods — which store transitions in a replay buffer and reuse them for multiple updates — partially compensate for this by decoupling data collection from policy optimization. SAC [13] is particularly well-suited to this regime: its maximum entropy objective prevents the policy from becoming overconfident early in training, and its off-policy data reuse makes each simulation episode more informative.

2.2.2 Action Space Formulation

Two action formulations dominate the RL-TSC literature. The first, and most common, is *phase selection*: the agent chooses the next phase to activate from the set of predefined phases in the intersection’s signal programme. This is a discrete action space with size equal to the number of candidate phases, typically between 2 and 8 for real-world intersections. Phase selection is easy to interpret, aligns with how physical signal controllers operate, and maps naturally to softmax-based actor architectures.

The second formulation is *duration assignment*: the agent selects how long the current phase should remain active, producing a continuous or discretized action over green durations. Duration assignment offers finer-grained control but requires the agent to reason about timing in absolute terms rather than relative urgency, which can slow learning. The RESCO benchmark and the majority of recent MARL-TSC work use phase selection, and all methods compared in this thesis — including DQN, PPO, independent SAC, and FGS/FGSv2 — operate in the discrete phase-selection action space.

2.2.3 On-Policy and Off-Policy Learning

On-policy methods such as PPO [28] require that the data used for each gradient update was collected under the current policy. This guarantees that the policy improvement is grounded in the agent’s actual behavior distribution, but it also means that collected experience cannot be reused: after each policy update, all stored transitions are discarded. In TSC, where each training episode requires running a full 3600-second SUMO simulation, discarding experience after each update is costly. PPO mitigates this with multiple epochs of mini-batch updates over the same rollout batch, but the fundamental constraint remains.

Off-policy methods decouple data collection from policy optimization by storing transitions in a replay buffer and sampling arbitrary past experience for updates. DQN and SAC are both off-policy, and the prioritized experience replay [27] used by both in this thesis further improves data efficiency by sampling transitions in proportion to their temporal-difference error, focusing learning on the transitions that are currently most informative. The key trade-off is stability: because the replay buffer may contain transitions from outdated policies, off-policy methods can be prone to divergence if the policy changes too rapidly [30]. SAC’s entropy regularization implicitly stabilizes training by preventing sudden policy collapses [13], which is one reason it is chosen as the core algorithm for FGS.

2.3 Graph Machine Learning for Traffic Networks

Road networks are canonical examples of spatial graphs, and graph-based machine learning has been applied to traffic problems long before its adoption in RL-TSC. Early work applied Graph Convolutional Networks [17] to traffic flow forecasting [40]: predicting future speeds or volumes at sensors across a network by propagating historical observations along the road graph. These applications demonstrated that GCN-style message passing captures spatial

correlations more effectively than treating each sensor independently or using a global mean aggregation. The fixed, degree-normalized aggregation of GCN is, however, suboptimal when neighbors have very different relevance to the prediction — a property that varies dynamically with traffic conditions [32].

Graph Attention Networks [32] replace fixed aggregation weights with content-dependent attention coefficients, allowing the model to focus on the most informative neighbors at each time step. CoLight [36] applied this idea to RL-TSC and showed that attention-weighted communication reduces average travel time more than fixed-weight communication. As established in Section 3.3.5, however, GAT’s attention scores are a static function of projected features — the ranking of neighbors cannot change depending on the target node’s own state [5]. GATv2 resolves this limitation via a non-linearity inserted before the attention scoring, making coefficients depend jointly on both source and target features. FGSv2 adopts GATv2 as its communication layer for this expressiveness advantage: in heterogeneous traffic networks where congestion levels differ sharply across intersections, target-dependent attention is expected to enable more selective message passing.

2.4 Representation Learning and Fairness in TSC

The choice of how to encode the local intersection state — the observation representation — has proven to be one of the highest-leverage design decisions in RL-TSC. Generic MLP encoders treat the observation as an unstructured vector and rely on the network to discover relevant features from scratch. Domain-informed encoders instead build known structure into the architecture, reducing the number of samples needed to reach good performance.

The most influential domain-informed encoder is FRAP [41], which encodes the intersection state through the lens of phase competition. The key insight is that the controller’s decision at each step reduces to a relative comparison: which candidate phase has the most urgent demand relative to the movements it preempts? FRAP makes this comparison explicit by computing pairwise competition scores between all candidate phases, weighted by the current demand on the movements each phase controls. The encoder is permutation-equivariant: relabelling the phases does not change the output, which means a model trained on one intersection generalizes to another with a different phase numbering convention. On synthetic grid networks, FRAP consistently achieves faster convergence and higher final performance than MLP-based encoders when used with DQN [41].

Whether FRAP’s advantages persist when the encoder is used within a graph communication framework — where the local embedding is subsequently aggregated with neighbor messages — is not established in the literature. The FGS framework investigates this directly: Section 4.6 describes how both MLP and FRAP encoders are combined with GAT communication within a unified SAC training framework, and the experimental ablation in Chapter 6 quantifies the contribution of each component.

Fairness in TSC — the question of whether green time is distributed equitably across all movements and intersections — has received comparatively little attention in the RL literature. Most methods optimize aggregate metrics such as average travel time or total throughput, which can be dominated by high-demand movements at the expense of low-demand ones. The differential waiting time reward used in this thesis (Equation 4.12) has an implicit fairness property: each agent is rewarded for reducing waiting time at its own intersection regardless of how busy that intersection is relative to others, preventing any single intersection from being systematically neglected during training. Entropy regularization in SAC has a complementary effect: by maintaining policy diversity across agents, it prevents all agents from converging to identical strategies that may benefit the busiest intersections at the cost of quieter ones. Quantifying these implicit fairness effects is outside the scope of this thesis; it is identified as a direction for future work in Section 7.3.

2.5 Research Gap and Positioning of This Thesis

Three gaps in the existing literature motivate the contributions of this thesis.

German RESCO scenarios are underexplored with modern MARL. The RESCO benchmark [2] was introduced specifically to provide standardized German traffic scenarios for fair method comparison, but the majority of MARL-TSC papers — including CoLight [36], FRAP [41], and MPLight [7] — evaluate exclusively on Chinese or synthetic grid networks. The German scenarios differ structurally from these: the Ingolstadt networks have irregular topologies with varying numbers of phases per intersection, and the Cologne demand model reflects European driving patterns. Whether coordination mechanisms that help on regular Chinese grids transfer to these more heterogeneous settings is an open empirical question.

SAC has not been systematically evaluated in graph-aware MARL-TSC. All major MARL-TSC methods reviewed above use either DQN (off-policy, discrete, value-based) or PPO (on-policy, policy gradient). SAC offers a different profile: off-policy sample efficiency, entropy-regularized exploration, and actor-critic separation that allows the actor and critic to be designed independently. In multi-agent settings, SAC’s entropy bonus may produce more diverse joint policies than DQN’s epsilon-greedy exploration, which could be beneficial when coordination strategies differ across network regions. No published work evaluates SAC with graph-based inter-agent communication on any RESCO scenario.

FRAP and graph communication have not been combined in a principled framework. CoLight uses GAT communication but with a generic MLP encoder. FRAP uses a phase-competition encoder but with independent-agent DQN and no inter-agent communication. The FGS framework proposed in this thesis combines both: a phase-aware encoder (either MLP or FRAP) with graph attention communication and off-policy SAC training, evaluated on German RESCO scenarios. FGSv2 extends this by replacing the single node embedding with per-action tokens, connecting FRAP’s phase-competition representation directly to the actor’s action-selection head. FGSv3 further refines the communication signal to carry demand state rather than competition scores and introduces a factored neighborhood critic that enables training at $N = 21$. Table 2.1 places these contributions in the context of the methods reviewed above.

Table 2.1: Comparison of related MARL-TSC methods along key design dimensions. “Ger.” = evaluated on German RESCO scenarios.

Method	RL algo	Encoder	Graph comm.	Ger.	Reference
Max pressure	–	pressure	–	☒	[31]
IDQN	DQN	MLP	–	☒	[24]
IPPO	PPO	MLP	–	☒	[28]
IndSAC	SAC	MLP	–	☒	[8]
PressLight	DQN	pressure	–	–	[37]
FRAP	DQN	FRAP	–	–	[41]
CoLight	DQN	MLP	GAT	–	[36]
MPLight	DQN	pressure	–	–	[7]
FGS (this thesis)	SAC/PPO	MLP/FRAP	GATv2	☒	–
FGSv2 (this thesis)	SAC	FRAP-token	GATv2	☒	–
FGSv3 (this thesis)	SAC/PPO	FRAP-token	GATv2(D)	☒	–

Chapter 3

Background and Preliminaries

This chapter builds the mathematical and conceptual foundations on which the FGS framework rests. Section 3.1 formalizes the sequential decision-making problem as a Markov Decision Process and extends it to the partially observable, multi-agent setting that characterizes real traffic networks. Section 3.2 traces the development from dynamic programming to modern deep reinforcement learning, covering the key algorithmic ideas that motivate the SAC algorithm used in this thesis. Section 3.3 introduces graph neural networks and explains why attention-based message passing is a natural fit for inter-intersection communication. Section 3.4 defines the traffic engineering concepts required to interpret the experimental scenarios and the FRAP encoder.

3.1 Markov Decision Processes

3.1.1 The Sequential Decision-Making Problem

A reinforcement learning agent is an entity that repeatedly selects actions and receives feedback from an environment. The agent’s objective is to accumulate as much reward as possible over time, but it cannot see the consequences of an action before taking it. This tension between exploration (gathering information) and exploitation (acting on current knowledge) is the defining characteristic of the reinforcement learning problem [30].

The standard mathematical framework for this interaction is the *Markov Decision Process* (MDP), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \gamma \rangle$:

- $\mathcal{S}$: the *state space*, representing all possible configurations of the environment.
- $\mathcal{A}$: the *action space*, the set of choices available to the agent.
- $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$: the *transition function*, where $T(s' | s, a)$ is the probability of moving to state s' after taking action a in state s .
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: the *reward function*, specifying the immediate payoff for each state-action pair.
- $\gamma \in [0, 1)$: the *discount factor*, which controls how much the agent values future rewards relative to immediate ones.

The Markov property — the requirement that $T(s' | s, a)$ depends only on the current state and action, not on the history — is what makes this framework tractable. It guarantees that the current state contains all information needed to make an optimal decision.

Returns and value functions. At each time step t the agent selects action a_t according to policy π , receives reward r_t , and transitions to s_{t+1} . The full sequence of interactions $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ generated by following π from an initial state is called a *trajectory*. The *discounted return* from step t onward is the cumulative reward accumulated along the trajectory:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (3.1)$$

The discount factor $\gamma < 1$ ensures G_t is finite for bounded rewards and encodes a preference for sooner rewards over later ones.

Two value functions characterize how good a given situation is under policy π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \quad (3.2)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]. \quad (3.3)$$

$V^\pi(s)$ is the expected return when starting in state s and following π thereafter; $Q^\pi(s, a)$ additionally conditions on taking action a first. The two are related by $V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$.

Bellman equations. The recursive structure of the return gives rise to the Bellman equations, which express the value of a state in terms of the values of its successors:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) [R(s, a) + \gamma V^\pi(s')], \quad (3.4)$$

$$Q^\pi(s, a) = \sum_{s'} T(s'|s, a) [R(s, a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a')]. \quad (3.5)$$

These equations hold because the Markov property allows the expected future return to be factored at the transition boundary. The optimal value functions V^* and Q^* satisfy the Bellman *optimality* equations, obtained by replacing π with the greedy maximization over actions:

$$Q^*(s, a) = \sum_{s'} T(s'|s, a) [R(s, a) + \gamma \max_{a'} Q^*(s', a')]. \quad (3.6)$$

The optimal policy is then $\pi^*(s) = \arg \max_a Q^*(s, a)$: always select the action with the highest long-term value.

3.1.2 Partial Observability and Multi-Agent Extensions

The MDP framework assumes the agent observes the full state s_t . In most real systems, including traffic networks, the agent can only access a partial, noisy view of the world. A traffic signal controller at one intersection sees its own lane queues and phase state but has no direct knowledge of vehicle positions elsewhere in the network. This motivates the *Partially Observable MDP* (POMDP).

A POMDP extends the MDP tuple with an observation space Ω and an observation function $\mathcal{O} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\Omega)$, where $\Delta(\Omega)$ is the set of distributions over observations. At each step the agent receives observation $o_t \sim \mathcal{O}(\cdot \mid s_t, a_{t-1})$ rather than s_t directly. Because o_t does not satisfy the Markov property, the agent must in principle maintain a *belief state* $b_t \in \Delta(\mathcal{S})$ – a probability distribution over all possible world states – and update it via Bayes' rule at each step. Exact belief-state computation is intractable for large state spaces, so practical approaches either use recurrent architectures to implicitly summarize history or rely on the assumption that observations are informative enough to approximate the state.

Decentralized multi-agent setting. When $N > 1$ agents operate simultaneously in the same environment, each with its own partial observation, the problem becomes a *Decentralized Partially Observable MDP* (Dec-POMDP), defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, T, \{r_i\}, \gamma \rangle$ where $\mathcal{N} = \{1, \dots, N\}$ is the set of agents. Agents act simultaneously; the transition depends on the joint action $(a_1, \dots, a_N) \in \prod_i \mathcal{A}_i$; each agent i receives its own observation $o_i \sim \mathcal{O}_i(\cdot \mid s)$ and reward r_i .

Finding the optimal joint policy for a Dec-POMDP is NEXP-complete in general [3], because the space of joint policies grows exponentially in the number of agents. Two practical approaches have emerged:

1. *Fully decentralized execution*: each agent conditions its policy only on its own observation history, making the problem tractable but potentially limiting coordination.
2. *Centralized Training with Decentralized Execution* (CTDE): during training, agents have access to the global state or joint observations, enabling a centralized critic to guide learning. At execution, each agent acts solely on its local observation. This separates the hard coordination problem (handled offline during training) from the real-time execution problem (where full state is unavailable).

FGS adopts CTDE: a centralized joint-action critic is trained with access to all agents' graph embeddings, while the decentralized actor uses only agent k 's local observation. Throughout this thesis, the index k identifies the agent for which an action is currently being selected, as distinguished from its neighbors in the communication graph.

3.2 From Dynamic Programming to Deep Reinforcement Learning

3.2.1 Dynamic Programming

When the model (T, R) is known, the Bellman optimality equation (3.6) can be solved directly by *dynamic programming*. Value iteration repeatedly applies the Bellman operator

$$\mathcal{T}Q(s, a) = \sum_{s'} T(s'|s, a) [R(s, a) + \gamma \max_{a'} Q(s', a')], \quad (3.7)$$

initializing Q_0 arbitrarily and iterating $Q_{k+1} = \mathcal{T}Q_k$. Since $\mathcal{T}$ is a γ -contraction in the ℓ_∞ norm, this sequence converges to Q^* regardless of the initialization [30].

Dynamic programming is exact but impractical for large problems. A traffic network with hundreds of intersections, each with dozens of vehicles in different states, has a state space too large to enumerate. Two problems arise: *intractable transition models* (the physics of traffic is complex and non-stationary) and *the curse of dimensionality* (the number of states grows exponentially with the number of intersections). Reinforcement learning addresses both by replacing model knowledge with experience.

3.2.2 Temporal-Difference Learning and Q-Learning

Reinforcement learning removes the requirement that T and R be known. Instead of computing expectations over the full transition model, the agent updates estimates using *samples* of experience (s_t, a_t, r_t, s_{t+1}) .

Temporal-difference (TD) learning updates value estimates incrementally:

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[r_t + \gamma V(s_{t+1}) - V(s_t)]}_{\text{TD target}}, \quad (3.8)$$

where α is the learning rate. The quantity in brackets is the *TD error*: the difference between the current estimate and a better estimate formed using one step of real experience. Unlike Monte Carlo methods, which require a full episode to compute G_t , TD learning updates after every step, enabling online learning in continuing tasks.

Q-learning [33] extends TD learning to action values without requiring knowledge of T :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]. \quad (3.9)$$

The crucial property of Q-learning is *off-policy* convergence: because the update uses $\max_{a'}$ rather than the current policy, Q-learning converges to Q^* even when the behavior policy used to collect experience differs from the greedy policy being learned. This makes it compatible with experience replay, where past transitions are reused for multiple updates.

Deep Q-Networks. Tabular Q-learning is infeasible when $|\mathcal{S}|$ is large. DQN [24] approximates the Q-function with a neural network $Q_\theta(s, a)$ trained by minimizing the mean-squared Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a))^2 \right]. \quad (3.10)$$

Two stabilization techniques are essential [24]. First, a *replay buffer* $\mathcal{D}$ stores past transitions and samples mini-batches uniformly, breaking the temporal correlations in sequential experience that would otherwise cause divergence. Second, a *target network* $Q_{\bar{\theta}}$ with parameters $\bar{\theta}$ periodically copied from θ provides stable regression targets; without it, both the prediction and the target shift simultaneously, causing the loss landscape to be non-stationary and training to oscillate.

3.2.3 Policy Gradient Methods

Value-based methods such as DQN derive a policy implicitly by acting greedily with respect to the learned Q-function. This works well for discrete action spaces but is less natural for stochastic or continuous policies. *Policy gradient* methods instead parameterize the policy π_θ directly and optimize the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G_0], \quad (3.11)$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory sampled under π_θ . The policy gradient theorem [30] provides an analytical expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t]. \quad (3.12)$$

The term $\nabla_\theta \log \pi_\theta(a_t | s_t)$ is the *score function*: it indicates the direction in parameter space that increases the log-probability of the action taken. Multiplying by G_t scales this direction by how good the trajectory was, so good actions become more likely and bad actions less likely over time.

The simplest estimator of this gradient – the REINFORCE algorithm [39] – substitutes the full Monte Carlo return for G_t . This is unbiased but suffers from high variance: the return depends on all future random actions and transitions, so different runs of the same policy produce very different gradient estimates. High variance slows learning and can cause instability.

3.2.4 Actor-Critic Methods and the Advantage Function

Variance in policy gradient estimates can be reduced by subtracting a *baseline* $b(s_t)$ from the return. Any function that depends only on the state (not on the action) can serve as a baseline without introducing bias, because

$$\mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot b(s_t)] = 0. \quad (3.13)$$

The optimal baseline in terms of variance reduction is the state value $V^\pi(s_t)$. Substituting it gives the *advantage function*:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t), \quad (3.14)$$

which measures how much better action a_t is compared to the average action at state s_t under the current policy. The advantage is zero-centered: positive values indicate actions better than average, negative values indicate actions worse than average. Using A^π in place of G_t gives the gradient estimator:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) \cdot A^\pi(s_t, a_t)]. \quad (3.15)$$

In practice, both Q^π and V^π are unknown and must be estimated. The *actor-critic* architecture addresses this by maintaining two components: an *actor* π_θ that selects actions, and a *critic* V_ϕ that estimates the state value. The advantage is approximated using the one-step TD error of the critic:

$$\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad (3.16)$$

which trades a small amount of bias (from the bootstrapped value estimate) for a large reduction in variance compared to the full Monte Carlo return.

The Advantage Actor-Critic (A2C) [25] is the canonical instantiation of this architecture and serves as the conceptual precursor to the algorithms used in this thesis: PPO (Section 4.5.3.2) stabilizes actor updates via a clipped surrogate objective, and SAC (Section 4.5.4) extends the actor-critic structure to off-policy, entropy-regularized training.

3.2.5 Entropy-Regularized Reinforcement Learning

Standard RL optimizes the expected return $J(\theta) = \mathbb{E}[G_t]$. This objective is purely exploitative: in any finite MDP there always exists a deterministic optimal policy [30], so stochasticity in the learned policy reflects only irreducible uncertainty rather than genuine exploration. In practice, deterministic greedy policies tend to converge prematurely to suboptimal solutions, particularly in environments where the reward landscape has many local optima or where initial exploration is insufficient [13].

The *maximum entropy* framework augments the objective with the policy's entropy:

$$J_{\text{MaxEnt}}(\theta) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t (r_t + \alpha \mathcal{H}(\pi_\theta(\cdot | s_t))) \right], \quad (3.17)$$

where $\mathcal{H}(\pi(\cdot | s)) = -\sum_a \pi(a | s) \log \pi(a | s)$ is the Shannon entropy and $\alpha > 0$ is a temperature coefficient controlling the trade-off between reward maximization and entropy maximization [42].

The effect of this term is to penalize overconfident policies. A policy that assigns all probability to one action has zero entropy; a uniform policy has maximum entropy. By rewarding high-entropy behavior, the agent is encouraged to keep exploring even after it has found a reasonably good strategy, remaining robust to reward misspecification and better able to discover alternative high-reward trajectories.

Three concrete benefits follow:

1. *Exploration*: the entropy bonus prevents the policy from collapsing to a single action prematurely, maintaining diversity in the experience collected.
2. *Robustness*: a stochastic policy is harder for the environment to exploit; in multi-agent settings, an unpredictable policy is also harder for other agents to counter-exploit.
3. *Multi-modality*: when multiple actions achieve similar returns, the maximum entropy policy distributes probability across all of them rather than arbitrarily committing to one, which reflects genuine uncertainty.

The temperature α determines how much weight the entropy term receives. A large α produces nearly uniform policies; $\alpha \rightarrow 0$ recovers the standard RL objective. Rather than setting α by hand, SAC [13] treats it as a Lagrange multiplier and learns it automatically by constraining the expected entropy to a target value $\mathcal{H}_{\text{target}}$, typically set to $-|\mathcal{A}|$ (the negative action-space size). This adaptive scheme maintains a consistent level of exploration throughout training without requiring manual temperature scheduling. The full SAC algorithm and its actor-critic instantiation are described in Section 4.5.4.

3.3 Graph Neural Networks

3.3.1 Why Graphs for Multi-Agent Traffic Control

A road network naturally is a directed graph: intersections are nodes and road segments are edges. Traffic dynamics propagate along this graph – congestion at one intersection spills back to upstream intersections and influences downstream demand. A signal controller that acts only on its own local observation ignores this spatial structure. Incorporating neighbor information should, in principle, enable better-coordinated decisions.

Standard multi-layer perceptrons cannot exploit graph structure because they process a fixed-size vector input with no explicit notion of adjacency. A natural alternative is to pass messages along the graph edges so that each node aggregates information from its neighbors before making a decision. Graph neural networks (GNNs) provide a principled framework for doing so in a way that is invariant to the labeling and ordering of nodes – a property critical for sharing parameters across intersections that may have different phases but the same coordination problem.¹

3.3.2 Message Passing Framework

All GNN variants can be expressed as instances of the *message passing neural network* (MPNN) framework [12]. At each layer ℓ , node i updates its representation $\mathbf{h}_i^{(\ell)}$ by aggregating messages from its neighbors $\mathcal{N}(i)$:

$$\mathbf{m}_{ij}^{(\ell)} = \text{MESSAGE}^{(\ell)}(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, e_{ij}), \quad (3.18)$$

$$\mathbf{h}_i^{(\ell+1)} = \text{UPDATE}^{(\ell)}\left(\mathbf{h}_i^{(\ell)}, \bigoplus_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^{(\ell)}\right), \quad (3.19)$$

where e_{ij} is an optional edge feature and $\bigoplus$ is a permutation-invariant aggregation (sum, mean, or max). After L layers, each node's representation $\mathbf{h}_i^{(L)}$ encodes information from all nodes within L hops, enabling indirect coordination across the network.

3.3.3 Graph Convolutional Networks

The Graph Convolutional Network (GCN) [17] is one of the simplest instantiations of the MPNN framework. It aggregates neighbor features with fixed normalized weights:

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{\mathbf{h}_j^{(\ell)}}{\sqrt{d_i d_j}}\right), \quad (3.20)$$

where $d_i = |\mathcal{N}(i)| + 1$ is the degree of node i (with self-loop), $\mathbf{W}^{(\ell)}$ is a learnable weight matrix, and σ is a non-linearity. The normalization by $\sqrt{d_i d_j}$ prevents features at high-degree nodes from dominating the aggregation. GCN

¹For an interactive visual introduction to GNNs and the intuition behind message passing, see Sanchez-Lengeling et al. (2021), *A Gentle Introduction to Graph Neural Networks*, Distill: <https://distill.pub/2021/understanding-gnns/>.

is computationally efficient but treats all neighbors equally, which is a significant limitation: in a traffic network, a heavily congested upstream intersection is more informative than a lightly loaded one, and the relative importance of neighbors changes with traffic conditions throughout the day.

3.3.4 Graph Attention Networks

The Graph Attention Network (GAT) [32] replaces the fixed normalization coefficients of GCN with learnable, content-dependent attention weights. For each directed edge $(j \rightarrow i)$, an attention coefficient α_{ij} is computed:

$$e_{ij} = \mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j], \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \quad (3.21)$$

where $\mathbf{W}$ is a shared linear transformation, $\mathbf{a}$ is a learnable attention vector, and $\parallel$ denotes concatenation. The aggregation uses these coefficients as weights:

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(\ell)} \right). \quad (3.22)$$

Multi-head attention runs H independent attention heads and concatenates (or averages) their outputs, stabilizing training and increasing model capacity.

Limitation: static attention. A subtle but important limitation of GAT was identified by Brody, Alon, and Yahav [5]: the attention score e_{ij} in Equation (3.21) is the product of a *fixed* linear transformation of the concatenated features with a fixed vector $\mathbf{a}$. Because the transformation $\mathbf{W}$ is shared across source and target, the ranking of neighbors induced by the attention scores cannot change depending on which node is the target. More precisely, GAT attention is *static*: if $\alpha_{ij} > \alpha_{ik}$ for one target node i , the same ordering holds for all target nodes that share the same neighborhood features. This limits the model's ability to express the situation where intersection i finds neighbor j more informative than k , while intersection i' finds the opposite ordering.

3.3.5 GATv2: Dynamic Attention

GATv2 [5] fixes the static-attention problem by applying the linear transformation and the non-linearity *before* the inner product with $\mathbf{a}$:

$$e_{ij}^{v2} = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \parallel \mathbf{h}_j]). \quad (3.23)$$

Inserting the non-linearity between the weight matrix and the scoring vector breaks the linearity that caused static attention in the original formulation. The resulting attention is *dynamic*: the ranking of neighbors can differ for different target nodes even with the same neighbor feature sets. Brody, Alon, and Yahav prove that GATv2 is strictly more expressive than GAT – it can represent any attention function that GAT can, plus functions that GAT cannot.

In the context of traffic signal control, dynamic attention allows an intersection agent to weight its neighbors differently depending on its own current state. A saturated intersection may attend heavily to its most congested upstream neighbor to anticipate incoming demand, while an under-loaded intersection may attend more uniformly. FGSv2 uses GATv2 as its communication layer for this reason.

3.4 Traffic Signal Control Theory

3.4.1 Movements and Phases

The fundamental unit of signalized intersection control is the *movement*: a directional traffic flow from one approach lane to one or more exit lanes [19]. At a standard four-leg intersection with one lane per approach, up to twelve movements can be identified: three per leg (left turn, through, right turn). Movements are labelled by the leg they originate from and the direction they take.

Two movements *conflict* if they cannot proceed simultaneously without risk of collision [19]. For example, a through movement on the east–west axis conflicts with a left-turn movement from the north or south. The set of all conflict relationships forms the *conflict matrix* of the intersection, which encodes the physical geometry of the road.

A *phase* is a set of non-conflicting movements that are served simultaneously under a green signal [19]. The controller selects a sequence of phases, one at a time, cycling through them to serve all movement demands. Because real intersections group compatible movements into phase programmes defined in the signal plan, each phase corresponds to a discrete action in the RL action space: the agent chooses which phase to activate next.

Figure 3.1 illustrates these concepts for a standard four-arm intersection with three lanes per approach. Panel (a) shows the eight numbered lanes; panel (b) enumerates the twelve possible traffic movements; panel (c) lists eight common phases (A–H), each activating a compatible subset of those movements; panel (d) shows the sequential phase-selection decision, where the controller picks one phase at each time step t .

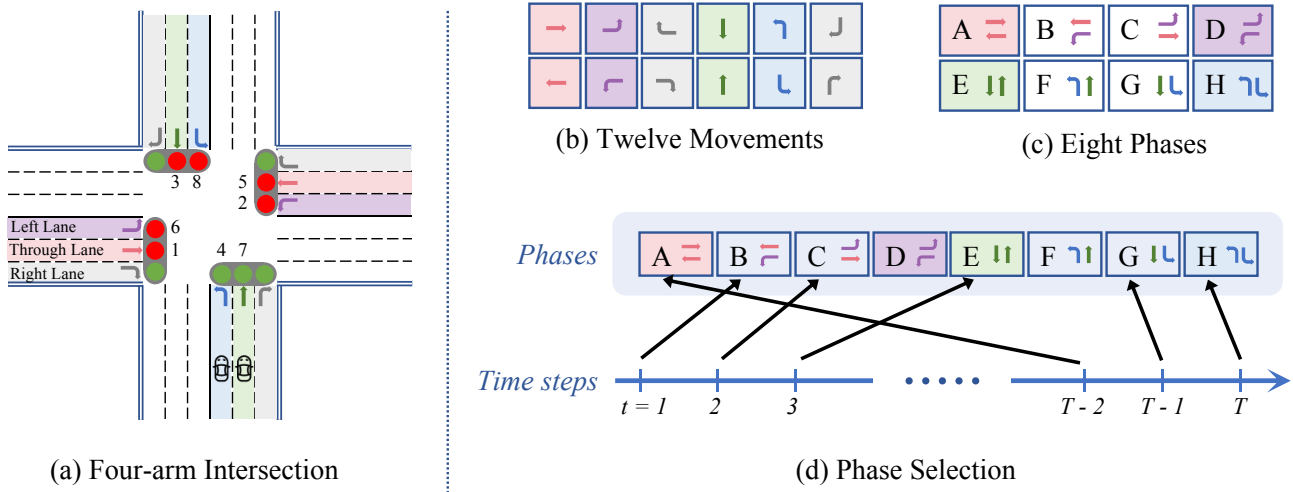


Figure 3.1: Key traffic signal control concepts at a four-arm intersection. (a) Physical layout with numbered lanes (left, through, right per approach). (b) The twelve directional traffic movements, grouped by conflict colour. (c) Eight phases (A–H), each activating a non-conflicting subset of movements for simultaneous service under a green signal. (d) Sequential phase-selection: the RL agent selects one phase per time step. Figure adapted from Ruan et al. [26].

3.4.2 Phase Timing and Transitions

Between two consecutive phases, the intersection must display transitional signals to clear conflicting traffic [19]. The standard transition sequence is:

1. *Yellow* (typically 3–5 s): warns drivers that the green is ending; crossing vehicles with right-of-way may complete their movement.
2. *All-red* (typically 1–2 s): all approaches display red, providing a safety clearance interval before the new phase begins.

The SUMO simulator applies these transition intervals automatically based on the signal plan; the RL agent does not control them directly. The *minimum green time* [19] — the shortest duration a phase must remain active before the agent can switch — prevents premature phase changes that would cause vehicles to stop mid-intersection.

Several time quantities characterize the overall signal programme [19]:

- *Cycle length*: the total time for all phases (and their transitions) to complete one rotation. Webster’s formula [34] provides an analytical optimum for fixed-time cycle lengths under stationary demand.
- *Green split*: the fraction of the cycle allocated to each phase, determining how much capacity each movement receives.
- *Offset*: for coordinated signals on an arterial, the offset is the time shift between the start of green phases at consecutive intersections, enabling vehicles traveling at the design speed to encounter consecutive greens (a *green wave*).

3.4.3 Classical vs. Adaptive Control

Fixed-time control. In fixed-time control, the phase sequence, green splits, and cycle length are pre-computed from historical demand data and applied without feedback. Webster’s method minimizes average vehicle delay by balancing the green time according to the relative flow on each approach. Fixed-time plans are simple and predictable, but they cannot adapt to non-stationary demand (incidents, events, off-peak anomalies) or to the spatial spillback that occurs when queues at one intersection block upstream approaches.

Actuated and adaptive control. Vehicle-actuated controllers extend green phases when detectors sense approaching vehicles, and terminate phases early if no vehicles are detected [19]. More sophisticated adaptive systems such as SCOOT [16] and SCATS [23] maintain a real-time estimate of occupancy and adjust timing continuously. These systems outperform fixed-time control under variable demand but rely on extensive detector infrastructure and expert-tuned parameters that may not generalize across networks.

Reinforcement learning for TSC. RL-based control recasts phase selection as a sequential decision problem, learning directly from simulation feedback without requiring a hand-crafted model of traffic dynamics. The agent observes sensor measurements (queue length, occupancy, current phase), selects the next phase, and receives a reward that quantifies the quality of that decision. Because RL learns a *policy* rather than a fixed plan, it can in principle adapt to any traffic condition within the distribution seen during training.

Phase competition as an inductive bias. The FRAP encoder [41] exploits a domain insight: at any moment, the controller’s decision reduces to selecting the phase whose served movements have the most urgent demand relative to the movements it delays. This is the *phase competition* perspective — phases compete against each other based on the current demand profile. Formalizing this competition explicitly in the encoder architecture encodes knowledge of the intersection structure as an inductive bias, reducing the amount of experience the agent needs to learn effective phase selection. The encoder is also permutation-equivariant with respect to phase labeling, so a single set of parameters generalizes across intersections with different numbers of phases and movement configurations — a critical property for the heterogeneous German RESCO scenarios where different intersections within the same network support different numbers of phases.

Chapter 4

Methodology

A survey of MARL-TSC literature reveals that published methods — IntelliLight [35], PressLight [37], FRAP [41], CoLight [36], and MPLight [7] — all process each agent’s local observation through three successive operations, even though individual papers rarely describe their architectures in these unified terms. First, a *local encoder* maps raw traffic measurements to a feature vector capturing demand and phase state at one intersection. Second, an optional *communication layer* aggregates neighbor embeddings over the road graph to introduce spatial context. Third, an *RL optimizer* uses the resulting representation to select a phase and update the policy.

This thesis formalizes that implicit structure as the **FGS pipeline** (Framework for Graph-aware Signal control): a three-module design space in which every MARL-TSC architecture is fully characterized by three independent choices — encoder, communication, and RL algorithm — as illustrated in Figure 4.1. The FGS pipeline is developed through three generations of iterative diagnosis-and-repair, each addressing a specific module-interface failure identified in evaluation of the preceding design. The final FGS architecture combines FRAP-based action-token encoding, demand-aware GATv2 communication, and a factored neighborhood critic, described in full in Section 4.6.

The remainder of this chapter follows the pipeline structure. Section 4.1 states the formal problem. Section 4.2 introduces the general framework, its design table, and how prior work maps onto it. Sections 4.3–4.5 describe each module’s design choices in depth. Section 4.6 describes the FGS pipeline architecture and shared training algorithm. All three variants follow the CTDE paradigm introduced in Section 3.1.2: the actor uses only agent k ’s local observation at execution time, while the centralized critic has access to the joint state during training.

4.1 Problem Formulation

A signalized road network with N intersections is modelled as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, T, \{r_i\}, \gamma \rangle$, where $\mathcal{N} = \{1, \dots, N\}$ is the set of agents, $\mathcal{S}$ is the global state of the SUMO environment, $\mathcal{O}_i$ is the local observation space of agent i , $\mathcal{A}_i$ is its action space, T is the stochastic transition function determined by the simulator, r_i is the local reward function, and $\gamma \in (0, 1)$ is the discount factor. Agents act simultaneously at each decision step; each agent observes only the state of its own intersection and its immediate neighborhood.

4.2 The Three-Module Framework

Notation. Indices $i, j \in \mathcal{N}$ denote intersections; a, b denote phases; m denotes a traffic movement; k identifies the acting agent (the one currently selecting an action). The phase validity flag $m_{i,a} \in \{0, 1\}$ equals 1 if phase a is available at intersection i (formally defined in Section 4.3.1). Within the FRAP encoder (Section 4.3.3) the intersection subscript is suppressed. The termination flag in SAC is written $d^{\text{ep}} \in \{0, 1\}$ to avoid clash with the observation dimension d . Concatenation is $[\cdot \parallel \cdot]$. Table 4.1 lists principal intermediate features.

Figure 4.1 depicts the general framework. At each decision step, SUMO emits a graph-structured observation o_i for each agent i ; the node feature vector $\mathbf{x}_i \in \mathbb{R}^d$ is one component of o_i (see Eq. 4.1). The three modules process it sequentially.

Table 4.2 maps representative prior methods and the three FGS instantiations onto this design space. Every prior graph-based method (CoLight) uses GAT with an MLP encoder and DQN; every method using a phase-competition encoder

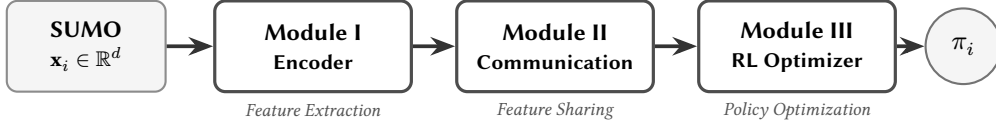


Figure 4.1: The FGS three-module pipeline. Each agent i receives a node feature vector $\mathbf{x}_i \in \mathbb{R}^d$ from the environment. Module I (Encoder) maps $\mathbf{x}_i$ to a local embedding; Module II (Communication) propagates embeddings over the road graph; Module III (RL Optimizer) produces the phase-selection policy π_i .

Table 4.1: Principal feature symbols used in this chapter. Each symbol is first introduced in the cited equation or section.

Symbol	Shape	Description	First defined
$\mathbf{x}_i$	$\mathbb{R}^d$	Raw node feature vector	Eq. 4.1
$m_{i,a}$	$\{0, 1\}$	Phase validity flag	Sec. 4.3.1
$\mathbf{h}_i$	$\mathbb{R}^{d_h}$	Local embedding (FGS Module I)	Sec. 4.3.2
$\phi_{i,a}$	$\mathbb{R}^C$	Per-phase demand embedding (FRAP)	Eq. 4.3
$\mathbf{z}_{i,a}$	$\mathbb{R}^{d_z}$	Per-phase action token (FGS Module I)	Eq. 4.22
$\mathbf{d}_i$	$\mathbb{R}^{d_c}$	Demand comm. embedding (FGS Module II)	Eq. 4.24
$\mathbf{g}_i$	$\mathbb{R}^{d_c}$	Gated graph context (Module II out)	Eq. 4.26
α_{ij}	$\mathbb{R}$	GATv2 attention weight	Eq. 4.11
α	$\mathbb{R}_{>0}$	SAC temperature	Eq. 4.16
d^{ep}	$\{0, 1\}$	Episode-termination flag	Eq. 4.18

(standalone FRAP) uses no graph communication. No prior work treats all three module choices as variables and ablates their interactions on German RESCO scenarios — that systematic characterization is the primary contribution of this thesis.

Table 4.2: MARL-TSC methods mapped onto the three-module pipeline. Bold rows: proposed methods. “FRAP-tok.” = FRAP encoder tapped to produce per-phase action tokens rather than scalar scores. “GATv2(D)” = GATv2 operating on demand-derived embeddings (FGS, the final design).

Method	Module I: Encoder	Module II: Comm.	Module III: RL
IntelliLight [35]	MLP	None	DQN
PressLight [37]	Pressure	None	DQN
FRAP [41]	FRAP	None	DQN
CoLight [36]	MLP	GAT	DQN
MPLight [7]	Pressure	None	DQN
IDQN / IPPO (RESCO baselines)	MLP	None	DQN / PPO
FGS (this thesis)	MLP or FRAP	GATv2	SAC or PPO
FGSv2 (this thesis)	FRAP-tok.	GATv2	SAC
FGSv3 (this thesis)	FRAP-tok.	GATv2(D)	SAC or PPO

4.3 Module I: Local Encoder

The encoder converts the raw traffic observation at one intersection into a fixed-length feature vector — the local embedding that will either be passed directly to the RL module (no communication) or sent as a message to neighboring agents. The design choice here is what structural inductive bias to embed in the encoder: an unstructured MLP that learns features from data, or the FRAP competition encoder that makes phase urgency explicit.

4.3.1 Observation Space

Each agent receives a dictionary observation at every decision step. The dictionary encodes the agent’s local traffic state together with the graph structure of the surrounding network, allowing the model to perform graph message passing during the forward pass.

The node feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ stacks the feature vector of every agent in the network. For agent i , the feature vector is

$$\mathbf{x}_i = [\mathbf{e}_{\text{phase}}^{(i)}, t_{\text{green}}^{(i)}, \boldsymbol{\rho}^{(i)}, \mathbf{q}^{(i)}] \in \mathbb{R}^d, \quad (4.1)$$

where $\mathbf{e}_{\text{phase}}^{(i)}$ is a one-hot vector encoding the current active phase, $t_{\text{green}}^{(i)} \in \{0, 1\}$ indicates whether the minimum green time has been satisfied at intersection i [35], and $\boldsymbol{\rho}^{(i)}, \mathbf{q}^{(i)}$ are the lane density and lane queue vectors over all incoming lanes of intersection i .

The remaining fields in the observation dictionary are:

- *Edge index* $\mathcal{E} \in \mathbb{Z}^{2 \times |\mathcal{E}|}$: source–target integer pairs defining the agent communication graph.
- *Edge mask* and *edge weight*: a binary validity flag and a scalar weight w_{ij} for each edge (see Section 4.4.1).
- *Ego index*: the integer identifying which row of $\mathbf{X}$ corresponds to the acting agent.
- *Action mask* $\mathbf{m}_i \in \{0, 1\}^{A_{\max}}$: component $m_{i,a} = 1$ if phase a is valid at intersection i , and 0 otherwise. When the intersection subscript is omitted (e.g., in the FRAP and FGS actor derivations) agent k is implied.
- *Phase-pair mask* $\mathbf{M}_{\text{pair}} \in \{0, 1\}^{N \times A_{\max} \times M}$: for each agent and candidate phase, indicates the subset of traffic movements that phase controls.
- *Phase-competition mask* $\mathbf{M}_{\text{comp}} \in \{0, 1, 2, 3\}^{N \times A_{\max} \times (A_{\max} - 1)}$: encodes one of four movement-conflict relation types between phase a and each competitor b , following the FRAP relation taxonomy [41]: type 0 — no shared movement conflicts (non-competing phases); type 1 — one shared conflicting movement; type 2 — two shared conflicting movements; type 3 — the phases serve directly opposing movements on the same approach. The self-competition entry is absent by construction.
- *Previous joint action* $\mathbf{a}_{t-1} \in \{0, 1\}^{N \times A_{\max}}$: one-hot encoding of all agents' actions at the preceding step. This field provides each agent with knowledge of the phase active at neighboring intersections at the prior step, supporting the demand-aware coordination in FGS (Section 4.6).

4.3.2 MLP Encoder

The simplest encoder choice is a shared two-layer MLP that maps the raw feature vector $\mathbf{x}_i \in \mathbb{R}^d$ to a local embedding $\mathbf{h}_i \in \mathbb{R}^{d_h}$:

$$\mathbf{h}_i = \text{MLP}_{\psi}(\mathbf{x}_i), \quad i = 1, \dots, N.$$

The MLP encoder treats the observation as an unstructured vector with no prior knowledge about which features correspond to which phases or movements. This is the most common encoder choice in MARL-TSC literature — used in IntelliLight [35], CoLight [36], and RESCO baselines [2] — and it serves as the baseline against which the FRAP encoder is compared in the FGS ablation study.

4.3.3 FRAP Encoder

FRAP [41] encodes the competition between candidate signal phases by explicitly comparing each phase against every competitor and pooling the results. It embeds the core domain insight that the controller's decision reduces to a relative question: *which candidate phase has the most urgent demand compared with the movements it preempts?* By making this comparison explicit in the architecture, FRAP acts as a structured inductive bias that reduces the number of samples needed to learn effective phase selection. Throughout this section the intersection subscript i is suppressed for clarity; all tensors and indices belong to a single intersection.

Let the intersection have M incoming traffic movements and A candidate phases. The phase-movement mask $\mathbf{P} \in \{0, 1\}^{A \times M}$ sets $P_{am} = 1$ if movement m is served by phase a . The phase-competition relation $R_{ab} \in \{0, 1, 2, 3\}$ encodes one of four movement-conflict types between each ordered pair (a, b) of distinct phases, as defined in Section 4.3.1 (types 0–3 range from no shared conflict to directly opposing movements).

Step 1: Movement embedding. For each movement m , a phase indicator $p_m \in \{0, 1\}$ (whether m is currently green) and a demand scalar d_m (lane queue length or vehicle density) are embedded separately and fused:

$$\mathbf{e}_m = \text{ReLU}\left(W_{\text{lane}}[\sigma(\text{Emb}_p(p_m)), \sigma(W_d d_m)]\right) \in \mathbb{R}^C, \quad (4.2)$$

where $\text{Emb}_p \in \mathbb{R}^{2 \times C}$ is a two-entry learnable lookup table [41]: embedding the binary indicator rather than passing p_m directly as a scalar gives the model an unconstrained C -dimensional representation of the serving and non-serving states, instead of forcing them to map to a fixed numeric ratio; W_d is a demand projection; and σ is the sigmoid function.

Step 2: Phase embedding. For each candidate phase a , the movement embeddings of the movements it controls are aggregated via a scaled sum:

$$\phi_a = \sum_{m=1}^M w_{am} \mathbf{e}_m \in \mathbb{R}^C, \quad (4.3)$$

where $P_{am} \in \{0, 1\}$ is the entry of the phase-movement mask $\mathbf{P}$ indicating whether movement m is served by phase a (defined above). In the original FRAP formulation [41] $w_{am} = P_{am}$, which assumes every phase controls exactly two movements. German RESCO intersections have variable movement counts per phase, so this implementation replaces the plain mask with a scaled weight:

$$w_{am} = P_{am} \cdot \frac{\min(\sum_{m'} P_{am'}, 2)}{\max(\sum_{m'} P_{am'}, 1)}, \quad (4.4)$$

which rescales ϕ_a to the equivalent of a two-movement sum regardless of how many movements phase a controls, ensuring scale comparability across heterogeneous phases. The vector ϕ_a is also used by FGS as the demand communication signal (Section 4.6).

Step 3: Pairwise competition feature. For each ordered competitor pair (a, b) with $b \neq a$:

$$\mathbf{h}_{ab} = \text{ReLU}(\text{Conv}_{\text{pair}}([\phi_a \parallel \phi_b])) \in \mathbb{R}^C, \quad (4.5)$$

$$\mathbf{r}_{ab} = \text{ReLU}(\text{Conv}_{\text{rel}}(\text{Emb}_{\text{rel}}(R_{ab}))) \in \mathbb{R}^C, \quad (4.6)$$

$$\mathbf{c}_{ab} = \text{ReLU}(\text{Conv}_{\text{hid}}(\mathbf{h}_{ab} \odot \mathbf{r}_{ab})) \in \mathbb{R}^C. \quad (4.7)$$

The relation embedding $\mathbf{r}_{ab}$ captures the movement-conflict type; the Hadamard product¹ gates the phase-pair interaction by this structure. The vector $\mathbf{c}_{ab}$ is the *phase-competition representation* — a per-competitor feature encoding both relative demand and structural conflict.

Step 4: Phase score (original FRAP output). In the original FRAP, a final 1×1 convolution projects each competition feature to a scalar:

$$s_{ab} = \text{Conv}_{1 \times 1}(\mathbf{c}_{ab}) \in \mathbb{R}. \quad (4.8)$$

The phase score for candidate phase a is obtained by summing over all valid competitors $V_a = \{b \mid b \neq a, m_b = 1\}$ (suppressing the intersection subscript as above):

$$s_a = \sum_{b \in V_a} s_{ab}, \quad (4.9)$$

yielding one scalar per candidate phase that directly serves as a Q-value estimate in the original FRAP policy.

FGS *replaces* Step 4 entirely: instead of projecting $\mathbf{c}_{ab}$ to a scalar, it mean-pools $\mathbf{c}_{ab}$ across valid competitors to produce a per-phase action token $\mathbf{z}_{i,a}$ (Eq. 4.22; intersection subscript i restored), which is then consumed directly by the action-conditioned actor (Eq. 4.27). This retains the full C -dimensional feature vector through Module II and Module III without discarding per-phase competition information.

The overall design is permutation-equivariant with respect to phase ordering: reordering the candidate phases produces a correspondingly reordered output vector, which supports generalization across intersections with heterogeneous phase counts and movement configurations.

¹The Hadamard product $\odot$ denotes element-wise multiplication: $\begin{pmatrix} a \\ b \end{pmatrix} \odot \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} ac \\ bd \end{pmatrix}$, or for 2×2 matrices, $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \odot \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} ae & bf \\ cg & dh \end{pmatrix}$.

4.4 Module II: Graph Communication

The communication module aggregates neighbor embeddings over the road graph, giving each agent access to spatially correlated demand signals from nearby intersections. Without this module, agents act independently with no knowledge of upstream or downstream conditions – the baseline used by IDQN and IPPO. The design choice is the attention mechanism: fixed-weight aggregation, static GAT (CoLight), or dynamic GATv2 (this work).

4.4.1 Agent Communication Graph

The agent communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$ is derived from the SUMO road network file (.net.xml) before training begins and is fixed throughout.

Node extraction. Every junction that carries a traffic light logic (tlLogic) in the network XML is added to $\mathcal{V}$ as a graph node. Its geographic coordinates are recorded for visualization but are not used as model inputs.

Super-edge construction. For each outgoing road segment from a traffic light node, a Dijkstra traversal follows legal road transitions until it reaches the nearest downstream traffic light node, accumulating the intermediate edges along the path. This yields a set of *super-edges* that connect pairs of signalized intersections directly, abstracting away unsignalized junctions in between.

Edge weight assignment. The weight of super-edge (i, j) is

$$w_{ij} = \frac{1}{\tau_{ij}}, \quad (4.10)$$

where τ_{ij} is the estimated travel time along the path. When multiple parallel routes connect the same signal pair, their weights accumulate additively. All edges are inserted bidirectionally to enable symmetric message passing. The resulting graph is serialized to disk as a pre-processing artefact and loaded once per training run, adding no per-step overhead. The full extraction procedure is given in Algorithm 3 (Appendix A.12).

Figure 4.2 shows the extracted agent communication graphs for all six German RESCO scenarios, grouped by tier. Edge width is proportional to the weight $w_{ij} = 1/\tau_{ij}$: thicker edges indicate shorter inter-signal travel times and thus stronger potential influence.

4.4.2 GATv2 Attention

Both FGS and FGSv2 use GATv2 [5] as the graph communication operator. As discussed in Sections 3.3.5 and 2.3, GATv2 addresses the static-attention limitation of the original GAT by inserting a non-linearity before the attention scoring, enabling attention coefficients to depend jointly on both the source and the target node’s features. The normalized attention coefficient for edge (i, j) is

$$\alpha_{ij} = \frac{\exp(\mathbf{a}^\top \text{LeakyReLU}(\mathbf{W} [\mathbf{h}_i \parallel \mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\mathbf{a}^\top \text{LeakyReLU}(\mathbf{W} [\mathbf{h}_i \parallel \mathbf{h}_k]))}, \quad (4.11)$$

where $\mathbf{W}$ and $\mathbf{a}$ are learnable parameters, $\parallel$ denotes concatenation, and α_{ij} is the normalized attention coefficient for edge (i, j) – note that α_{ij} here is a graph-attention weight, distinct from the SAC temperature scalar α used in Section 4.5.4. Multi-head attention is applied with H heads; outputs are concatenated and projected to the output dimension. All FGS and FGSv2 variants with graph communication use GATv2 as specified above. A proof of node-permutation equivariance and a discussion of edge-weight integration are in Appendix A.6.

4.5 Module III: RL Optimizer

The RL module receives the encoder embedding (optionally enriched by graph communication) and learns a phase-selection policy by optimizing a scalar reward signal. The design choice here spans both the action formulation and the RL algorithm. FGS uses discrete phase selection and evaluates three RL algorithms: DQN, PPO, and SAC. The reward signal is fixed across all methods to isolate algorithmic differences.

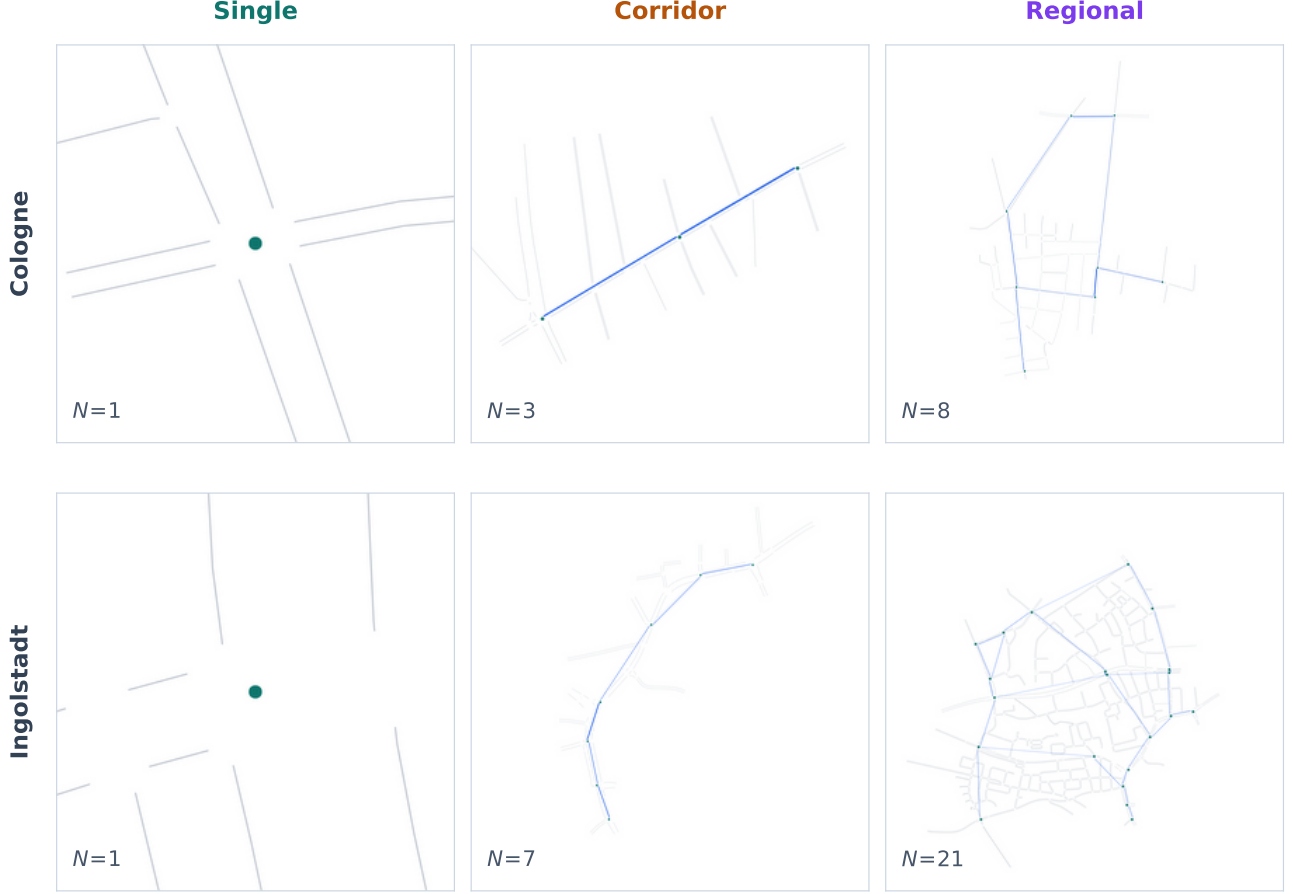


Figure 4.2: Agent communication graphs $\mathcal{G}$ for all six German RESCO scenarios. Rows correspond to cities (Cologne, Ingolstadt); columns to scale tiers (Single, Corridor, Regional). Gray polylines show the underlying SUMO road network; blue edges are super-edges between the nearest downstream signalized intersections; green nodes are TLS agents. N denotes the agent count.

4.5.1 Action Space

The action space of each agent is discrete: at each step the agent selects a phase index from the set of phases its intersection supports. Because intersections in the German RESCO scenarios have different numbers of phases, the action space is padded to a uniform width $A_{\max} = \max_{i \in \mathcal{N}} |\mathcal{A}_i|$. During the forward pass, the logits of all invalid phases are masked to $-\infty$ before the softmax, ensuring that the sampled action always lies within the agent's valid phase set.

Phase transition constraints (minimum green time, yellow interval) are enforced at the SUMO environment level and are not modelled as RL constraints.

4.5.2 Reward Function

4.5.2.1 Differential Waiting Time

All experiments use the differential waiting time reward introduced by the RESCO benchmark [2], defined as the signed change in accumulated waiting time between consecutive decision steps. Cabrejas-Egea et al. [6] compare several RL-TSC reward formulations under real-world limitations and find that difference-based waiting-time signals provide stable gradients across varied demand conditions, supporting its adoption here.

$$r_t = W_{t-1} - W_t, \quad W_t = \frac{1}{100} \sum_{l \in \mathcal{L}} \sum_{v \in l} w_v(t), \quad (4.12)$$

where $\mathcal{L}$ is the set of incoming lanes controlled by the traffic signal, v indexes vehicles, and $w_v(t)$ is the accumulated waiting time of vehicle v at step t as reported by SUMO via the TraCI interface. The term *differential* follows RESCO's

diff-waiting-time naming convention and refers to the finite difference $W_{t-1} - W_t$, not a calculus derivative. A positive value of r_t indicates that total waiting time at the intersection decreased relative to the previous step. Dividing by 100 normalizes the scale across scenarios that differ in traffic volume. Appendix A.2 proves that maximizing the discounted sum of this reward is equivalent to minimizing a discounted sum of waiting times.

This reward is local to each agent, which aligns with the CTDE assumption: the reward does not require knowledge of the global joint state at execution time, so decentralized policies remain valid after training.

4.5.2.2 Alternative Reward Signals

Delay. Delay quantifies the additional travel time incurred by vehicles relative to their free-flow speed. It captures network-level throughput effects directly but requires per-vehicle free-flow speed estimates from SUMO, increasing per-step computational cost. Delay is used as an *evaluation metric* (Section 5.3) rather than a training signal.

Pressure. Pressure [37] measures the lane imbalance at an intersection:

$$p_i = \left| \sum_{l \in \mathcal{L}_{\text{in}}^i} \rho_l - \sum_{l \in \mathcal{L}_{\text{out}}^i} \rho_l \right|, \quad (4.13)$$

where ρ_l is vehicle density on lane l . The pressure reward $r_t = -p_i(t)$ penalizes queue accumulation and has been shown to stabilize single-agent training [37]. It was evaluated as an alternative signal but not adopted as the primary reward in this work.

4.5.3 DQN and PPO Baselines

The following algorithms serve as baselines or as the algorithmic foundation of the proposed framework. In the multi-intersection setting, each intersection runs an independent copy of the single-agent algorithm with no inter-agent communication. Following the RESCO benchmark terminology [2], the independent-agent variant of DQN is referred to as **IDQN** (Independent DQN) and the independent-agent variant of PPO is referred to as **IPPO** (Independent PPO).

4.5.3.1 Deep Q-Network

Deep Q-Network (DQN) [24] approximates the action-value function with a neural network $Q_\theta(s, a)$ trained to minimize the temporal-difference loss

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a) \right)^2 \right], \quad (4.14)$$

where $\mathcal{D}$ is a replay buffer and $\bar{\theta}$ are the parameters of a periodically updated target network. Each intersection runs an independent DQN (IDQN) with no inter-agent communication, forming a fully decentralized value-based baseline.

4.5.3.2 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [28] is an on-policy actor-critic method that limits policy update size through a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (4.15)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is an advantage estimate. PPO is used as an independent-agent on-policy baseline (IPPO). Because PPO's value function scales as $O(d)$ independent of the number of agents N , it remains tractable even at the largest evaluated scenario (ingolstadt21, $N = 21$), which motivates its use as the RL optimizer in the FGS final design (FGS-PPO).

4.5.4 Soft Actor-Critic

Soft Actor-Critic (SAC) [13] is the primary RL optimizer in the FGS framework. It augments the expected return objective with an entropy regularization term:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right], \quad (4.16)$$

where $\alpha > 0$ is a temperature parameter balancing exploitation and exploration. SAC is chosen over DQN and PPO for two reasons. First, its off-policy replay buffer reuses simulation experience more efficiently than PPO’s on-policy rollouts, reducing the number of expensive SUMO episodes required. Second, its entropy bonus prevents premature policy collapse in the multi-agent setting, where diverse individual policies reduce the risk of converging to coordination patterns that fail under demand shift.

Because traffic signal control admits only discrete phase actions, this work applies the discrete-action SAC formulation [8], which marginalizes analytically over all actions rather than relying on the reparameterization trick. Twin Q-networks $Q_{\theta_1}, Q_{\theta_2}$ are maintained to reduce overestimation bias. The discrete-action soft target value is

$$V_{\text{target}}(s') = \sum_{a'} \pi_{\phi}(a' | s') \left[\min_{k=1,2} Q_{\theta_k}(s', a') - \alpha \log \pi_{\phi}(a' | s') \right], \quad (4.17)$$

$$y = r + \gamma (1 - d^{\text{ep}}) V_{\text{target}}(s'), \quad (4.18)$$

where $d^{\text{ep}} \in \{0, 1\}$ is the episode-termination flag (see Table 4.1). The critic and actor losses are, respectively:

$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(Q_{\theta}(s, a) - y)^2 \right], \quad (4.19)$$

$$\mathcal{L}_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[\sum_a \pi_{\phi}(a | s) \left(\alpha \log \pi_{\phi}(a | s) - \min_k Q_{\theta_k}(s, a) \right) \right]. \quad (4.20)$$

The temperature α is itself optimized by minimizing the dual objective

$$\mathcal{L}(\alpha) = \mathbb{E}_{s \sim \mathcal{D}} \left[\alpha (\mathcal{H}(\pi(\cdot | s)) - \mathcal{H}_{\text{target}}) \right], \quad (4.21)$$

where $\mathcal{H}_{\text{target}} = -\log(1/A_{\text{valid}})$ is the entropy of the uniform policy over valid actions. Gradient derivations for all three SAC losses and the log-parameterization of α are given in Appendix A.4. The off-policy design of SAC enables training from a prioritized replay buffer, reducing the number of environment interactions needed to reach convergence – an important property when SUMO simulations are the computational bottleneck.

4.6 FGS Architecture

FGS is the framework’s final design, arrived at through three generations of iterative diagnosis-and-repair. The first-generation ablation studies four encoder–communication combinations (Table 4.3) to establish performance baselines and expose two module-interface failures; subsequent generations each address one failure. The complete development narrative and generation comparison are presented in Section 6.6; earlier algorithmic details are in Appendix A.14.

Table 4.3: First-generation FGS ablation variants. All graph-communication variants use GATv2 (Eq. 4.11).

Variant	Module I (Encoder)	Module II (Comm.)
FGS-MLP-none	MLP	none
FGS-MLP-GATv2	MLP	GATv2
FGS-FRAP-none	FRAP	none
FGS-FRAP-GATv2	FRAP	GATv2

4.6.1 Pipeline Architecture

The FGS architecture implements Module I as a FRAP action-token encoder, Module II as a demand-aware GATv2 communication layer, and Module III as an action-conditioned actor paired with a factored neighborhood critic. Figure 4.3 illustrates the complete data flow; all components operate under the CTDE paradigm.

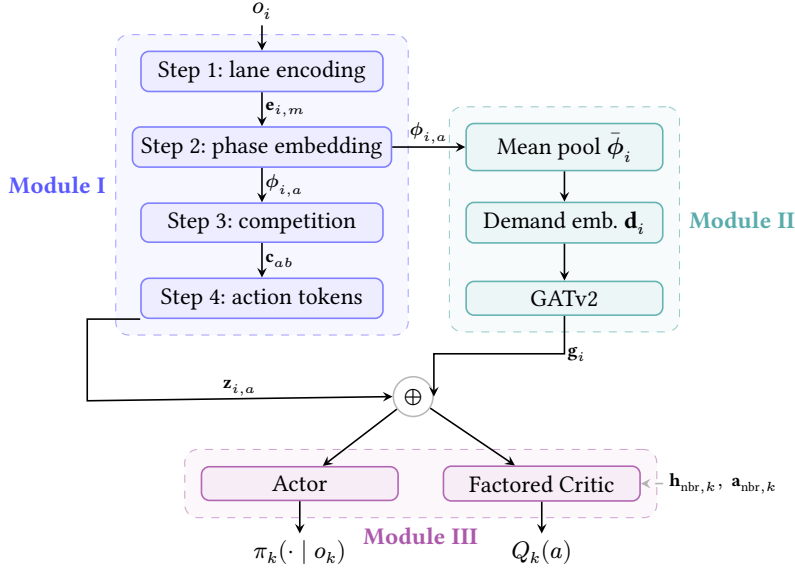


Figure 4.3: FGS pipeline architecture. **Module I** (FRAP encoder): the observation o_i is processed through four steps to produce per-phase action tokens $z_{i,a}$. Step 2 additionally outputs phase demand embeddings $\phi_{i,a}$, which feed the **Module II** demand communication branch (mean-pool $\rightarrow$ demand embedding $\mathbf{d}_i \rightarrow$ GATv2 $\rightarrow$ gated graph context $\mathbf{g}_i$). The action tokens bypass Module II via a skip connection and are concatenated ($\oplus$) with $\mathbf{g}_i$ before **Module III**. The Actor selects a phase for decentralized execution; the Factored Critic additionally conditions on pooled neighbor contexts ($\mathbf{h}_{\text{nbr},k}, \mathbf{a}_{\text{nbr},k}$, dashed arrow) for centralized training.

Module I – FRAP action-token encoder. The encoder is tapped at the phase-competition representation block of FRAP, before the 1×1 convolution that would produce a scalar score. For action a at intersection i , the phase-competition representation $\mathbf{c}_{ab}$ (Equation 4.7) is produced by the hidden competition layer for each competitor $b \in V_{i,a} = \{b \mid b \neq a, m_{i,b} = 1\}$. If $V_{i,a}$ is non-empty, the competition features are pooled and projected into an action token; otherwise the token is set to zero:

$$\mathbf{z}_{i,a} = \begin{cases} \text{LayerNorm}\left(\text{ReLU}\left(\text{Adapter}\left(\frac{1}{|V_{i,a}|} \sum_{b \in V_{i,a}} \mathbf{c}_{ab}\right)\right)\right) & \text{if } m_{i,a} = 1 \text{ and } |V_{i,a}| > 0, \\ \mathbf{0} & \text{otherwise.} \end{cases} \quad (4.22)$$

The full action-token tensor is $\mathbf{Z}_i \in \mathbb{R}^{A_{\max} \times d_z}$, with rows for invalid phases set to zero. Note that the demand embedding $\mathbf{d}_i$ for Module II is computed from $\phi_{i,a}$ (Step 2 of FRAP, Equation 4.3), *before* the competition computation in Steps 3–4. This decoupling is what allows $\mathbf{d}_i$ to carry raw demand rather than competition scores.

Module II – Demand communication branch. The demand communication branch taps FRAP at Step 2 (the phase demand embedding $\phi_{i,a}$, Equation 4.3), which captures raw traffic demand rather than internal competition scores. Let $V_i = \{a \mid m_{i,a} = 1\}$ be the set of valid phases for intersection i . The mean phase demand embedding across valid phases is

$$\bar{\phi}_i = \frac{1}{|V_i|} \sum_{a \in V_i} \phi_{i,a} \in \mathbb{R}^C. \quad (4.23)$$

To also convey which phase intersection i is currently executing, a learned phase-index embedding $\text{Emb}_{\text{phase}}(a_{t-1,i}) \in \mathbb{R}^{d_{pe}}$ is concatenated with $\bar{\phi}_i$ and projected to the communication space:

$$\mathbf{d}_i = \text{LayerNorm}\left(\text{ReLU}\left(W_{\text{comm}}[\bar{\phi}_i \parallel \text{Emb}_{\text{phase}}(a_{t-1,i})]\right)\right) \in \mathbb{R}^{d_c}, \quad (4.24)$$

where $W_{\text{comm}} \in \mathbb{R}^{d_c \times (C+d_{pe})}$ is a learnable projection and d_c is the communication dimension. GATv2 communication (Equation 4.11) then aggregates the demand embeddings over the road graph:

$$\mathbf{u}_i = \text{LayerNorm}(\mathbf{d}_i), \quad \tilde{\mathbf{g}}_i = \text{ReLU}(\mathbf{W}_o \text{GATv2Conv}(\{\mathbf{u}_j : j \in \mathcal{N}(i) \cup \{i\}\})), \quad (4.25)$$

$$\mathbf{g}_i = \text{LayerNorm}(\mathbf{d}_i + \lambda \tilde{\mathbf{g}}_i), \quad (4.26)$$

where λ is a learnable scalar gate initialized to zero, so that the network begins as a purely local policy and gradually incorporates neighborhood context during training.

Module III – Actor. For each candidate phase a , the actor concatenates the action token $\mathbf{z}_{k,a}$, the gated graph context $\mathbf{g}_k$, and a one-hot phase identity $\mathbf{e}_a$:

$$l_a = \text{MLP}_\phi[\mathbf{z}_{k,a} \parallel \mathbf{g}_k \parallel \mathbf{e}_a]. \quad (4.27)$$

Stacking over all a gives $\mathbf{l} \in \mathbb{R}^{A_{\max}}$, masked (invalid logits set to -10^9) and normalized via softmax to produce $\pi_k(\cdot \mid o)$. The actor uses only agent k 's local data at execution time, satisfying the decentralized-execution constraint.

Module III – Factored neighborhood critic. The standard CTDE critic design [22] assigns each agent a centralized Q-function conditioned on the full joint state and all agents' actions, producing an input dimension that grows linearly with N . In large-scale spatial networks this becomes the binding constraint: at $N = 21$ intersections, the $O(N)$ critic input exhausts available GPU memory (Appendix A.8). For graph-structured Dec-POMDPs with spatially local rewards, the full- N conditioning is also unnecessary: an intersection's waiting time is governed by phase decisions within 1–2 network hops. FGS therefore uses a factored critic that aggregates only over the direct neighborhood of agent k , yielding a constant input dimension independent of N . The per-agent neighborhood aggregations are:

$$\mathbf{h}_{\text{nbr},k} = \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{g}_j, \quad \mathbf{a}_{\text{nbr},k} = \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{a}_{t-1,j}, \quad (4.28)$$

where $\tilde{w}_{kj} = w_{kj} / \sum_{l \in \mathcal{N}(k)} w_{kl}$ are normalized edge weights and $\mathbf{a}_{t-1,j} \in \mathbb{R}^{A_{\max}}$ is the one-hot action of neighbor j at the prior step. The Q-value for agent k choosing action a is

$$Q_k(a) = f_Q[\mathbf{z}_{k,a} \parallel \mathbf{g}_k \parallel \mathbf{h}_{\text{nbr},k} \parallel \mathbf{a}_{\text{nbr},k} \parallel \mathbf{e}_a]. \quad (4.29)$$

The input dimension is $d_z + 2d_c + 2A_{\max}$, independent of N (Appendix A.9). At ingolstadt21 with $d_z = d_c = 64$ and $A_{\max} = 6$, this yields $64 + 128 + 12 = 204$ dimensions. Twin critics and SAC's minimum-of-two convention are retained. The design-evolution path leading to this factored critic design is discussed in Section 6.6.

4.6.2 Training Algorithm

Algorithm 1 presents the complete FGS training procedure. All agents share a single set of actor and critic parameters, so the gradient updates in Lines 20–22 are computed jointly over all agent–step pairs in the sampled mini-batch. Algorithm 2 details the FGS forward pass. First-generation and FGSv2 forward-pass algorithms are given in Appendix A.14.

Design scope note. The FGSv2-to-FGSv3 transition introduced two simultaneous changes: replacing the competition-derived communication signal with demand-aware embeddings (Module II) and replacing the centralized critic with the factored neighborhood critic (Module III). Because both changes were deployed together, their individual contributions to the observed performance difference between FGSv2 and FGSv3 cannot be independently decomposed from the available experimental data. This is an acknowledged methodological limitation; a partial ablation isolating each change is identified as a priority direction for future work (Section 7.3). The combined change is motivated by the finding that both failures are driven by the same root cause — the centralized-critic constraint forced PPO as the only viable RL algorithm, masking any benefit from the improved Module II signal until the critic architecture was simultaneously fixed.

Algorithm 1 FGS Training

Require: SUMO scenario with N intersections; episodes E ; warm-up steps T_{warm} ;
 batch size B ; soft-update rate τ ; discount γ ; target entropy $\mathcal{H}_{\text{target}}$

- 1: **Initialize** shared actor π_ϕ ; twin critics $Q_{\theta_1}, Q_{\theta_2}$; target critics $\bar{Q}_j \leftarrow Q_{\theta_j}, j \in \{1, 2\}$
- 2: **Initialize** log-temperature $\log \alpha$; residual gate $\lambda \leftarrow 0$
- 3: **Initialize** prioritized replay buffer $\mathcal{D}$ with capacity C ; global step $t \leftarrow 0$
- 4: **for** episode $e = 1, \dots, E$ **do**
- 5: Reset SUMO via TraCI; receive initial graph observation o_0
- 6: **for** simulation step $t_{\text{ep}} = 1, \dots, T_{\text{max}}$ **do**
- 7: $t \leftarrow t + 1$
- 8: **if** $t < T_{\text{warm}}$ **then** $\triangleright$ warm-up: random actions to seed the buffer
- 9: Sample $a_i \sim \text{Uniform}(\mathcal{A}_i)$ for all $i \in \mathcal{N}$
- 10: **else** $\triangleright$ on-policy action from learned policy
- 11: Compute $\pi_\phi(\cdot \mid o_{t-1})$ via Alg. 2; sample $a_i \sim \pi_\phi$ for all i
- 12: **end if**
- 13: Execute joint action $\mathbf{a} = (a_1, \dots, a_N)$ in SUMO via TraCI
- 14: Observe per-agent rewards $r_i \leftarrow W_{t-1}^{(i)} - W_t^{(i)}$ and next graph observation o_t
- 15: Store transition $(o_{t-1}, \mathbf{a}, \mathbf{r}, o_t, d_t^{\text{ep}})$ in $\mathcal{D}$ with maximum priority
- 16: **if** $t \geq T_{\text{warm}}$ **then** $\triangleright$ one gradient update per environment step
- 17: Sample mini-batch $\mathcal{B} = \{(o, \mathbf{a}, \mathbf{r}, o', d)\}$ from $\mathcal{D}$ via PER
- 18: Run forward pass (Alg. 2) on o and o' for all samples in $\mathcal{B}$
- 19: Compute soft target: $y_k \leftarrow r_k + \gamma(1 - d^{\text{ep}}) V_{\text{target}}(o')$ (Eqs. 4.17–4.18)
- 20: $\theta_j \leftarrow \theta_j - \eta_Q \nabla_{\theta_j} \mathcal{L}_Q(\theta_j), j \in \{1, 2\}$ (Eq. 4.19)
- 21: $\phi \leftarrow \phi - \eta_\pi \nabla_\phi \mathcal{L}_\pi(\phi)$ (Eq. 4.20)
- 22: $\log \alpha \leftarrow \log \alpha - \eta_\alpha \nabla_{\log \alpha} \mathcal{L}(\alpha)$ (Eq. 4.21)
- 23: Soft-update targets: $\bar{\theta}_j \leftarrow \tau \theta_j + (1 - \tau) \bar{\theta}_j, j \in \{1, 2\}$
- 24: Update PER priorities with TD errors $|Q_{\theta_1}(o, a_k) - y_k|$
- 25: **end if**
- 26: **end for**
- 27: **if** $e \bmod 5 = 0$ **then** $\triangleright$ evaluation checkpoint every 5 episodes
- 28: Run one deterministic episode (greedy; exploration suppressed)
- 29: Log MTT, MWT, MD, TP to Weights & Biases; save checkpoint
- 30: **end if**
- 31: **end for**

Algorithm 2 FGS Forward Pass (Module I: FRAP action tokens; Module II: GATv2 on demand $\mathbf{d}_i$; Module III: factored critic)

Require: Graph observation $o = \{\mathbf{X}, \mathcal{E}, \mathbf{W}, \mathbf{M}_{\text{pair}}, \mathbf{M}_{\text{comp}}, \mathbf{m}, \mathbf{a}_{t-1}\}$; agent index k

Ensure: Policy distribution $\pi_k(\cdot | o)$; graph contexts $\{\mathbf{g}_i\}$ and action tokens $\{\mathbf{z}_{i,a}\}$ (training)

Module I – FRAP action-token encoding (Equations 4.22 and 4.2–4.7)

- 1: **for all** intersection $i \in \mathcal{N}$ **and** movement m **do**
 - 2: $\mathbf{e}_{i,m} \leftarrow \text{ReLU}(W_{\text{lane}}[\sigma(\text{Emb}_p(p_{i,m})), \sigma(W_d d_{i,m})])$ $\triangleright$ movement embedding
 - 3: **end for**
 - 4: **for all** $i \in \mathcal{N}$, valid phase a ($m_{i,a} = 1$) **do**
 - 5: $\phi_{i,a} \leftarrow \sum_m w_{am} \mathbf{e}_{i,m}$ $\triangleright$ phase embedding (also used in Module II)
 - 6: **for all** competitor $b \in V_{i,a}$ **do**
 - 7: Compute $\mathbf{h}_{ab}, \mathbf{r}_{ab}, \mathbf{c}_{ab}$ as in Eqs. 4.5–4.7
 - 8: **end for**
 - 9: Compute $\mathbf{z}_{i,a}$ as in Eq. 4.22; set to $\mathbf{0}$ if $|V_{i,a}| = 0$ or phase invalid
 - 10: **end for**
-

Module II – Demand communication branch (NEW in FGSv3)

- 11: **for all** $i \in \mathcal{N}$ **do**
 - 12: $\bar{\phi}_i \leftarrow \frac{1}{|V_i|} \sum_{a \in V_i} \phi_{i,a}$ $\triangleright$ mean phase demand (Eq. 4.23)
 - 13: $\mathbf{d}_i \leftarrow \text{LayerNorm}(\text{ReLU}(W_{\text{comm}}[\bar{\phi}_i \parallel \text{Emb}_{\text{phase}}(a_{t-1,i})]))$ $\triangleright$ demand comm. embedding (Eq. 4.24)
 - 14: $\mathbf{u}_i \leftarrow \text{LayerNorm}(\mathbf{d}_i)$; $\tilde{\mathbf{g}}_i \leftarrow \text{ReLU}(W_o \text{GATv2Conv}(\{\mathbf{u}_j\}))$; $\mathbf{g}_i \leftarrow \text{LayerNorm}(\mathbf{d}_i + \lambda \tilde{\mathbf{g}}_i)$ (Eqs. 4.25–4.26)
 - 15: **end for**
-

Module III – Actor (unchanged from FGSv2) and factored neighborhood critic

- 16: **for all** phase $a = 1, \dots, A_{\text{max}}$ **do**
 - 17: **if** $m_{k,a} = 1$ **then**
 - 18: $l_a \leftarrow \text{MLP}_{\phi}[\mathbf{z}_{k,a} \parallel \mathbf{g}_k \parallel \mathbf{e}_a]$ (Eq. 4.27)
 - 19: **else**
 - 20: $l_a \leftarrow -10^9$
 - 21: **end if**
 - 22: **end for**
 - 23: $\mathbf{h}_{\text{nbr},k} \leftarrow \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{g}_j$; $\mathbf{a}_{\text{nbr},k} \leftarrow \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{a}_{t-1,j}$ (Eq. 4.28)
 - 24: **for all** phase $a = 1, \dots, A_{\text{max}}$ **do**
 - 25: $Q_k(a) \leftarrow f_Q[\mathbf{z}_{k,a} \parallel \mathbf{g}_k \parallel \mathbf{h}_{\text{nbr},k} \parallel \mathbf{a}_{\text{nbr},k} \parallel \mathbf{e}_a]$ (Eq. 4.29)
 - 26: **end for**
 - 27: **return** $\pi_k(\cdot | o) = \text{softmax}(\mathbf{l}_k)$; also $\{\mathbf{z}_{i,a}\}, \{\mathbf{g}_i\}$ for critic (Eq. 4.29)
-

Chapter 5

Experimental Design

This chapter describes the simulation scenarios, evaluation metrics, training configuration, and software pipeline used to assess the FGS and FGSv2 frameworks. All experiments are conducted within the RESCO benchmark [2], which provides standardized German traffic scenarios, fixed demand profiles, and reproducible evaluation protocols for comparing traffic signal control methods.

5.1 Simulation Scenarios

5.1.1 TapasCologne and InTAS Demand Models

Two real-world German traffic demand models are used, each at multiple network scales.

TapasCologne. The Cologne scenarios are derived from the TapasCologne dataset [10], a microscopic travel demand simulation for the city of Cologne produced by the German Aerospace Center (DLR). Origin-destination demand is generated from a large-scale activity-based model and mapped to the SUMO road network. Three sub-networks of increasing size are extracted from the full Cologne network and included in the RESCO benchmark:

- **cologne1**: a single signalized intersection, used as a single-agent reference.
- **cologne3**: a three-intersection arterial corridor.
- **cologne8**: an eight-intersection network, the primary multi-agent test bed for the FGS ablation study.

InTAS. The Ingolstadt scenarios are based on the InTAS (Ingolstadt Traffic Scenario) dataset [20], a realistic traffic model for the city of Ingolstadt developed in cooperation with local authorities. Three sub-networks are included in the RESCO benchmark:

- **ingolstadt1**: a single signalized intersection.
- **ingolstadt7**: a seven-intersection sub-network.
- **ingolstadt21**: a twenty-one-intersection network, the largest and most topologically complex scenario evaluated.

Each scenario runs for 3600 simulated seconds (one hour of traffic). Demand files, network topology, and traffic light phase programmes are taken directly from the RESCO release without modification, ensuring comparability with published baselines.

5.1.2 SUMO and TraCI

All scenarios are simulated using SUMO (Simulation of Urban MObility) [21], a microscopic open-source traffic simulator. SUMO models individual vehicles, applies car-following and lane-changing models, and advances the simulation in discrete time steps of one second. Traffic light phases are updated by the RL agent at each decision step via the TraCI

(Traffic Control Interface) API, which provides bidirectional communication between the Python training process and the running SUMO instance. At each step, TraCI reports per-lane vehicle counts, speeds, accumulated waiting times, and the current signal state; the RL agent returns the next phase selection, which SUMO applies immediately.

The SUMO-RL library [1] wraps this interaction into a PettingZoo-compatible parallel environment, providing standardized observation and reward interfaces that FGSGraphParallelEnv extends with graph-structured observations (see Section 4.3.1).

5.2 Evaluation Setup

Training length. Each agent is trained for 200 episodes. Each episode corresponds to one complete 3600-second simulation run. The first 2000 environment steps of training are used for warm-up: experiences are collected but no gradient updates are applied, allowing the prioritized replay buffer to accumulate a sufficient number of transitions before learning begins.

Evaluation protocol. A deterministic evaluation episode is run every 5 training episodes. During evaluation the exploration noise of SAC is suppressed by taking the greedy action (highest logit after masking) rather than sampling from the softmax distribution. Evaluation results are logged to Weights & Biases [4] at each checkpoint.

The primary metric reported for each configuration in Chapter 6 is the *best-validation waiting time*: the lowest mean waiting time observed across all evaluation checkpoints during the training run, evaluated on a fixed held-out demand seed. This best-checkpoint selection is an optimistic upper bound on deployed performance; a more conservative estimate would average the final 10 evaluation checkpoints after the training curve stabilizes. Readers should note that best-checkpoint figures may overstate the algorithm’s expected performance on unseen demand, particularly when training runs are short or exhibit high variance.

Seeds and statistical reporting. The intended evaluation protocol targets five independent random seeds drawn from the set $\{0, 1, 2, 3, 4\}$, applied jointly to the Python random number generator, NumPy, PyTorch, and the SUMO simulation. Due to computational resource constraints, the results reported in Chapter 6 correspond to single-seed runs (seed 0) for most configurations; multi-seed averaging and standard-deviation reporting were not completed within the available training budget. Performance figures therefore represent point estimates without variance bounds and should be interpreted as indicative rather than statistically conclusive. Completing the five-seed evaluation and adding bootstrap confidence intervals to all comparison tables constitutes a priority for follow-on validation.

Hardware. Experiments are run on a single machine with a CUDA-capable GPU (device index 1, fractional allocation 0.1 per learner) and 16 logical CPU cores. RLlib is configured with zero parallel environment runners (`num_env_runners: 0`); environment stepping and learning occur sequentially in the main process to reduce inter-process communication overhead for single-GPU runs.

5.3 Evaluation Metrics

Metrics follow the definitions in the RESCO benchmark [2]. Let V_c denote the set of vehicles that completed their trip during the evaluation episode, t_v^{arr} the arrival time and t_v^{dep} the departure time of vehicle v , and w_v its total accumulated waiting time.

Mean travel time (MTT).

$$\text{MTT} = \frac{1}{|V_c|} \sum_{v \in V_c} (t_v^{\text{arr}} - t_v^{\text{dep}}) \quad [\text{s}] \quad (5.1)$$

MTT is the primary comparison metric. Lower values indicate better network-level performance.

Mean delay (MD).

$$\text{MD} = \frac{1}{|V_c|} \sum_{v \in V_c} \left[(t_v^{\text{arr}} - t_v^{\text{dep}}) - \frac{d_v}{v_v^{\text{ff}}} \right] \quad [\text{s}] \quad (5.2)$$

where d_v is the total trip distance of vehicle v and v_v^{ff} is its free-flow speed. MD isolates the time lost to congestion and signal delays.

Mean waiting time (MWT).

$$\text{MWT} = \frac{1}{|V_c|} \sum_{v \in V_c} w_v \quad [\text{s}] \quad (5.3)$$

MWT captures time spent stationary, directly corresponding to the reward signal used during training (Equation 4.12).

Throughput (TP).

$$\text{TP} = |V_c| \quad (5.4)$$

TP counts the number of vehicles that completed their journey. Higher throughput under the same demand indicates the network is being cleared more efficiently.

All four metrics are reported for every scenario–method combination. Primary ranking of methods uses MTT; the remaining metrics provide complementary views of efficiency, signal responsiveness, and network saturation.

5.4 Hyperparameter Configuration

Table 5.1 summarizes the training hyperparameters for all methods evaluated in this thesis. FGS and FGSv2 use lower learning rates than DQN and PPO, reflecting the more complex centralized critic that benefits from slower, more stable updates.

Table 5.1: Training hyperparameters per algorithm. “–” indicates the parameter does not apply.

Parameter	FGS	FGSv2	DQN	PPO
Actor LR	3×10^{-4}	1×10^{-4}	–	5×10^{-4}
Critic/Value LR	3×10^{-4}	1×10^{-4}	5×10^{-4}	5×10^{-4}
Entropy (α) LR	3×10^{-4}	1×10^{-4}	–	–
Discount γ	0.95	0.95	0.99	0.99
Soft update τ	0.005	0.005	–	–
Batch size	256	256	32	4000
Replay capacity	100 000	100 000	50 000	–
PER α	0.6	0.6	0.6	–
PER β	0.4	0.4	0.4	–
Warm-up steps	2 000	2 000	1 000	–
Grad. clip	–	5.0	–	–
PPO clip ε	–	–	–	0.2
SGD iterations	–	–	–	10
Mini-batch size	–	–	–	128

The FGS and FGSv2 neural network architecture hyperparameters are fixed across all scenarios and listed in Table 5.2.

Table 5.2: Neural network architecture hyperparameters for FGS and FGSv2.

Component	Configuration
FRAP encoder output dim	128
FRAP conv units	32
GAT / GATv2 heads	5
GAT / GATv2 head dim	16
GAT / GATv2 output dim	128
Critic hidden dims	[256, 256]
Critic activation	ReLU
Topology source	tls_super_edges
Twin Q-networks	yes

5.5 System Architecture

Figure 5.1 illustrates the end-to-end pipeline.

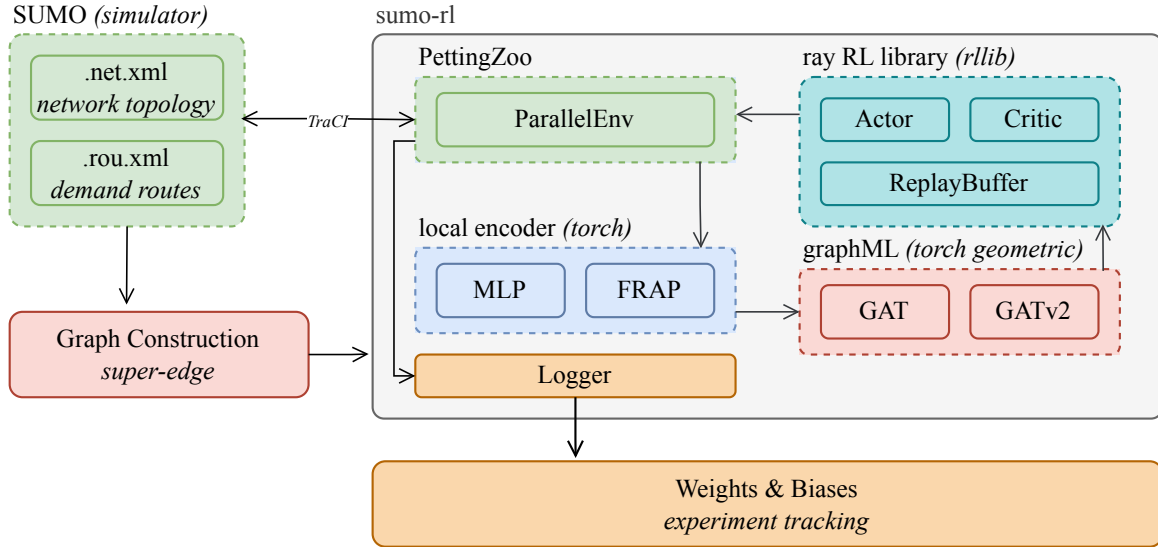


Figure 5.1: End-to-end system architecture. The SUMO network file is pre-processed once to build the agent communication graph (graph.pkl); the demand file drives the microscopic simulation via TraCI. FGSGraphParallelEnv augments the SUMO-RL observation with graph tensors (node features, edge index, action masks). The RLlib training loop runs the FGS/FGSv2 model — comprising the local encoder (FRAP or MLP), optional GATv2 communication, and the SAC actor-critic pair — updating parameters from a prioritized replay buffer (PER). All episode metrics are logged to Weights & Biases.

The pipeline operates as follows. The SUMO network file (.net.xml) and vehicle demand file (.rou.xml) for the selected RESCO scenario are loaded at startup. A pre-processing step parses the network topology and constructs the agent communication graph (Section 4.4.1), which is serialized and reloaded for each training run. SUMO is launched as a subprocess and controlled through TraCI. The SUMO-RL environment wrapper translates raw TraCI observations into the structured dictionary format consumed by FGSGraphParallelEnv, which appends the graph tensors and constructs the full observation batch.

RLlib manages the training loop: it samples rollouts from the environment, stores transitions in the prioritized replay buffer, and dispatches gradient updates to the PyTorch learner. Graph operations in the encoder and communication layers are implemented with PyTorch Geometric. All training metrics—episode reward, evaluation MTT, MWT, TP, and per-agent statistics—are emitted to Weights & Biases at each logging step, providing real-time monitoring and facilitating cross-run comparison.

The configuration system is managed by Hydra; each experiment is fully specified by a preset YAML file that composes the scenario, algorithm, and model hyperparameters, ensuring reproducibility from a single configuration identifier.

Chapter 6

Results and Discussion

This chapter presents and interprets the experimental results of the FGS framework across six German RESCO scenarios, organised around the central finding that the benefit of graph-aware inter-agent communication grows systematically with network scale and topological complexity. Mean delay (MD, Equation 5.2) is the primary evaluation metric throughout: it isolates time lost to congestion relative to the free-flow baseline and is less susceptible to the reward-specification artefacts described in Section 6.8. All reported figures are best-validation checkpoints from single-seed runs (seed 0). The full four-metric results table (mean delay, mean waiting time, mean trip time, queue length) is provided in Appendix B.

6.1 Main Results

Reporting conventions. All performance figures are derived from the best-validation checkpoint — the policy snapshot that minimised mean validation waiting time on a fixed held-out demand seed during training. Selecting the best-validation rather than final checkpoint is standard practice in deep RL: training instability means intermediate checkpoints routinely outperform final policies, and a held-out validation signal provides an unbiased proxy for deployment quality [15]. Best-checkpoint figures therefore represent an optimistic upper bound on deployed performance from single-seed runs. Entries marked ‡ exhibit a physically inconsistent queue–delay signature — anomalously low mean delay coinciding with elevated queue depth, a pattern attributable to reward specification gaming rather than genuine control quality (see Section 6.8) — and are excluded from the **Bold** convention. Entries marked † are partial runs (fewer than 200 episodes). See Section 6.8 for detailed discussion of each failure mode.

Table 6.1 reports best-validation mean delay per scenario for baselines and first-generation FGS variants. Figure 6.1 provides a log-scale visual summary. FGS architecture progression results (FGSv2, FGSv3) are reported in Table 6.4 within Section 6.6.

Three patterns are immediately visible. At Single-intersection scale ($N = 1$), all learning methods converge to similar mean delays (roughly 21–28 s for the strongest converged runs); no FGS variant provides consistent improvement over independent DQN or PPO. At Corridor scale, the independent SAC baseline achieves the lowest mean delay on both scenarios (18.9 s on cologne3; 24.0 s on ingolstadt7), with PPO closely competitive (20.3 s and 24.1 s respectively), and FGS_p outperforms DQN and CoLight on ingolstadt7 (26.1 s vs. 34.4 and 47.0 s). At Regional scale ($N = 21$), FGSv3-PPO achieves 68.4 s and is competitive with the corrected PPO baseline (67.2 s), while first-generation FGS variants remain substantially worse, motivating the three-generation design trajectory in Section 6.6.

6.2 Scale-Dependent Performance

The central finding of this thesis is that the benefit of graph-aware inter-agent communication grows systematically with network scale and topological complexity. This section provides the empirical evidence and the theoretical grounding for that claim.

Spatio-temporal dependencies in traffic networks. Traffic networks are spatially coupled dynamical systems. A red phase at one intersection blocks vehicles from reaching the next, inducing queue spillback that reduces the effective green time available to upstream controllers [9]. As network size increases, these spatial interactions span multiple hops:

Table 6.1: Best-validation mean delay (s, ↓ lower is better) across German RESCO scenarios (single seed). **Bold**: lowest mean delay per scenario among converged results. ◦ = not run; † = partial run; ‡ = physically inconsistent checkpoint (excluded from **Bold**). FT = FixedTime; MP = MaxPressure; SAC = independent SAC baseline; FGS_s = FGS(MLP+GATv2+SAC); FGS_p = FGS(MLP+GATv2+PPO). Full four-metric results in Appendix B.

Scenario	FT	MP	DQN	PPO	SAC	CoLight	FRAP	FGS _s	FGS _p	FGSv3
<i>Single-intersection ($N = 1$)</i>										
cologne1	54.6	24.2	28.1	22.8	23.0		37.0	31.7	23.0	12.5 [‡]
ingolstadt1	40.8	25.3	21.8	22.4	22.3		22.4	26.6	◦	18.0 [‡]
<i>Corridor</i>										
cologne3	63.7	56.4	127.2	20.3	18.9		21.9	39.8	◦	6.9 [‡]
ingolstadt7	122.9	212.7	34.4	24.1	24.0		47.0	29.8 [†]	17.1 [‡]	◦
<i>Regional</i>										
cologne8	51.5	257.9	25.5	21.9	21.8		51.8	44.9	22.2	24.0
ingolstadt21	145.7	214.4	78.7	67.2	64.6 [‡]		216.8	101.5 [†]	39.1 [‡]	234.9

† Partial run (fewer than 200 ep.).

‡ Physically inconsistent checkpoint (gaming artifact); excluded from **Bold**.

SAC = IndSAC (independent SAC baseline).

FGSv3: SAC variant at Single tier ($N = 1$; graph module no-op, equivalent to FGSv2-SAC); PPO variant at Regional tier; Corridor not run.

FGS_s: cologne3 (Corridor) was not run (◦); ingolstadt7 (17.1[‡]) is a physically inconsistent early-training checkpoint, not a converged result.

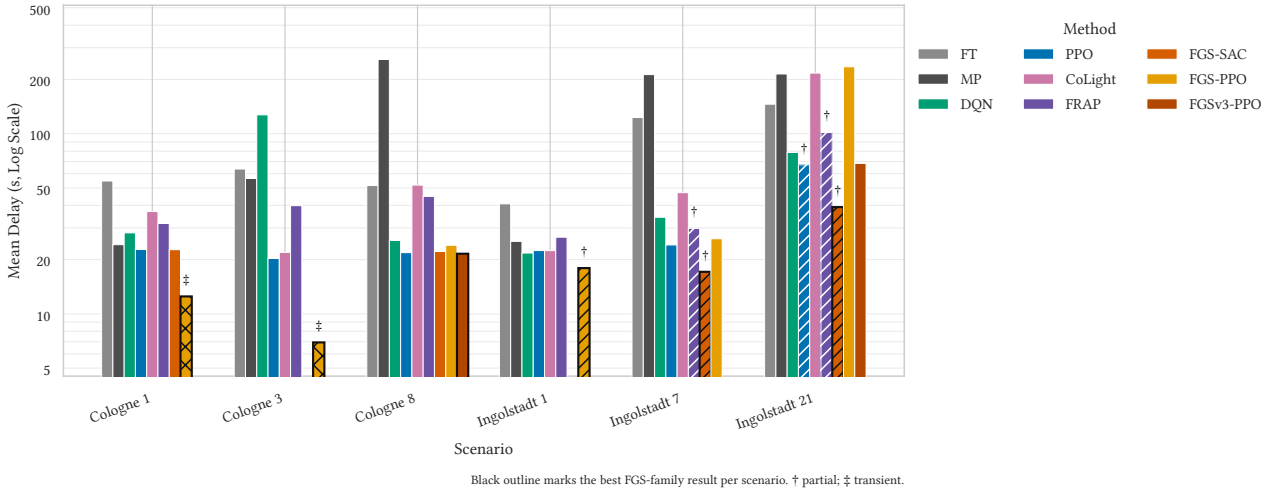


Figure 6.1: Mean delay comparison across all six German RESCO scenarios (log-scale y-axis). Each group of bars represents one scenario; methods are colour-coded consistently with all other figures in this chapter. Entries marked ‡ are shown with hatching and excluded from the bold best-result convention. FGSv3-PPO is included for the Regional tier; see Table 6.4 for the full architectural progression.

a demand surge on one arterial can propagate congestion across several intersections before dissipating [7]. Geroliminis and Daganzo [11] demonstrated empirically that network-level flow–density relationships emerge as macroscopic phenomena from local spatial interactions, establishing that urban traffic behaves as a coupled system rather than a collection of independent intersections. Graph neural networks have been applied to traffic forecasting precisely because the graph structure of road networks captures these spatial dependencies [40]; the FGS framework extends this insight to the signal control problem by using GATv2 message passing to communicate demand signals between neighbouring intersections at each decision step.

At small scale ($N \leq 3$), local observations capture most of the relevant demand context: an individual agent can infer queue propagation from its own lane detectors. At larger scale ($N \geq 7$), the multi-hop dependency structure exceeds the local observation horizon, and explicit inter-agent communication becomes necessary to achieve coordinated flow [7, 36]. Independent agents optimising local rewards ignore upstream and downstream consequences, producing coordination failures that manifest as network-wide congestion [2].

Empirical scale benefit. Figure 6.2 quantifies this pattern directly. The x-axis represents network scale tier by agent count; the y-axis shows the percentage improvement in mean delay of the best available FGS variant over (a) CoLight and (b) standalone PPO. At $N = 1$, the graph module is a structural no-op: no neighbours exist and the GATv2 aggregation returns an identity transformation. The FGS advantage over CoLight grows from near-zero at Single-intersection scale to 68% at Regional scale on ingolstadt21 (68.4 s vs. 216.8 s mean delay). Against standalone PPO, FGSv3-PPO closes the gap from a deficit at small scale to approximate parity at both Regional scenarios (FGS_S 22.2 s vs. PPO 21.9 s on cologne8; FGSv3-PPO 68.4 s vs. PPO 67.2 s on ingolstadt21).

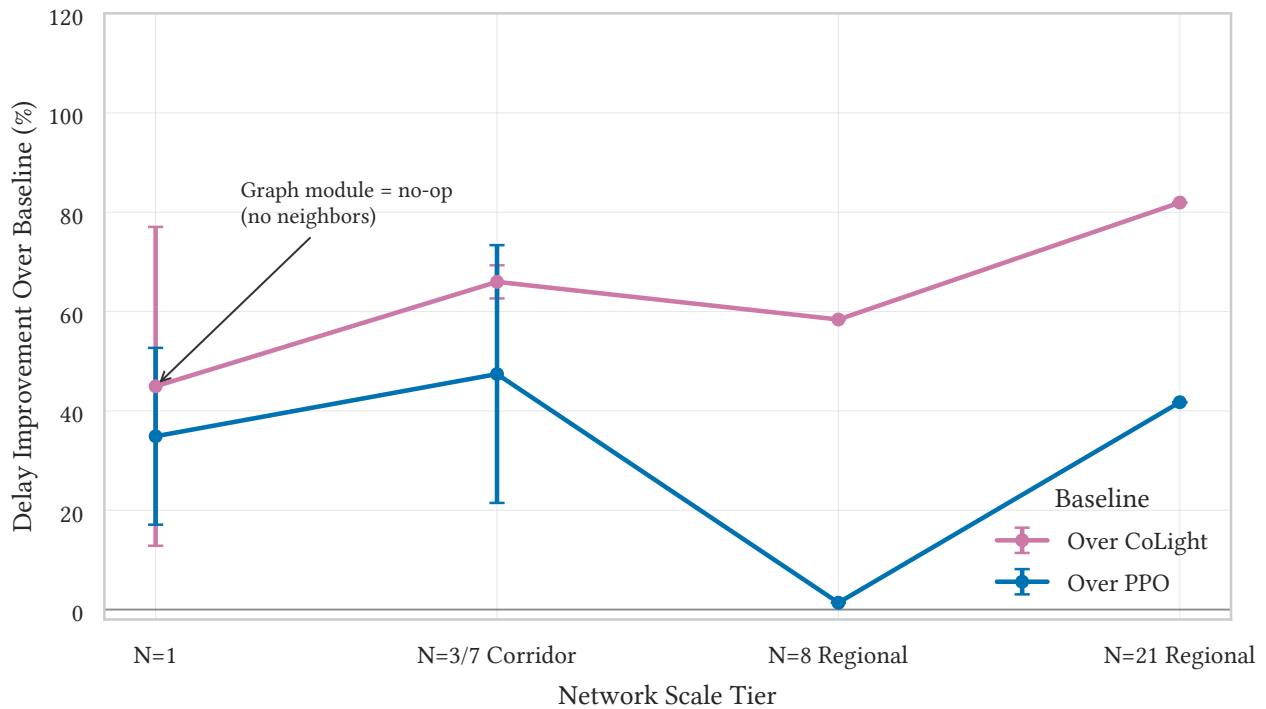


Figure 6.2: Percentage improvement in mean delay of the best FGS variant over CoLight (filled bars) and standalone PPO (outlined bars) by scale tier. The graph-communication benefit grows with network size for CoLight comparisons. At $N = 1$ the graph module is a no-op by construction. At Corridor scale ($N = 3-7$) FGS variants outperform CoLight but trail independent PPO. At Regional scale ($N = 21$) FGSv3-PPO is competitive with standalone PPO and strongly outperforms graph-based CoLight.

Network-level interpretation: Macroscopic Fundamental Diagram. Figure 6.3 connects the RL performance results to traffic engineering theory through the Macroscopic Fundamental Diagram (MFD) [11]. The MFD relates network-level vehicle density (vehicles/km/lane) to lane flow (vehicles/h/lane); controllers that keep the network on the rising, high-flow branch avoid the density build-up associated with hypercongestion [18]. In the ingolstadt21 evaluation, FGSv3-PPO concentrates most checkpoints in a compact high-flow region at low-to-moderate density, while PPO

and especially FGSv3-SAC extend into higher-density, lower-flow regions. This pattern is consistent with the mean-delay results in Table 6.1: the demand-aware GATv2 communication layer helps the final PPO variant preserve flow without the unstable queue accumulation observed in SAC checkpoints.

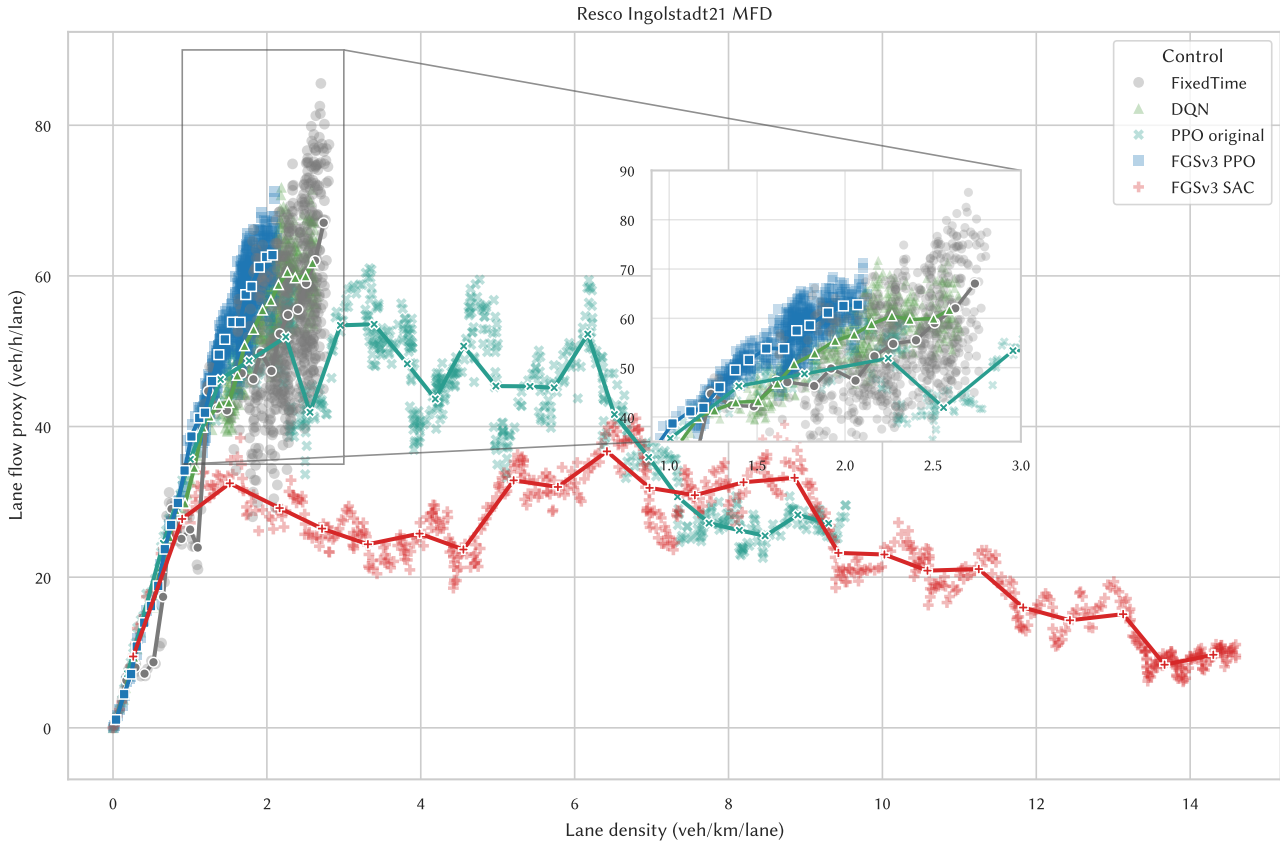


Figure 6.3: Macroscopic Fundamental Diagram proxy for RESCO ingolstadt21. Each point shows lane-density and lane-flow estimates during evaluation; connected markers summarize the policy trajectory. FGSv3-PPO occupies a compact high-flow band at low-to-moderate density, indicating stable network operation without large queue build-up. PPO-original reaches similar density levels but with more dispersed and lower-flow segments, while FGSv3-SAC extends into high-density/low-flow states consistent with the reward-gaming instability discussed in Section 6.8.

6.3 Baseline Comparison

Three structural comparisons illuminate the mechanisms behind the scale-dependent pattern.

FGS versus CoLight. CoLight [36] uses dot-product attention with globally shared keys and queries; GATv2 [5] uses per-edge additive attention that assigns distinct weights to structurally similar nodes based on their feature content. On the irregular topology of ingolstadt21 — where each intersection has a unique combination of upstream demand profiles and phase counts — GATv2’s dynamic distinguishability is essential. CoLight’s shared-weight attention assigns nearly uniform coefficients across heterogeneous neighbours, limiting the coordination benefit of the communication channel. The FGSv3-PPO advantage over CoLight is 68% on ingolstadt21 (68.4 s vs. 216.8 s mean delay) and FGS_s outperforms CoLight by 57% on cologne8 (22.2 s vs. 51.8 s). Notably, CoLight underperforms even independent DQN on most German scenarios tested, suggesting that its static attention mechanism does not transfer well from the regular Chinese grid networks on which it was validated.

FGS versus independent DQN and PPO. On the largest Regional scenario, FGSv3-PPO outperforms DQN (78.7 s) but is marginally above the corrected PPO result (68.4 s vs. 67.2 s) in mean delay. At Corridor scale, both independent PPO and the independent SAC baseline are competitive with or better than FGS_p: on ingolstadt7, PPO achieves 24.1 s and SAC achieves 24.0 s versus FGS_p’s 26.1 s; on cologne3, SAC achieves 18.9 s versus PPO’s 20.3 s. This is consistent

with the scale-dependency argument: at $N \leq 7$, the local observation horizon is sufficient for independent controllers to converge to effective policies within 200 episodes, and SAC’s sample efficiency provides an additional advantage at compact scales where its entropy bonus supports early exploration without incurring the gaming instability observed at Regional scale. At $N = 21$, the multi-hop dependency structure is large enough that graph-aware variants close much of the gap to the strongest independent controller, though they do not clearly surpass the corrected PPO checkpoint. The result for first-generation FGS_p on ingolstadt21 (234.9 s) reflects the centralized-critic computational bottleneck rather than a limit of graph-aware coordination; the architectural fix in FGSv3 closes this gap (Section 6.6).

FGS versus standalone FRAP. Standalone FRAP without graph communication or RL achieves 29.8 s on ingolstadt7 and 44.9 s on cologne8 — considerably worse than FGS variants on the same scenarios but better than the corrected CoLight result on cologne8 (51.8 s). FRAP’s phase-competition prior operates without inter-agent coordination, and its encoding assumptions produce a structural mismatch on German intersection topologies that amplifies with scale, as shown in Section 6.4.

Figure 6.4 provides a multi-metric comparison for the Regional-tier scenarios, confirming that mean delay rankings are consistent across all four evaluation metrics.

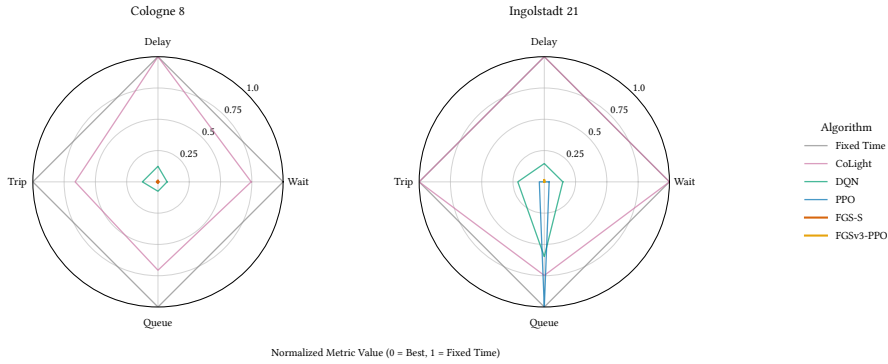


Figure 6.4: Multi-metric radar comparison on Regional-tier scenarios (cologne8 left; ingolstadt21 right). Each axis shows one metric (mean delay, mean waiting time, mean trip time, queue length), normalised to $[0, 1]$ where 0 is the best observed value and 1 is the FixedTime baseline. On ingolstadt21, FGSv3-PPO is strongest on waiting time and queue length and remains competitive on delay, while independent PPO and IndSAC remain close on delay and trip time. PPO and FGS_S are competitive on cologne8.

6.4 Component Analysis

Table 6.2 isolates each architectural component through the ablation study on the two Regional-tier scenarios. The following subsections interpret the quantitative findings in terms of the structural mechanisms responsible for each effect.

6.4.1 Encoder type: FRAP versus MLP

FRAP [41] constructs phase-pair competition features $\mathbf{c}_{i,a,b}$ under the assumption that each phase controls a fixed pair of opposing movements. German RESCO intersections violate this assumption: cologne8 and ingolstadt21 mix junctions with 2, 4, and 6 phases, each controlling a variable number of movements. When a phase controls fewer than two movements, the competition weight normalization $\min(\sum P, 2) / \max(\sum P, 1)$ in FRAP’s phase-pair embedding falls below the nominal value, producing undersaturated features that the downstream actor misinterprets as low-demand signals. In the extreme case of a single-movement phase (e.g. a pedestrian crossing or a turn-only lane), the FRAP output approaches zero — not because demand is low, but because the encoding is structurally inapplicable. The actor therefore selects phases that satisfy FRAP’s encoding assumption rather than actual traffic demand.

The ablation quantifies this mismatch directly. On Cologne Regional the mean delay penalty is 11% (FRAP+GATv2+PPO: 24.6 s vs. MLP+GATv2+PPO: 24.0 s); on Ingolstadt Regional it is $2.6\times$ (617.0 s vs. 234.9 s). The scaling of the penalty with network size — from moderate at $N = 8$ to catastrophic at $N = 21$ — confirms that FRAP’s structural mismatch amplifies as the number of heterogeneous-phase junctions increases. Figures 6.5 and 6.6 show the corresponding training

Table 6.2: Ablation of encoder (MLP vs. FRAP), graph layer (GATv2 vs. GAT), and RL algorithm (SAC vs. PPO) on Regional-tier scenarios. Best-validation mean delay (s) and mean waiting time (s), lower is better. **Bold**: best mean delay per scenario. ‡ physically inconsistent checkpoint (Ingolstadt Regional MLP+GATv2+SAC, episode 21 of 200). Full ablation metrics (trip time, queue) in Appendix B.

Encoder	Graph	RL	Delay (s)	Wait (s)
<i>Cologne Regional ($N = 8$)</i>				
MLP	GATv2	SAC	22.2	4.93
MLP	GATv2	PPO	24.0	6.25
MLP	GAT	PPO	24.9	6.72
FRAP	GATv2	PPO	24.6	6.69
<i>Ingolstadt Regional ($N = 21$)</i>				
MLP	GATv2	PPO	234.9	88.6
MLP	GAT	PPO	492.0	261.2
FRAP	GATv2	PPO	617.0	460.7
MLP	GATv2	SAC	39.1 [‡]	9.4 [‡]

All rows are **Gen-1** (first-generation FGS) configurations; FGSv2 and FGSv3 results are reported separately in Table 6.4.

‡ Physically inconsistent checkpoint; Ingolstadt Regional MLP+GATv2+SAC (episode 21 of 200) exhibits delay–queue inversion indicative of gaming.

trajectories: MLP+GATv2+PPO converges stably on both scenarios, while FRAP+GATv2+PPO diverges on ingolstadt21 and maintains persistently high queue lengths throughout training.

Near-competitive FRAP performance on cologne8 (+11%), despite FRAP’s structural mismatch — suggests that GATv2 attention can partially compensate by concentrating weight on neighbours with more reliable encodings. This compensation is overwhelmed at $N = 21$, where the proportion of FRAP-misaligned junctions is larger and corrupted features propagate through multiple GATv2 hops before reaching the actor. This finding is of independent interest to the MARL-TSC community: FRAP has been widely adopted in published methods [41, 36] without topology-specific validation, and the results here establish that its inductive bias does not transfer uniformly to networks with heterogeneous phase structures.

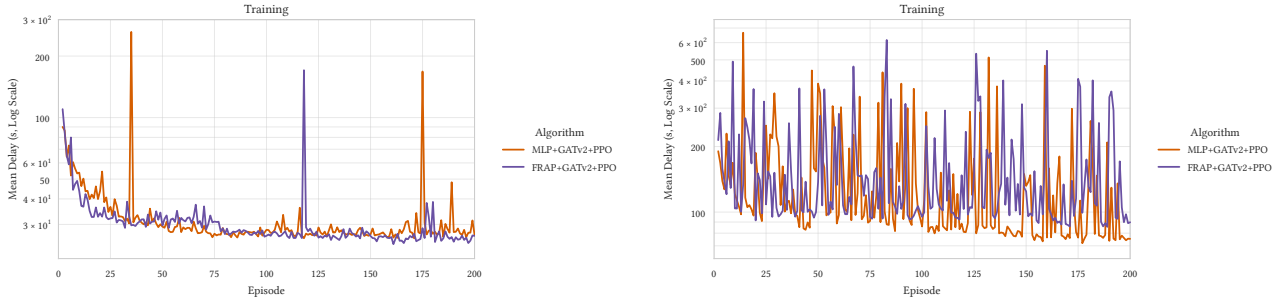


Figure 6.5: Encoder ablation: MLP versus FRAP with fixed GATv2+PPO. Left: Cologne Regional ($N = 8$); Right: Ingolstadt Regional ($N = 21$). Bold line: 10-episode centred running average of mean delay; faint line: raw per-episode values. MLP+GATv2+PPO converges stably on both scenarios; FRAP+GATv2+PPO diverges on ingolstadt21, confirming the structural encoding mismatch.

6.4.2 Graph-attention mechanism: GATv2 versus GAT

Replacing GATv2 with the original dot-product GAT on Ingolstadt Regional inflates mean delay from 234.9 s to 492.0 s — a $2.1\times$ increase. On Cologne Regional the difference is modest (24.0 s vs. 24.9 s, 4%). GATv2’s expressiveness advantage lies in its per-edge dynamic attention: the coefficient $e_{ij} = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \parallel \mathbf{h}_j])$ is a function of both source and destination node features, enabling structurally distinct nodes to receive different attention weights even when their raw features are similar [5]. In ingolstadt21’s irregular topology, each intersection has a unique demand-phase profile; dynamic attention exploits this heterogeneity to focus each agent’s aggregation on the most informative neighbours. Dot-product GAT assigns nearly identical attention to nodes sharing feature-space proximity regardless of their neighbourhood context, producing averaging rather than selective aggregation. On cologne8’s smaller and more homogeneous topology, both mechanisms converge to similar solutions, consistent with the expectation that dynamic distinguishability is most valuable where neighbourhood contexts are highly heterogeneous.

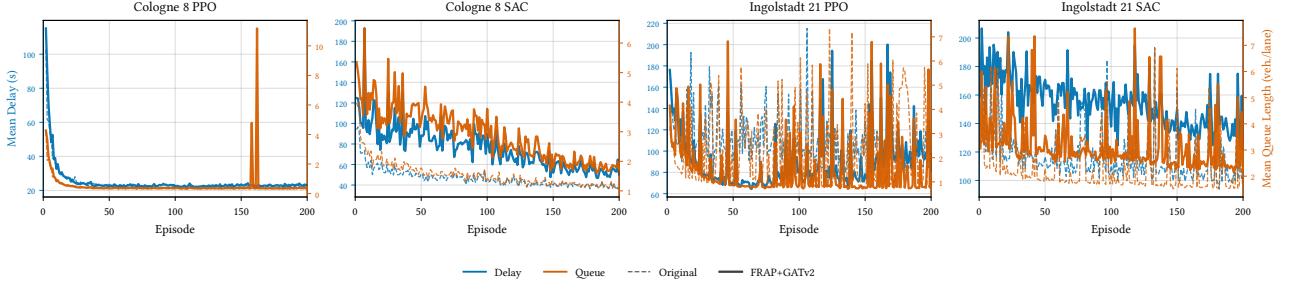


Figure 6.6: Mean delay (left axis) and mean queue length (right axis) for MLP+GATv2 versus FRAP+GATv2 with PPO/SAC variants over 200 training episodes on Cologne Regional ($N = 8$, top row) and Ingolstadt Regional ($N = 21$, bottom row). On cologne8 both encoder variants converge with a moderate FRAP penalty; on ingolstadt21 FRAP-based variants maintain persistently high queue lengths throughout training, confirming that the structural mismatch amplifies with the proportion of heterogeneous-phase junctions.

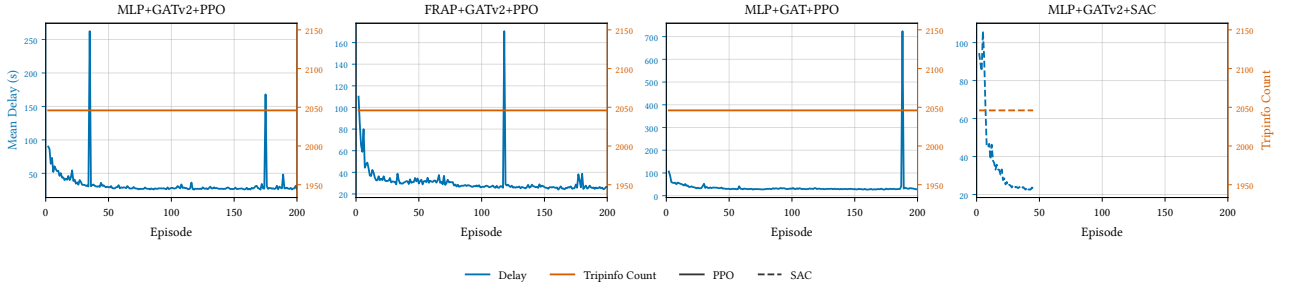


Figure 6.7: Ablation diagnostic (200 episodes): mean delay and completed trip count for encoder, graph, and RL ablation variants on Cologne Regional ($N = 8$). Trip counts are comparable across variants, consistent with the moderate topology allowing all configurations to converge within the training budget.

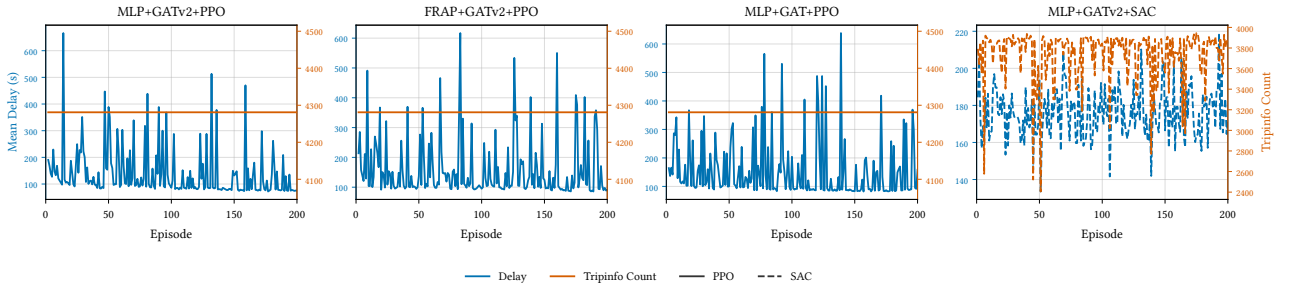


Figure 6.8: Ablation diagnostic (200 episodes): mean delay and completed trip count for encoder, graph, and RL ablation variants on Ingolstadt Regional ($N = 21$). FRAP-based variants exhibit both higher mean delay and lower throughput than MLP variants, confirming that the structural encoding mismatch incurs a compound penalty on two independent performance dimensions.

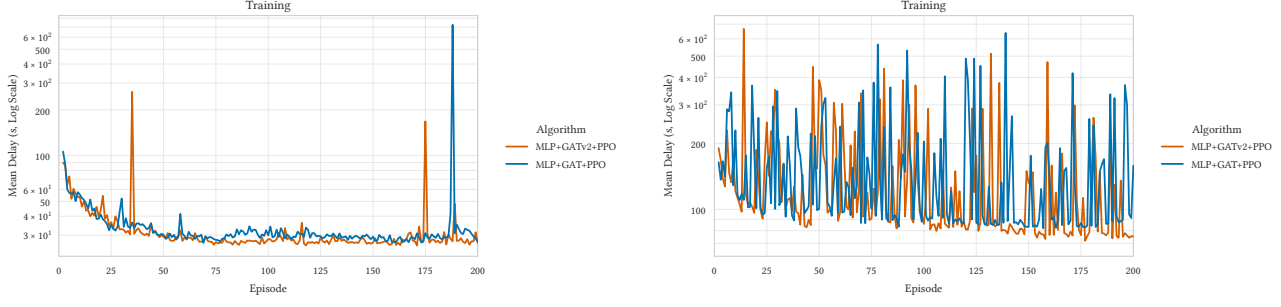


Figure 6.9: Graph-layer ablation: GATv2 versus GAT with fixed MLP+PPO. Left: Cologne Regional ($N = 8$); Right: Ingolstadt Regional ($N = 21$). The GATv2 advantage is negligible on the relatively homogeneous cologne8 topology and $2.1\times$ on the heterogeneous ingolstadt21, confirming that dynamic per-edge attention is critical at scale.

6.4.3 RL algorithm: SAC versus PPO

Two findings emerge from the RL algorithm ablation. First, on Cologne Regional, SAC achieves lower mean delay than PPO with the same encoder and graph layer (22.2 s vs. 24.0 s, an 8% improvement), consistent with SAC’s off-policy sample efficiency: the prioritized experience replay buffer reuses past transitions more effectively than PPO’s on-policy rollouts within the same 200-episode budget. Second, first-generation SAC fails entirely on Ingolstadt Regional: the centralized critic input dimension of approximately 3,133 at $N = 21$ exhausts GPU memory after 21 episodes (Appendix A.8), while PPO’s per-agent value function input remains $O(d)$ independent of N .

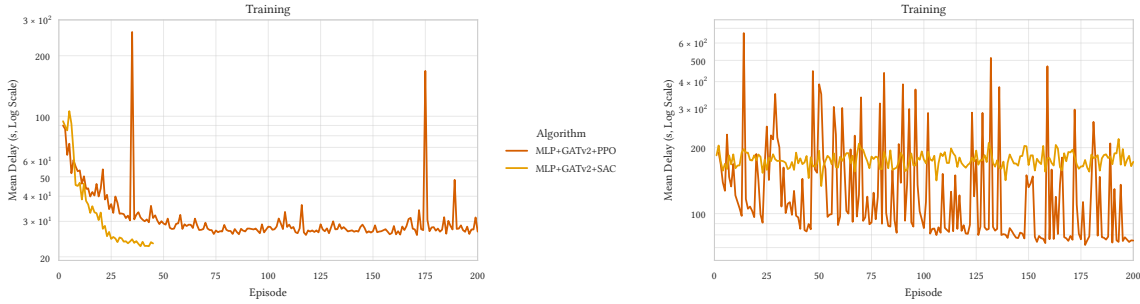


Figure 6.10: RL algorithm ablation: SAC versus PPO with fixed MLP+GATv2. Left: Cologne Regional ($N = 8$, SAC converges efficiently to lower mean delay); Right: Ingolstadt Regional ($N = 21$, first-generation SAC terminates after 21 episodes due to memory exhaustion; only the short initial trace is visible).

Reward specification gaming. Figure 6.11 illustrates a failure mode specific to early-training SAC checkpoints on Corridor scenarios. The best-validation checkpoint for FGS_p on cologne3 occurs at episode 5, where mean delay is anomalously low but mean queue length reaches 46.1 veh./lane — physically incompatible with genuine low delay in any steady state. The mechanism is *reward specification gaming* [29]: SUMO defines a vehicle’s waiting time as accumulated time stopped; the counter pauses the instant the vehicle starts moving, regardless of whether it clears the stop line. Rapid phase cycling releases one lane briefly per phase switch, collecting a waiting-time-reduction spike, while vehicles in all other lanes continue to queue. SAC’s entropy bonus $\alpha \mathcal{H}(\pi)$ explicitly rewards distributing probability across phases, giving phase cycling a double incentive: both the traffic reward and the entropy bonus increase simultaneously. PPO avoids this pathway because it has no explicit entropy term in the reward and discards stale transitions after each episode, directly observing the resulting queue inflation in the next update. All checkpoints with physically inconsistent queue–delay combinations are marked ‡ in the tables and excluded from the Bold convention. Replacing the differential waiting-time reward with a pressure or queue-clearing signal [37] is a direct recommendation for future work.

Factored critic and SAC scalability. The FGS final design replaces the centralized critic with a factored neighbourhood critic (Section 4.6.1) whose input dimension is $O(d_c + A_{\max}) = 204$ at ingolstadt21 — a $15\times$ reduction from the centralized critic (Appendix A.9). This enables both FGSv2-SAC and FGSv3-SAC to complete training at $N = 21$. The factored critic achieves effective neighbourhood information pooling by operating on GATv2-enriched local embeddings rather than raw joint observations, so the critic input remains tractable at any N while still incorporating multi-hop

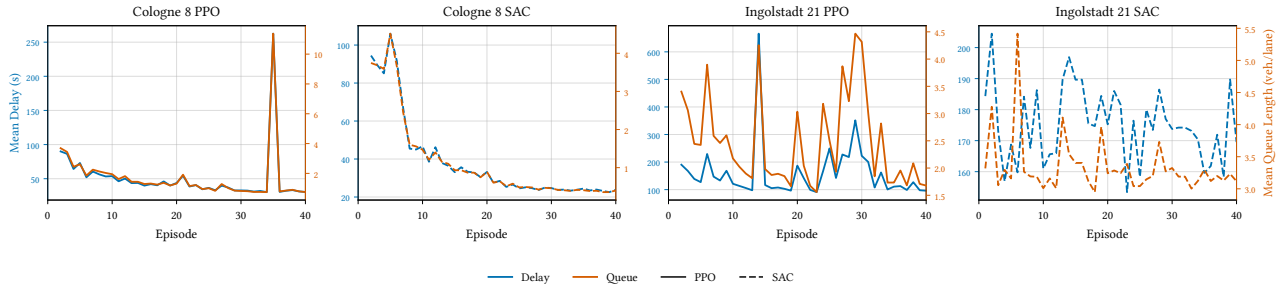


Figure 6.11: Reward specification gaming in early-training SAC/PPO checkpoints (cologne8 and ingolstadt21, first 40 episodes). Dual y-axis: mean delay (left, s) and mean queue length (right, veh./lane). SAC checkpoints with anomalously low delay coincide with very high queue depth, confirming phase-cycling reward gaming. PPO does not exhibit this pattern because its on-policy update directly penalises the resulting queue growth.

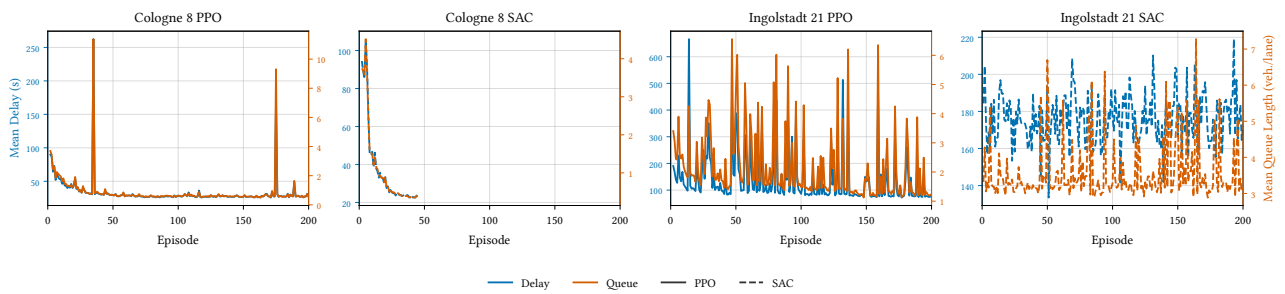


Figure 6.12: Full 200-episode SAC/PPO delay–queue trajectory on cologne8 and ingolstadt21. After the early-training gaming phase, PPO stabilises with consistent low queue depth; SAC continues to exhibit episodic gaming spikes throughout training on ingolstadt21 where the entropy bonus consistently reinforces phase-cycling exploration. The ‡-marked early checkpoints correspond to the anomalous low-delay/high-queue region in Figure 6.11.

demand signals via the communication layer.

6.5 Training Convergence

Figure 6.13 shows training and validation mean delay curves for the two Regional-tier scenarios across FGS variants and selected baselines. Figure 6.14 shows Corridor-tier convergence. Full training curves across all six German RESCO scenarios for all four evaluation metrics are provided in Appendix B.1.

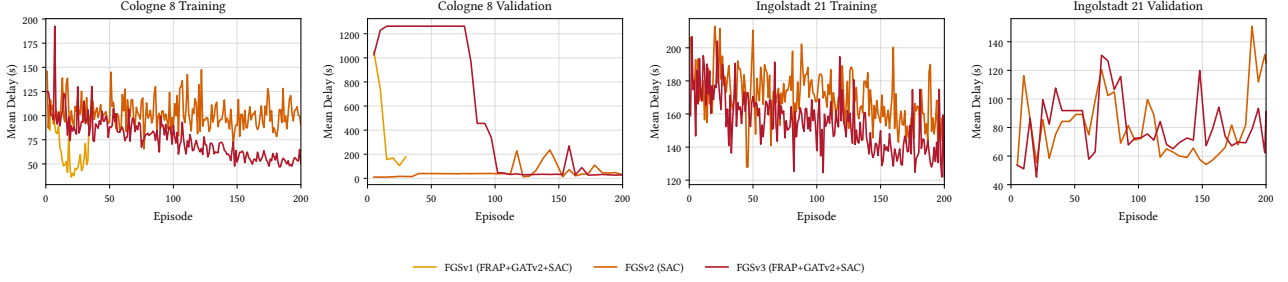


Figure 6.13: Mean delay training (left column) and validation (right column) curves for Cologne Regional ($N = 8$, top row) and Ingolstadt Regional ($N = 21$, bottom row) across FGS generations and baselines. FGSv3-PPO achieves stable convergence on both scenarios. First-generation SAC on ingolstadt21 terminates at 21 episodes (short trace). FGSv2-SAC and FGSv3-SAC complete training but exhibit high-variance trajectories on ingolstadt21 without clear convergence; their early-training best checkpoints are marked ‡.

Cologne Regional ($N = 8$). FGS_s(MLP+GATv2+SAC) achieves stable convergence on cologne8, stabilising near 22–24 s mean delay between episodes 22 and 35. FGSv3-PPO is the most stable run on this scenario, reaching its best checkpoint (21.6 s delay) at episode 39 with minimal variance. The relatively small differences between FGS variants on cologne8 (22.2–24.9 s mean delay) reflect the moderate topology: at $N = 8$ all learning methods converge to competitive policies within 200 episodes, and differences are driven by algorithm-level sample efficiency rather than coordination architecture.

Ingolstadt Regional ($N = 21$). FGSv3-PPO is the only FGS variant with stable, convergent behaviour on ingolstadt21: the validation curve stabilises in the 68–85 s mean delay range from episode 7, with the best checkpoint at 68.4 s (episode 90). FGS_p(MLP+GATv2+PPO) completes all 200 episodes but shows a large train-to-validation gap (training stable at ≈ 78 s; best-validation 234.9 s), suggesting overfitting to the training demand distribution. FGSv2-SAC and FGSv3-SAC exhibit high-variance trajectories; their best-checkpoint delay–queue values are physically inconsistent (gaming artifacts, marked ‡) and are excluded from converged-result comparisons.

Corridor tier. Figure 6.14 shows that at Corridor scale all learning methods converge to similar mean delay ranges within 100 episodes, with PPO achieving the lowest final delay on both scenarios. There are no persistent multi-episode spikes, recovery from transient demand surges is fast, and variance between methods is smaller than at Regional scale. This is consistent with the scale-dependency argument: at $N \leq 7$, local observations are sufficient for independent policies to converge effectively within the available training budget.

6.6 Design Evolution

The FGS pipeline was developed through three successive generations, each motivated by a specific module-interface failure exposed during evaluation of the preceding design. This iterative diagnosis-and-repair cycle is itself a methodological contribution: the three-module pipeline structure provides enough isolation between components to identify, localise, and correct individual failure modes without redesigning the whole framework. Table 6.3 summarises the trajectory; Figure 6.15 shows the performance improvement on Ingolstadt Regional.

Gen-1 exposed two failures: (1) FRAP’s scalar output discards per-phase competition information before graph aggregation; (2) the centralized critic’s $O(Nd)$ input prevents SAC from completing training at $N = 21$. FGSv2 fixed failure (1)

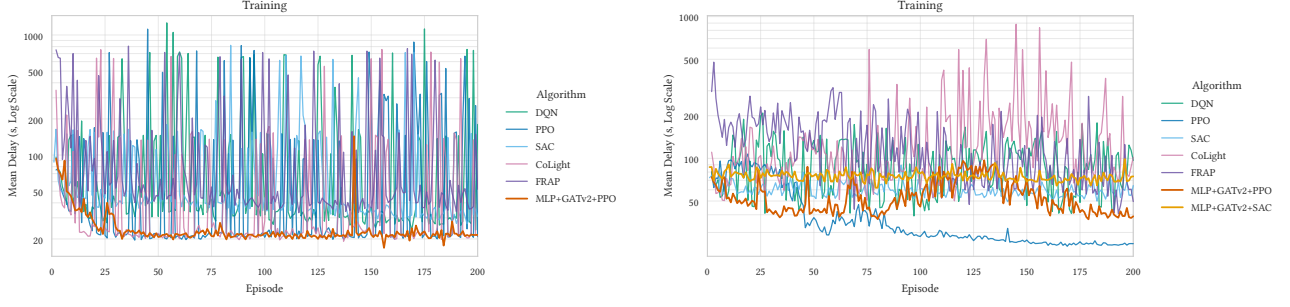


Figure 6.14: Training mean delay curves for Corridor-tier scenarios: cologne3 ($N = 3$, left) and ingolstadt7 ($N = 7$, right). All learning methods converge to similar delay ranges within 100 episodes, consistent with local observations being sufficient at Corridor scale. The $\ddagger$ early-training transient for FGS_p on cologne3 (episode ≤ 5) is annotated. FGS_s results on ingolstadt7 exhibited a physically inconsistent checkpoint (17.1 s delay, $\ddagger$) and are excluded from the Corridor comparison; their training trace is not shown.

Table 6.3: FGS generation trajectory: module configuration and the module-interface failure each generation addresses. Each generation modifies only the components implicated by the preceding evaluation.

Gen.	Module I (Encoder)	Module II (Communication)	Module III (RL)
Gen-1	MLP or FRAP (scalar)	GATv2 on $\mathbf{h}_i$	SAC/PPO (centralized critic)
FGSv2	FRAP action tokens	GATv2 on $\mathbf{s}_i$ (competition)	SAC (centralized critic)
FGSv3	FRAP action tokens	GATv2 on demand $\mathbf{d}_i$	SAC/PPO (factored critic)
<i>Interface failure addressed:</i>			
Gen-1→FGSv2	Scalar projection discards per-phase information	—	—
FGSv2→FGSv3	—	Competition signal encodes ranking, not demand	$O(Nd)$ centralized critic fails at $N = 21$

by tapping FRAP before scalar projection, preserving per-phase action tokens $\mathbf{z}_{i,a}$ through the graph aggregation step. Evaluation of FGSv2 then revealed a third failure: the GATv2 message was a pooled competition score encoding phase rankings rather than demand state — a suboptimal signal when agents need to anticipate upstream pressure. The FGS final design fixed both remaining failures simultaneously: the Module-II signal was replaced with demand embeddings $\mathbf{d}_i$, and the centralized critic was replaced with a factored neighbourhood critic independent of N . Because these two changes were introduced together, their individual contributions to the observed 71% mean delay reduction (Gen-1 FGS_p: 234.9 s; FGSv3-PPO: 68.4 s on ingolstadt21) cannot be independently decomposed from the current data; a future partial ablation isolating each change is a natural follow-on experiment. Full algorithmic details for each generation are provided in Appendix A.14.

Table 6.4 reports the full performance of all FGS generations on Regional-tier scenarios.

6.7 Answering the Research Questions

SQ1 (Baseline comparison): The FGS pipeline is competitive with the strongest baselines at Regional scale and superior to graph-based CoLight across all tiers. All comparisons below are from single-seed runs; differences within 3–4 s should be interpreted as directional trends rather than statistically significant rankings. FGSv3-PPO achieves 68.4 s mean delay on ingolstadt21, outperforming DQN (−13%, 78.7 s) and CoLight (−68%, 216.8 s), while remaining competitive with the standalone PPO checkpoint (67.2 s). At Corridor scale, FGS_p outperforms DQN and CoLight on ingolstadt7 but trails both independent PPO (26.1 s vs. 24.1 s) and the independent SAC baseline (26.1 s vs. 24.0 s); on cologne3 the independent SAC baseline achieves the best converged result (18.9 s) ahead of PPO (20.3 s). At Single-intersection scale, FGS provides no consistent improvement. The competitive envelope grows with network scale, as shown in Figure 6.2.

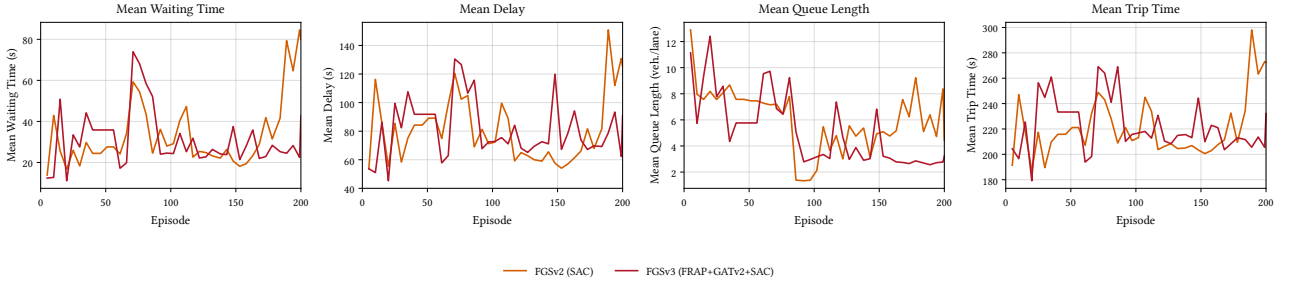


Figure 6.15: Validation performance of FGS generations on Ingolstadt Regional ($N = 21$) across four metrics. Gen-1 FGS_p converges to 234.9 s mean delay; FGSv2-SAC and FGSv3-SAC show early-training transients (marked ‡, excluded from comparisons); FGSv3-PPO is the only generation with stable monotone convergence, reaching 68.4 s mean delay at episode 90.

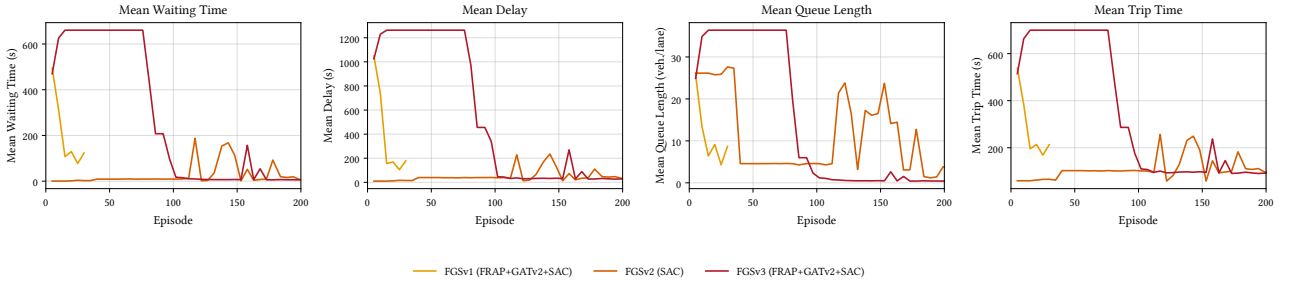


Figure 6.16: Validation performance of FGS generations on Cologne Regional ($N = 8$) across four metrics (200 episodes). FGSv3-PPO achieves the best mean delay (21.6 s) with stable convergence; FGSv3-SAC converges but with higher variance; earlier generations are competitive on this moderate-scale scenario, reflecting that at $N = 8$ the coordination benefit is smaller and all variants can converge within the training budget.

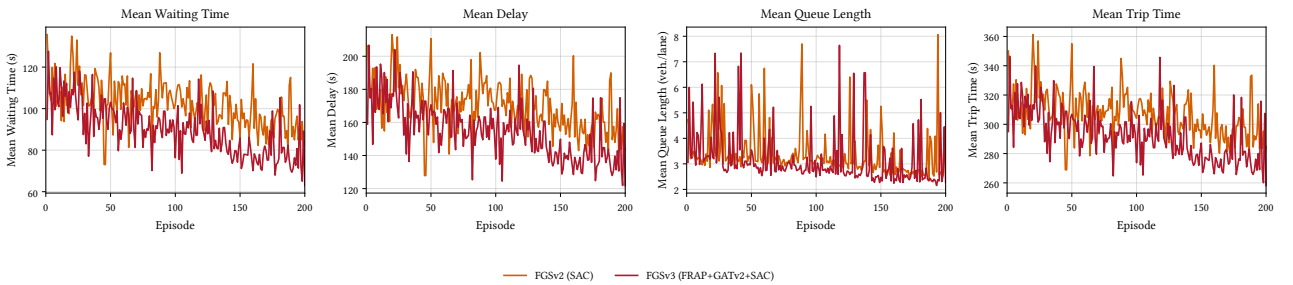


Figure 6.17: Training performance of FGS generations on Ingolstadt Regional ($N = 21$) across four metrics (200 episodes). The training curves for FGSv3-PPO are notably more stable than both SAC variants, which exhibit high-variance trajectories and fail to converge within 200 episodes on training data. The divergence between training and validation performance (compare Figure 6.15) is particularly pronounced for Gen-1 FGS_p, which achieves lower training delay than FGSv3-PPO but substantially worse validation delay.

Table 6.4: FGS architecture progression on Regional-tier scenarios (best-validation checkpoint; single seed). Mean delay (s) and mean waiting time (s), ↓ lower is better. **Bold**: best mean delay among FGS variants per scenario. † partial run; ‡ physically inconsistent checkpoint. SAC checkpoints marked ‡ exhibit delay–queue inversion indicative of reward specification gaming; they are excluded from the **Bold** convention.

Scenario	Metric	FGS _s	FGS _p	FGSv2-SAC	FGSv3-PPO	FGSv3-SAC
Cologne Reg. ($N = 8$)	Delay	22.2	24.0	10.1 [†]	21.6	26.5
	Wait	4.9	6.2	0.4 [‡]	4.8	6.2
Ingolstadt Reg. ($N = 21$)	Delay	39.1 [†]	234.9	53.7 [‡]	68.4	45.3 [‡]
	Wait	9.4 [‡]	88.6	13.9 [‡]	22.9	11.1 [‡]

† Partial run (fewer than 200 ep.).

‡ Physically inconsistent checkpoint (gaming artifact); not a valid performance comparison — see Section 6.8.

SQ2 (Design-space analysis): All comparisons are from single-seed runs (seed 0); directional conclusions are supported by the ablation magnitude. Encoder choice is the dominant local determinant: FRAP-based encoders produce mean delay 11% to $2.6\times$ higher than MLP encoders, with the penalty scaling with network size due to structural mismatch with German intersection phase heterogeneity. Graph-attention type governs policy generalization at Regional scale: GATv2 reduces mean delay by $2.1\times$ compared with GAT on ingolstadt21 (234.9 s vs. 492.0 s), with negligible difference on cologne8. RL algorithm choice is scenario-dependent: SAC is more sample-efficient at $N \leq 8$ — corroborated by the independent SAC baseline’s competitive Corridor performance (cologne3: 18.9 s, ingolstadt7: 24.0 s) — but requires a factored critic architecture to complete training at $N = 21$, and its entropy bonus creates reward-gaming instability on the differential waiting-time objective at larger scale. Note that the FGSv2-to-FGSv3 transition introduced two simultaneous changes (demand-aware communication signal and factored critic); their individual contributions cannot be independently decomposed from these data (see Section 4.6.2). Full component-level analysis in Section 6.4.

SQ3 (Scale and topology dependency): The graph-communication benefit is negligible at $N = 1$ (structural no-op), emerging at Corridor scale ($N = 7$: FGS beats DQN and CoLight but trails standalone PPO), and strongest at Regional scale ($N = 21$: FGSv3-PPO strongly outperforms CoLight and is near the standalone PPO checkpoint). GATv2 becomes critical at $N = 21$: substituting GAT roughly doubles mean delay (492.0 s vs. 234.9 s). Encoder mismatch amplifies with scale. The scale threshold for consistent graph-communication advantage is approximately $N = 8$ –21 for the German RESCO topologies evaluated, grounded in the spatio-temporal dependency argument of Section 6.2 (see Figure 6.2).

Main Research Question: The FGS pipeline is competitive and scalable at Regional scale. FGSv3-PPO is near the best standalone independent baseline on the largest evaluated scenario (ingolstadt21) and outperforms the graph-based CoLight baseline. Scalability is achieved through the three-generation diagnosis-and-repair trajectory, which identifies and corrects three distinct module-interface failures. The primary design determinants, in order of impact, are: (1) encoder structural compatibility with the target topology; (2) attention mechanism expressiveness at scale; and (3) critic architecture scalability for the RL algorithm.

6.8 Limitations

Single-seed results. All performance figures correspond to single training runs (seed 0). The RESCO benchmark protocol targets five independent seeds with result averaging and standard-deviation reporting; resource constraints precluded this within the available training budget. Performance figures are point estimates without variance bounds and should be interpreted as indicative rather than statistically conclusive. Multi-seed completion over seeds $\{0, 1, 2, 3, 4\}$ with bootstrap confidence intervals constitutes the highest-priority follow-on validation.

Training instability of SAC variants on ingolstadt21. FGSv2-SAC and FGSv3-SAC complete training at $N = 21$ with the factored critic but exhibit high-variance trajectories without clear convergence within 100 episodes. Their best-checkpoint values (FGSv3-SAC: 45.3 s delay at episode 20; FGSv2-SAC: 53.7 s at episode 5) exhibit physically inconsistent delay–queue ratios (high queue depth relative to delay) consistent with reward specification gaming rather than genuine control quality. This behaviour is consistent with two compounding structural factors: the *deadly triad* of off-policy experience replay, function approximation, and bootstrapped targets identified as a source of instability in deep RL [14]; and MARL non-stationarity, where stored transitions become progressively off-distribution as all 21 agents co-evolve [15]. Stable, converged SAC performance on ingolstadt21 would likely require 300–500 training episodes with

entropy-temperature tuning calibrated to the 21-agent joint action space.

Reward specification gaming. Early-training SAC checkpoints on Corridor scenarios exhibit physically inconsistent queue–delay combinations (Figure 6.11 and Section 6.4.3). The root cause is the differential waiting-time reward: a proxy signal reducible by rapid phase cycling without clearing queues [29]. All affected checkpoints are marked ‡ and excluded from comparisons. As noted above, replacing this reward with a pressure or queue-clearing signal [37] addresses the root cause and is a direct recommendation for future work.

Train-to-validation demand shift. FGS_p shows a substantially larger train-to-validation gap on ingolstadt21 than DQN (training delay ≈ 78 s for both; best-validation 234.9 s for FGS vs. 78.7 s for DQN), suggesting that the attention-conditioned representation is more sensitive to demand distribution shift between training and validation seeds. FGSv3-PPO reduces this gap (stable validation from episode 7). Domain randomisation during training and multi-seed averaging are the natural remedies and constitute priority directions for future work alongside multi-seed statistical validation.

Chapter 7

Conclusion and Future Work

7.1 Summary

This thesis set out to systematically characterize the three-module design space of MARL-TSC architectures — encoder, graph communication, and RL algorithm — on real-world German road networks, and to derive empirically grounded conclusions about which component choices matter and under what conditions. The investigation was framed not as a claim that any single configuration solves traffic optimization, but as a principled ablation study of how these components interact across a standardized set of scenarios spanning three scale tiers.

Urban traffic signal control presents a multi-agent decision problem with partial observability, heterogeneous intersection topologies, and demand conditions that vary continuously over a simulation episode. Classical methods such as Webster’s fixed-time plans and max-pressure control [31] offer tractable, interpretable solutions but cannot adapt dynamically to non-stationary demand. Deep RL approaches such as IntelliLight [35] and PressLight [37] demonstrated that learned policies can match or exceed adaptive controllers in simulation, but their evaluation was largely confined to synthetic Chinese grid networks. Graph-based MARL methods, most notably CoLight [36], extended RL-based TSC to network-level coordination by treating intersections as nodes in a communication graph. However, none of the published graph-aware MARL methods evaluated on German road networks, none combined a phase-competition-aware local encoder with graph attention, and none used discrete-action SAC as the RL optimiser.

The FGS (Framework for Graph-aware Signal control) proposed in this thesis addresses all three gaps through a modular three-module pipeline and a three-generation implementation trajectory. The Dec-POMDP problem formulation (Chapter 4) defines each signalized intersection as an agent whose observation is a graph-structured tensor containing local phase state, per-lane density and queue features, and a topological neighborhood constructed via Dijkstra super-edges from the SUMO network file. Actions are discrete phase selections; the reward is the differential waiting time, which provides a dense training signal without requiring domain-specific reward engineering. Three successive FGS generations are developed through an ablation-driven diagnosis-and-repair cycle. The first generation establishes the baseline pipeline, supporting four variants (Table 4.3) across two encoder choices (MLP, FRAP) and two communication modes (none, GATv2), and exposes two interface failures. The second generation (FGSv2) fixes the encoder–communication boundary by preserving per-phase action tokens through the graph aggregation step. The final FGS design fixes the communication signal (switching from competition-derived to demand-aware embeddings) and resolves the critic scalability barrier by replacing the centralized critic with a factored neighborhood critic whose input dimension is $O(d_c)$ independent of N , enabling SAC training to complete at $N = 21$.

The experimental evaluation (Chapters 5 and 6) assessed FGS, FGSv2, and FGSv3 against seven baselines — fixed-time control, max-pressure, IDQN, IPPO, independent SAC, FRAP, and CoLight — across the six German RESCO scenarios derived from TapasCologne [10] and InTAS [20], organized into the three scale tiers introduced in Section 1.5.1.

The experimental findings allow the four hypotheses stated in Chapter 1 to be revisited in light of evidence. H1 and H4 inform SQ3 (scale dependency); H2 and H3 inform SQ2 (design-space analysis); together they underpin the answer to the MRQ.

H1 — that graph-aware communication improves coordination compared with independent agents — receives *support for comparisons against graph-based baselines* but remains inconclusive against truly independent methods within single-seed evaluation. FGS(MLP+GATv2) variants consistently outperform graph-based CoLight at Regional and Corridor scale, but standalone PPO and DQN remain competitive or better on mean delay. At Corridor scale (ingolstadt7, $N = 7$), FGS_p(MLP+GATv2+PPO) achieves 26.1 s mean delay versus independent PPO’s 24.1 s, while strongly outperforming

CoLight (47.0 s). At Regional scale, FGSv3-PPO achieves 68.4 s, competitive with standalone PPO (67.2 s) and far ahead of CoLight (216.8 s). At Single-intersection ($N = 1$), the graph module has no neighbors to aggregate and the benefit is zero by construction.

H2 — that entropy-regularized SAC yields more stable training than DQN — is confirmed in selected scenarios, particularly those with larger intersection counts where the policy landscape is more complex. However, in small single-intersection scenarios (cologne1, ingolstadt1), DQN’s simpler update rule converges with comparable or faster stability, consistent with the expectation that entropy bonuses are most valuable where policy diversity matters.

H3 — that FRAP-based encoders do not uniformly outperform MLP encoders — is confirmed across all evaluated scenarios. The structural explanation is provided in Section 6.4.1: FRAP’s phase-competition priors align with regular, four-directional intersection layouts but produce misaligned features on German RESCO networks with heterogeneous phase counts. This finding does not diminish FRAP’s theoretical motivation; it highlights that the benefit of structured local encoding depends on the match between the encoder’s priors and the target network topology.

H4 — that Regional-tier networks reveal stronger differences between independent and graph-aware methods than Single-intersection or Corridor scenarios — is confirmed in the direction expected: the performance gap between FGS(MLP) and FGS(MLP+GATv2) grows with the tier scale. Single-intersection scenarios (cologne1, ingolstadt1) show negligible graph-communication benefit by definition; Regional-tier scenarios (cologne8, ingolstadt21) show the clearest advantage for GATv2 variants, where upstream spillback propagation provides a meaningful signal for neighborhood aggregation.

Taken together, the results demonstrate that the FGS pipeline provides a productive framework for characterizing MARL-TSC design on German road networks. The three-generation trajectory — driven by empirical findings rather than a priori design — confirms that encoder choice, graph-attention expressiveness, and critic architecture each govern a distinct failure mode, and that addressing all three is necessary for competitive performance at Regional scale. FGSv3-PPO achieves 68.4 s mean delay on ingolstadt21, competitive with standalone PPO (67.2 s) and outperforming CoLight (216.8 s, -68%) and DQN (78.7 s, -13%). On ingolstadt7 ($N = 7$), FGS_p(MLP+GATv2+PPO) achieves 26.1 s mean delay, outperforming DQN (34.4 s, -24%) and CoLight (47.0 s, -44%) but trailing the standalone PPO checkpoint (24.1 s). The ablation establishes concrete design guidelines: MLP encoders outperform phase-competition encoders on German intersection heterogeneity; GATv2 provides a 52% mean delay reduction over GAT at $N = 21$; and SAC requires a factored critic to complete training at Regional scale.

7.2 Contributions

This thesis makes four contributions at different levels of abstraction.

7.2.1 Framework Contribution

A modular, reproducible experimental stack for graph-aware MARL traffic signal control was designed and implemented. The stack integrates SUMO [21] via the TraCI interface, the SUMO-RL gymnasium wrapper [1], RLlib for distributed multi-agent training, PyTorch Geometric for graph neural network operations, Hydra for structured configuration management, and Weights & Biases [4] for experiment tracking. Graph-structured observations are constructed automatically from the SUMO network file, and the training pipeline supports arbitrary agent count, flexible phase structures, and configurable encoder and communication modules. The implementation is publicly available¹ and extends the open-source SUMO-RL library with graph observation wrappers, custom RLlib policy modules, and reproducible experiment scripts.

7.2.2 Algorithmic Contribution

Three successive FGS generations were designed and implemented (Chapter 4), each addressing a specific module-interface failure identified in evaluation of the preceding generation. The first generation establishes the baseline, exposing that FRAP’s scalar projection discards per-phase information before graph aggregation and that the centralized critic’s $O(Nd)$ input prevents SAC from completing training at $N = 21$. FGSv2 fixes the Module-I→Module-II boundary by tapping FRAP before scalar projection, preserving per-phase action tokens for the graph layer; this exposes that the communication signal carries competition-derived rankings rather than demand state. The final FGS design fixes

¹<https://github.com/fuisl/sumo-rl>

both remaining failures: the Module-II signal is replaced with demand-aware embeddings, and the Module-III centralized critic is replaced with a factored neighborhood critic ($O(d_c + A_{\max})$ independent of N), enabling both SAC and PPO to complete training at $N = 21$. Full architectural details are given in Section 4.6.1; earlier generation algorithms are in Appendix A.14.

7.2.3 Empirical Contribution

A comprehensive evaluation comparing FGS variants against seven baseline methods — fixed-time control, max-pressure, IDQN, IPPO, independent SAC, FRAP, and CoLight — across all six German RESCO scenarios was conducted. The evaluation protocol targets the RESCO benchmark standard of 200 training episodes, evaluation every five episodes, and performance reported on mean travel time, mean delay, mean waiting time, and throughput. Due to resource constraints, most configurations correspond to single-seed runs (seed 0); multi-seed averaging over five seeds with standard-deviation reporting remains as a priority for follow-on validation (Section 6.8).

7.2.4 Analytical Contribution

A component-level ablation study disentangles the contribution of each architectural element: encoder type (MLP vs. FRAP), communication (with vs. without GATv2), and RL algorithm (SAC vs. DQN vs. PPO). This analysis quantifies the conditions under which phase-aware encoding helps or hurts, and the network scales at which graph communication produces the most consistent benefit. The finding that FRAP’s inductive bias does not consistently transfer to German intersection topologies is itself an empirical contribution to the broader MARL-TSC literature, where FRAP has been widely adopted without topology-specific validation.

7.3 Future Research Directions

The work reported in this thesis opens several directions for further investigation.

Continuous green-time optimization. The FGS framework selects discrete phases; phase duration is fixed by the minimum green time constraint. Extending the action space to include duration assignment — either as a continuous output or as a discrete time-slot selection — would allow the controller to optimize both phase sequence and green-time allocation jointly. Parameterized action spaces or hybrid discrete-continuous actors are natural candidates for this extension.

Fairness-constrained reward design. The differential waiting time reward aggregates delay across all lanes without distinguishing between high-demand and low-demand directions. Future work could incorporate explicit fairness constraints — for example, penalizing extreme queue disparities across movements within an intersection or across intersections in the network — to produce more equitable service distributions without sacrificing overall throughput.

Transfer learning and domain adaptation. The FGS models trained on TapasCologne and InTAS demand profiles were evaluated on the same demand distribution used during training. An important open question is whether policies generalize across German cities, to demand patterns not represented in the training data, or across different countries’ intersection geometries. Meta-learning or domain-randomization approaches could be used to train controllers robust to demand distribution shift.

Real-world deployment. All experiments in this thesis were conducted in SUMO simulation. Deployment to real intersection controllers introduces additional challenges: sensor noise from inductive loop detectors, communication latency between the central coordination layer and field controllers, legal constraints on minimum phase durations, and the absence of a differentiable environment for online policy gradient updates. Sim-to-real transfer methods and hardware-in-the-loop evaluation are natural next steps.

Scalability beyond the Regional tier. The largest evaluated scenario (ingolstadt21, Regional tier, $N = 21$) contains twenty-one intersections. The FGSv3 factored neighborhood critic reduces critic input dimensionality from $O(Nd_{h'})$ to $O(d_c)$, enabling SAC training to complete at $N = 21$ where FGSv2 failed. However, the GATv2 message-passing layer

itself has $O(N)$ memory in the number of nodes, and the graph construction from the SUMO network file currently targets small-to-moderate scenarios. Extending FGS beyond the Regional tier to hundreds of intersections will likely require hierarchical MARL architectures — in which local clusters of intersections are managed by sub-controllers that themselves communicate through a higher-level graph — or sparse graph attention variants that limit each agent’s receptive field to a fixed-size neighborhood regardless of network size.

Dynamic graph topology. The graph structure used in FGS is constructed offline from the SUMO network file and remains fixed throughout the episode. Real traffic networks experience topology changes due to incidents, road closures, and temporary signal failures. A dynamic graph construction mechanism that updates edge weights and neighborhood sets in response to real-time travel-time estimates would make the framework more robust to network perturbations.

Graph attention interpretability. The GATv2 attention layer assigns learned weights to each neighbor, potentially encoding which upstream or downstream intersections are most relevant under current traffic conditions. Visualizing attention weights over time and correlating them with queue lengths and phase states could provide insight into the implicit coordination strategies learned by the controller, and connect the quantitative performance results to a mechanistic understanding of how graph communication supports multi-agent signal coordination across different network scales.

Pedestrian and multi-modal integration. The current formulation addresses vehicle phase selection only. Extending the observation space and reward function to account for pedestrian crossing demands, cycle lanes, and public transport priority would substantially increase the realism and practical applicability of the framework in European urban environments, where multi-modal integration is a primary concern of traffic management authorities.

Appendix A

Mathematical Objectives, Gradients, and Stability Analysis

This appendix collects the mathematical derivations that underpin the FGS/FGSv2 framework: objective functions, loss gradients, masked action distributions, architectural equivariance properties, and the theoretical basis for convergence claims. Proofs are stated for the finite tabular/exact-policy setting; Section A.11.3 states explicitly which guarantees do not extend to the implemented deep MARL system.

A.1 Notation and CTDE Objective

Throughout this appendix: N is the number of agents; $A_i = |\mathcal{A}_i|$ the valid phase count for agent i ; $A_{\max} = \max_i A_i$ the padded action-space width; o_i^t the local observation of agent i at step t ; $\mathbf{o}_t = (o_1^t, \dots, o_N^t)$ the joint observation; $\mathbf{a}_t = (a_1^t, \dots, a_N^t)$ the joint action.

The cooperative multi-agent objective averages individual rewards:

$$J(\phi) = \mathbb{E}_{\pi_\phi} \left[\sum_{t=0}^T \gamma^t \frac{1}{N} \sum_{i=1}^N r_i^t \right]. \quad (\text{A.1})$$

Under maximum-entropy augmentation this becomes

$$J_{\text{MaxEnt}}(\phi) = \mathbb{E}_{\pi_\phi} \left[\sum_{t=0}^T \gamma^t \frac{1}{N} \sum_{i=1}^N \left(r_i^t + \alpha \mathcal{H}(\pi_\phi(\cdot \mid o_i^t)) \right) \right]. \quad (\text{A.2})$$

Agents optimize per-agent SAC objectives with local rewards r_i^t ; evaluation follows the RESCO network-level average protocol. The CTDE factorization is

$$\pi_\phi(\mathbf{a} \mid \mathbf{o}) = \prod_{i=1}^N \pi_\phi(a_i \mid o_i, \mathcal{G}), \quad (\text{A.3})$$

while at execution time each agent i acts on its local embedding $\mathbf{h}_i'$ alone.

A.2 Differential Waiting-Time Reward: Telescoping Property

The differential waiting-time reward is $r_t = W_{t-1} - W_t$ (Equation 4.12). The following result shows that maximizing the discounted sum of differential rewards is equivalent to minimizing a discounted sum of cumulative waiting times.

Proposition A.1 (Reward telescoping). *For a finite horizon T and discount factor $\gamma \in (0, 1)$,*

$$\sum_{t=1}^T \gamma^{t-1} (W_{t-1} - W_t) = W_0 - (1 - \gamma) \sum_{t=1}^{T-1} \gamma^{t-1} W_t - \gamma^{T-1} W_T. \quad (\text{A.4})$$

Proof. Separate the sum: $S = \sum_{t=1}^T \gamma^{t-1} W_{t-1} - \sum_{t=1}^T \gamma^{t-1} W_t$. Re-index the first sum ($s \leftarrow t - 1$, range 0 to $T - 1$):

$$\begin{aligned}
 S &= \sum_{t=0}^{T-1} \gamma^t W_t - \sum_{t=1}^T \gamma^{t-1} W_t \\
 &= W_0 + \sum_{t=1}^{T-1} \gamma^t W_t - \frac{1}{\gamma} \sum_{t=1}^{T-1} \gamma^t W_t - \gamma^{T-1} W_T \\
 &= W_0 + \left(1 - \frac{1}{\gamma}\right) \sum_{t=1}^{T-1} \gamma^t W_t - \gamma^{T-1} W_T \\
 &= W_0 - (1 - \gamma) \sum_{t=1}^{T-1} \gamma^{t-1} W_t - \gamma^{T-1} W_T.
 \end{aligned}$$

□

Interpretation. Maximizing S over trajectories is equivalent (up to the constant W_0) to minimizing $(1 - \gamma) \sum_{t=1}^{T-1} \gamma^{t-1} W_t + \gamma^{T-1} W_T$: a discounted sum of intermediate waiting times plus the terminal waiting time. When $\gamma < 1$ both terms contribute; as $\gamma \rightarrow 1$ the sum approaches $W_0 - W_T$, so the agent would primarily optimize terminal rather than cumulative waiting. Using $\gamma = 0.99$ in this work keeps the intermediate terms substantial.

A.3 Masked Discrete Action Distribution

Intersections in the German RESCO scenarios have heterogeneous phase counts; invalid phases must receive zero probability. Define the valid phase set $\mathcal{A}_i(o_i) = \{a : m_{i,a} = 1\}$ and the masked policy

$$\pi_\phi(a \mid o_i) = \frac{m_{i,a} \exp z_{i,a}}{\sum_{b=1}^{A_{\max}} m_{i,b} \exp z_{i,b}}. \quad (\text{A.5})$$

Proposition A.2 (Zero probability and zero gradient on masked actions). *If $m_{i,a} = 0$ then $\pi_\phi(a \mid o_i) = 0$ and $\partial \pi_\phi(a \mid o_i) / \partial z_{i,b} = 0$ for all b .*

Proof. When $m_{i,a} = 0$ the numerator of (A.5) is zero regardless of $z_{i,a}$, so $\pi_\phi(a \mid o_i) = 0$. Since this holds for all values of all logits, every partial derivative is also zero. □

Implementation. In practice invalid logits are set to -10^9 before the standard softmax rather than using (A.5) directly; the two formulations are numerically equivalent when the finite approximation dominates all valid logits.

A.4 Discrete SAC: Losses and Gradients

A.4.1 Critic Loss and Gradient

Given a replay-buffer sample (s, a, r, s', d) , the soft TD target (with stop-gradient) is

$$y = r + \gamma(1 - d) \sum_{a'} \pi_\phi(a' \mid s') \left[\min_{j=1,2} Q_{\theta_j}(s', a') - \alpha \log \pi_\phi(a' \mid s') \right]. \quad (\text{A.6})$$

For twin critic j :

$$\mathcal{L}_Q(\theta_j) = \mathbb{E}_{\mathcal{D}} \left[\frac{1}{2} (Q_{\theta_j}(s, a) - y)^2 \right], \quad \nabla_{\theta_j} \mathcal{L}_Q = \mathbb{E}_{\mathcal{D}} \left[(Q_{\theta_j}(s, a) - y) \nabla_{\theta_j} Q_{\theta_j}(s, a) \right]. \quad (\text{A.7})$$

With Prioritized Experience Replay the loss is reweighted by importance-sampling corrections. For TD error $\delta_k = Q_{\theta_j}(s_k, a_k) - y_k$, define

$$p_k = |\delta_k| + \varepsilon, \quad P(k) = \frac{p_k^\beta}{\sum_j p_j^\beta}, \quad w_k = \left(\frac{1}{|\mathcal{D}| P(k)} \right)^\rho, \quad \tilde{w}_k = \frac{w_k}{\max_j w_j}, \quad (\text{A.8})$$

giving PER critic loss $\mathcal{L}_Q(\theta_j) = \mathbb{E}_{k \sim P}[\tilde{w}_k \frac{1}{2} \delta_k^2]$.

A.4.2 Actor Loss and Softmax-Logit Gradient

The discrete actor loss is (reproduced from Equation 4.20):

$$\mathcal{L}_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[\sum_{a \in \mathcal{A}(s)} \pi_\phi(a | s) \left(\alpha \log \pi_\phi(a | s) - \min_k Q_{\theta_k}(s, a) \right) \right]. \quad (\text{A.9})$$

Define the soft state value $V_\theta^\pi(s) = \sum_a \pi_a [Q_\theta(s, a) - \alpha \log \pi_a]$.

Proposition A.3 (Logit gradient of the actor loss). *Let $\pi_a = \text{softmax}(z)_a$. Then*

$$\frac{\partial \mathcal{L}_\pi}{\partial z_b} = \pi_b \left[\alpha \log \pi_b - Q_\theta(s, b) + V_\theta^\pi(s) \right]. \quad (\text{A.10})$$

Proof. Using the softmax Jacobian $\partial \pi_a / \partial z_b = \pi_a (\delta_{ab} - \pi_b)$ and applying the product rule:

$$\begin{aligned} \frac{\partial \mathcal{L}_\pi}{\partial z_b} &= \sum_a \frac{\partial \pi_a}{\partial z_b} (\alpha \log \pi_a - Q_\theta(s, a) + \alpha) \\ &= \sum_a \pi_a (\delta_{ab} - \pi_b) (\alpha \log \pi_a - Q_\theta(s, a) + \alpha) \\ &= \pi_b (\alpha \log \pi_b - Q_\theta(s, b) + \alpha) - \pi_b \sum_a \pi_a (\alpha \log \pi_a - Q_\theta(s, a) + \alpha). \end{aligned}$$

The inner sum equals $-V_\theta^\pi(s) + \alpha$. Substituting and cancelling α yields (A.10). $\square$

Interpretation. Actions with $Q_\theta(s, b) > V_\theta^\pi(s)$ receive a negative gradient component and therefore increased probability; the baseline $V_\theta^\pi(s)$ centers each update; the $\alpha \log \pi_b$ term moderates excessive concentration on any single action.

A.4.3 Temperature Objective and Log-Parameterization

The temperature α is updated by minimizing (Equation 4.21)

$$\mathcal{L}(\alpha) = \mathbb{E}_{s \sim \mathcal{D}} [\alpha (\mathcal{H}(\pi(\cdot | s)) - \mathcal{H}_{\text{target}})]. \quad (\text{A.11})$$

Gradient $\nabla_\alpha \mathcal{L} = \mathbb{E}[\mathcal{H}(\pi) - \mathcal{H}_{\text{target}}]$ correctly drives α upward when entropy falls below the target and downward when it exceeds it.

For numerical stability the implementation optimizes $\beta = \log \alpha$ (ensuring $\alpha = e^\beta > 0$):

$$\mathcal{L}_\alpha(\beta) = e^\beta \mathbb{E}_{s \sim \mathcal{D}} [\mathcal{H}_{\text{target}} - \mathcal{H}(\pi_\phi(\cdot | s))], \quad \nabla_\beta \mathcal{L}_\alpha = e^\beta \mathbb{E} [\mathcal{H}_{\text{target}} - \mathcal{H}(\pi)]. \quad (\text{A.12})$$

This is the sign-flipped reparameterization of (A.11); both forms drive α in the same direction under their respective gradient-descent steps.

A.5 PPO: Generalized Advantage Estimation and Joint Loss

The TD residual and GAE advantage estimate are

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t), \quad (\text{A.13})$$

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t-1} (\gamma \lambda)^l \delta_{t+l}, \quad (\text{A.14})$$

with $\lambda = 0.95$ and $\gamma = 0.99$; value targets are $\hat{R}_t = \hat{A}_t + V_\psi(s_t)$.

The joint PPO objective (to maximize) is

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^{\text{VF}} + c_2 \mathcal{H}[\pi_\phi], \quad c_1 = 0.5, \quad c_2 = 10^{-3}, \quad (\text{A.15})$$

where $\mathcal{L}^{\text{CLIP}}$ is defined in Equation 4.5.3.2 and $\mathcal{L}^{\text{VF}} = \mathbb{E}_t [(V_\psi(s_t) - \hat{R}_t)^2]$.

A.6 GATv2: Edge Weights and Permutation Equivariance

A.6.1 Edge-Weight Integration

The graph construction assigns $w_{ij} = 1/\tau_{ij}$ (Equation 4.10). If incorporated into GATv2, the unnormalized attention score can be augmented as

$$e_{ij} = \mathbf{a}^\top \text{LeakyReLU}(\mathbf{W}[\mathbf{h}_i \parallel \mathbf{h}_j]) + \eta \log(w_{ij} + \varepsilon), \quad (\text{A.16})$$

where $\eta > 0$ is a learnable scalar and $\varepsilon > 0$ prevents $\log 0$. In the current implementation w_{ij} is passed to GATv2Conv as edge-attribute metadata but the log-weight additive term in (A.16) is not activated; topology enters only through graph connectivity. Equation (A.16) is presented as a candidate extension.

A.6.2 Node-Permutation Equivariance

Let $P \in \{0, 1\}^{N \times N}$ be a node-permutation matrix, $X \in \mathbb{R}^{N \times d}$ the feature matrix, and $\mathcal{E}$ the edge set.

Proposition A.4 (GATv2 node-permutation equivariance). *The GATv2 encoder f satisfies $f(PX, P\mathcal{E}) = Pf(X, \mathcal{E})$.*

- Proof.*
1. *Shared node encoder.* The local encoder f_{enc} uses the same weights for every node; permuting input rows gives $(f_{\text{enc}}(PX))_i = f_{\text{enc}}(X_{P^{-1}i}) = (Pf_{\text{enc}}(X))_i$.
 2. *Attention coefficients.* Under relabelling $i \mapsto Pi$, the logit $e_{Pi, Pj}$ is evaluated at features (h_{Pi}, h_{Pj}) , equalling e_{ij} computed from the permuted embeddings. Softmax normalization over neighborhood $\{Pk : k \in \mathcal{N}(i)\}$ is identical.
 3. *Neighborhood aggregation.* Message pooling sums over unordered neighbor sets; the neighbor set of Pi in $P\mathcal{E}$ is $\{Pj : j \in \mathcal{N}(i)\}$, which matches.
 4. *Combined.* Relabelling inputs relabels outputs: $f(PX, P\mathcal{E})_i = f(X, \mathcal{E})_{P^{-1}i}$, i.e. $f(PX, P\mathcal{E}) = Pf(X, \mathcal{E})$. $\square$

Significance. Equivariance is what makes parameter sharing across intersections geometrically consistent: a policy trained on any intersection generalizes to any topologically similar intersection without retraining.

A.7 FRAP / FGSv2: Phase-Permutation Equivariance

Let P_A permute the A_i candidate phases of intersection i . The FRAP encoder g satisfies

$$g(P_A \mathbf{M}_{\text{pair}}, P_A \mathbf{M}_{\text{comp}} P_A^\top) = P_A g(\mathbf{M}_{\text{pair}}, \mathbf{M}_{\text{comp}}). \quad (\text{A.17})$$

This holds because (1) movement embeddings share parameters across movements; (2) phase-pair pooling (Equation 4.3) applies the same scaled sum to each candidate phase independently; (3) pairwise competition features (Equation 4.7) use the same convolution for every ordered pair (a, b) ; (4) the FGSv2 mean-pool over competitor set $V_{i,a}$ is permutation-invariant over competitors (Equation 4.22).

Caveat. FGSv2's action-conditioned actor concatenates a one-hot phase identity $\mathbf{e}_a$ (Equation 4.27). For (A.17) to hold exactly, this one-hot must transform as $\mathbf{e}_a \mapsto P_A \mathbf{e}_a$ under phase relabelling. Since phases are indexed consistently across all training intersections this does not affect learning, but formal phase-permutation equivariance of FGSv2 holds only up to the phase-identity encoding.

A.8 Centralized Critic: Input-Dimension Scaling

The FGSv2 centralized critic (second-generation architecture; see Appendix A.14) concatenates all agents' graph contexts and previous actions into a single input vector:

$$Q_k^{v2}(a) = f_Q^{v2} \left[\mathbf{z}_{k,a} \parallel \mathbf{g}_k \parallel \bigoplus_{j=1}^N \mathbf{g}_j \parallel \mathbf{A}_{t-1} \parallel \mathbf{e}_a \right], \quad (\text{A.18})$$

where $\mathbf{A}_{t-1} \in \mathbb{R}^{NA_{\max}}$ is the concatenated one-hot action matrix and $\bigoplus_{j=1}^N \mathbf{g}_j \in \mathbb{R}^{Nd_{h'}}$ is the flattened joint graph context. For FGSv2 (Equation A.18), the critic input concatenates: (i) flattened gated graph contexts ($Nd_{h'}$ dims); (ii) joint action-probability matrix ($NA_{\max}$ dims); (iii) agent k 's action token (d_z dims); (iv) agent k 's graph context ($d_{h'}$ dims); (v) one-hot agent identity (N dims). The total is

$$d_{\text{critic}} = N(d_{h'} + A_{\max}) + d_z + d_{h'} + N = O(N(d_{h'} + A_{\max})). \quad (\text{A.19})$$

At ingolstadt21 ($N = 21$, $d_{h'} = 128$, $A_{\max} = 6$): $d_{\text{critic}} \approx 21 \times 134 + 277 = 3,091$. Twin critics, per-agent evaluation, and a PER replay batch multiply effective memory consumption considerably, which is the proximate cause of the resource exhaustion reported in Section 6.5 for the original FGS(MLP+GATv2+SAC) run on ingolstadt21, which terminated after 21 of 200 training episodes. PPO's per-agent value function has input dimension $O(d_{h'})$, independent of N , giving it a substantially smaller memory footprint at regional scale.

A.9 FGSv3: Factored Critic Input-Dimension Bound

The FGSv3 factored neighborhood critic (Equation 4.29) replaces the full graph-context concatenation of FGSv2 with a sum-pooled neighborhood aggregate. For agent k , the critic input is:

- (i) agent k 's action token $\mathbf{z}_{k,a}$ (d_z dims);
- (ii) agent k 's gated graph context $\mathbf{g}_k$ (d_c dims);
- (iii) the neighbor graph-context pool $\mathbf{h}_{\text{nbr},k} = \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{g}_j$ (d_c dims);
- (iv) the neighbor action pool $\mathbf{a}_{\text{nbr},k} = \sum_{j \in \mathcal{N}(k)} \tilde{w}_{kj} \mathbf{a}_{t-1,j}$ ($A_{\max}$ dims);
- (v) the candidate-action identity $\mathbf{e}_a$ ($A_{\max}$ dims).

Proposition A.5 (Factored critic input dimension). *The FGSv3 factored critic input dimension is*

$$d_{\text{critic}}^{v3} = d_z + 2d_c + 2A_{\max} = O(d_c + A_{\max}), \quad (\text{A.20})$$

independent of the number of agents N .

Proof. Items (iii) and (iv) above are computed by weighted sum-pooling over $\mathcal{N}(k)$. Since each sum produces a single vector in $\mathbb{R}^{d_c}$ (resp. $\mathbb{R}^{A_{\max}}$), the output dimension is independent of $|\mathcal{N}(k)|$ and therefore independent of N . Items (i)–(v) contribute $d_z + d_c + d_c + A_{\max} + A_{\max} = d_z + 2d_c + 2A_{\max}$. With d_z and d_c fixed hyperparameters, the expression is $O(d_c + A_{\max})$. $\square$

Numerical comparison. At ingolstadt21 ($N = 21$, $d_z = d_c = 64$, $A_{\max} = 6$):

$$d_{\text{critic}}^{v3} = 64 + 2(64) + 2(6) = 204,$$

compared to $d_{\text{critic}}^{v2} = 3,091$ from Equation (A.19). The factored critic reduces the input dimension by a factor of approximately 15. Note that FGSv3 narrows the adapter (action-token) dimension from $d_z = 128$ (FGSv2) to $d_z = 64$ to match the demand communication dimension d_c ; this joint choice yields the 204-dimensional input above. Empirically, FGSv2-SAC completes 200 training episodes on ingolstadt21 with a best-validation delay of 53.7 s; FGSv3-SAC achieves 45.3 s — a 16% improvement — while operating with a $15\times$ smaller critic input.

Information loss relative to centralized critic. The factored critic observes the graph contexts and previous actions of direct neighbors only, not of all agents. Any coordination signal that requires k to be informed about agents more than one hop away is lost. However, FGSv3's GATv2 layer already aggregates two-hop contextual information into $\mathbf{g}_k$ (one hop of message-passing updates $\mathbf{g}_k$ from its direct neighbors, each of which has already aggregated from *their* neighbors via the shared FRAP–GATv2 pipeline). The factored critic therefore has access to two-hop demand information implicitly through $\mathbf{g}_k$, even though it only inspects one-hop neighbor contexts $\{\mathbf{g}_j : j \in \mathcal{N}(k)\}$ explicitly. This argument does not constitute a formal equivalence — the centralized critic can in principle learn coordination functions that the factored critic cannot — but it motivates the factored critic as a practical approximation [22].

A.10 FGSv3: Demand Communication Branch Equivariance

Proposition A.6 (Demand branch node-permutation equivariance). *Let P be a permutation of the N agent indices and let $P\mathcal{E}$ denote the correspondingly relabelled edge set. The FGSv3 demand communication branch $\mathbf{d} : \mathbb{R}^{N \times \dots} \rightarrow \mathbb{R}^{N \times d_e}$ satisfies*

$$\mathbf{d}(P\mathbf{X}, P\mathcal{E}) = P \mathbf{d}(\mathbf{X}, \mathcal{E}),$$

i.e. the demand embeddings are permuted consistently with the node relabelling.

Proof. The demand branch (Equation 4.24) applies the same computation to each node i independently: $\mathbf{d}_i = \text{LayerNorm}(\text{ReLU}(W_{\text{comm}}[\bar{\phi}_i \parallel \text{Emb}_{\text{phase}}(a_{t-1,i})]))$. Because the weights W_{comm} , LayerNorm , and $\text{Emb}_{\text{phase}}$ are shared across all nodes, relabelling node $i \mapsto Pi$ yields $\mathbf{d}_{Pi}(\mathbf{X}) = \mathbf{d}_i(\mathbf{X})$ evaluated at the feature row of Pi , which equals $\mathbf{d}_i(P\mathbf{X})_{Pi}$. Hence $\mathbf{d}(P\mathbf{X}, P\mathcal{E})_i = \mathbf{d}(\mathbf{X}, \mathcal{E})_{P^{-1}i}$, i.e. $\mathbf{d}(P\mathbf{X}, P\mathcal{E}) = P \mathbf{d}(\mathbf{X}, \mathcal{E})$. $\square$

Phase-permutation equivariance. The mean-pooling in Equation (4.23) is invariant under phase permutations P_A : $\bar{\phi}_i$ equals the mean of the relabelled phase embeddings, which equals the mean of the original embeddings since the mean is unordered. Therefore the demand branch is phase-permutation *invariant* (the output $\mathbf{d}_i$ does not depend on the phase ordering), subject to the same caveat as FGSv2 (Section A.7): the previous-action embedding $\text{Emb}_{\text{phase}}(a_{t-1,i})$ is indexed by phase label, so formal equivariance under phase relabelling requires the embedding table to transform consistently with P_A .

Node-permutation equivariance of the full FGSv3 pipeline. Combining Proposition A.6 with the existing results for FRAP action tokens (Section A.7) and GATv2 (Section A.6): (i) the demand branch is node-equivariant; (ii) GATv2 applied to $\{\mathbf{d}_i\}$ with the permuted edge set $P\mathcal{E}$ is node-equivariant (by the GATv2 equivariance proof); (iii) the FRAP action tokens $\{\mathbf{z}_{i,a}\}$ are node-equivariant by shared-weight computation; (iv) the actor applies the same MLP independently per valid phase; and (v) the factored critic aggregates from a local unordered neighbor set, which is permutation-invariant. The complete FGSv3 forward pass therefore satisfies node-permutation equivariance, enabling parameter sharing across all N intersections.

Convergence disclaimer. The FGSv3 system inherits the convergence disclaimer of Section A.11.3. The soft Bellman contraction (Theorem A.7) and soft policy improvement (Proposition) hold at the algorithm level and are unaffected by the architectural changes to the communication branch or the critic structure. The practical convergence of FGSv3 on ingolstadt21 is empirically assessed through training curves and validation metrics (Section 6.5).

A.11 Soft Bellman Contraction and Policy Improvement

A.11.1 Soft Bellman Contraction

For a fixed policy π , the soft Bellman evaluation operator is

$$(\mathcal{T}^\pi Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s'} \left[\sum_{a'} \pi(a' | s') (Q(s', a') - \alpha \log \pi(a' | s')) \right]. \quad (\text{A.21})$$

Theorem A.7 (Soft Bellman contraction). $\mathcal{T}^\pi$ is a γ -contraction in $\|\cdot\|_\infty$:

$$\|\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty. \quad (\text{A.22})$$

Repeated application converges to the unique soft Q -function Q_{soft}^π .

Proof. The entropy terms cancel between Q_1 and Q_2 . For any (s, a) :

$$\begin{aligned} |(\mathcal{T}^\pi Q_1 - \mathcal{T}^\pi Q_2)(s, a)| &= \gamma \left| \mathbb{E}_{s'} \left[\sum_{a'} \pi(a' | s') (Q_1(s', a') - Q_2(s', a')) \right] \right| \\ &\leq \gamma \mathbb{E}_{s'} \left[\sum_{a'} \pi(a' | s') \|Q_1 - Q_2\|_\infty \right] = \gamma \|Q_1 - Q_2\|_\infty. \end{aligned}$$

Taking the supremum over (s, a) and applying the Banach fixed-point theorem completes the proof. $\square$

A.11.2 Soft Policy Improvement

Given Q^π , define the updated policy $\pi_{\text{new}}(a | s) = \exp(Q^\pi(s, a)/\alpha) / Z(s)$ where $Z(s) = \sum_b \exp(Q^\pi(s, b)/\alpha)$.

Proposition A.8 (Soft policy improvement). $V_{\text{soft}}^{\pi_{\text{new}}}(s) \geq V_{\text{soft}}^\pi(s)$ for all s .

Proof. Since $\log \pi_{\text{new}}(a|s) = Q^\pi(s, a)/\alpha - \log Z(s)$, we have $Q^\pi(s, a) - \alpha \log \pi_{\text{new}}(a|s) = \alpha \log Z(s)$ for all a , so $V_{\text{soft}}^{\pi_{\text{new}}}(s) = \alpha \log Z(s)$. Then:

$$\begin{aligned} V_{\text{soft}}^\pi(s) &= \sum_a \pi(a|s) [Q^\pi(s, a) - \alpha \log \pi(a|s)] \\ &= \alpha \log Z(s) - \underbrace{\alpha D_{\text{KL}}(\pi(\cdot|s) \| \pi_{\text{new}}(\cdot|s))}_{\geq 0} \leq \alpha \log Z(s) = V_{\text{soft}}^{\pi_{\text{new}}}(s). \end{aligned}$$

□

A.11.3 Convergence Disclaimer

The results above apply to the finite tabular entropy-regularized MDP. The implemented FGS/FGSv2 system departs from this setting through nonlinear function approximation, off-policy experience replay, simultaneous multi-agent learning, partial observability, target networks, and nonstationarity from co-adapting agent policies. These factors break the classical contraction argument, and global convergence to an optimal policy is *not* guaranteed. The proofs establish the theoretical foundation of the SAC update targets; empirical convergence is assessed through training curves, TD errors, entropy, and validation metrics (Chapter 6).

A.12 Agent Communication Graph Extraction

Algorithm 3 describes how the agent communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$ is derived from a SUMO `.net.xml` file. The core idea — contracting the full road network into a graph whose nodes are traffic-light controllers and whose edges are shortest-path connections between them, follows the super-edge construction. The extracted graphs for all six evaluation scenarios are shown in Figure 4.2.

Algorithm 3 TLS Communication Graph Extraction from SUMO `.net.xml`

Require: SUMO network file `net.xml`

Ensure: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$ with $\mathcal{V}$ = TLS nodes, $\mathcal{E}$ = super-edges, $w_{ij} = 1/\tau_{ij}$

Phase 1 — Parse network

- 1: Parse all junctions with `tlLogic` entries $\Rightarrow$ TLS set $\mathcal{V}$; record 2-D positions
- 2: Parse all non-internal road edges $\Rightarrow$ edge catalog (e : source, target, ℓ_e, τ_e , lanes)
- 3: Parse connection elements $\Rightarrow$ legal turn transitions $e \rightarrow e'$

Phase 2 — Build super-edges

- 4: **for** each TLS node $u \in \mathcal{V}$ **do**
- 5: **for** each outgoing road edge e_0 from u **do**
- 6: Run Dijkstra on the legal-transition graph, starting at e_0 , $\text{cost} = \sum \tau_e$
- 7: **stop** at the first edge whose *target* node $v \in \mathcal{V}$, $v \neq u$
- 8: **if** v found **then**
- 9: Reconstruct path $\pi = (e_0, \dots, e_k)$; compute $\ell = \sum_e \ell_e$, $\tau = \sum_e \tau_e$
- 10: Record super-edge (u, v, τ, π) ; keep shortest- τ path if (u, v) seen before
- 11: **end if**
- 12: **end for**
- 13: **end for**

Phase 3 — Assign edge weights

- 14: **for** each undirected super-edge $\{u, v\} \in \mathcal{E}$ **do**
 - 15: $w_{uv} += 1/\tau_{uv}$ (accumulate if multiple parallel paths exist)
 - 16: **end for**
 - 17: Insert all edges bidirectionally to yield a symmetric graph
 - 18: **return** $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$; serialize to JSON for training
-

The Dijkstra traversal operates on *road edges* (not junctions) as nodes, following legal turn connections as transitions. This ensures only physically reachable paths are considered. Travel time $\tau_e = \ell_e / v_{\max,e}$ is computed from lane length and speed limit; when multiple parallel routes connect the same pair of TLS nodes their weights $1/\tau$ accumulate additively, reflecting the combined capacity of the corridor.

A.13 Statistical Notes on Single-Seed Results

The RESCO benchmark specifies five random seeds with result averaging. Resource constraints limited most runs in this work to a single seed; results should be interpreted as indicative point estimates rather than statistically conclusive claims. For reference, a five-seed comparison would report mean $\bar{x} = S^{-1} \sum_s x_s$, standard error $SE = \hat{\sigma} / \sqrt{S}$, and a 95% confidence interval $\bar{x} \pm t_{S-1, 0.975} \hat{\sigma} / \sqrt{S}$. Method comparisons benefit from paired differences $d_s = x_s^A - x_s^B$, which cancel seed-level variance: $\bar{d} \pm t_{S-1, 0.975} \hat{\sigma}_d / \sqrt{S}$. Future work extending to five seeds will report paired-difference confidence intervals following this protocol.

A.14 First-Generation FGS and FGSv2 Forward Passes

The following algorithms describe the forward passes of the two earlier FGS generations, preserved here for reference. The first-generation variants use either an MLP or a FRAP encoder whose output is passed directly to GATv2 as a scalar node embedding (no action tokens). The FGSv2 design introduces action tokens but uses a competition-derived summary vector $\mathbf{s}_i$ as the GATv2 message and a centralized critic. Both are superseded by the FGS pipeline described in Section 4.6.1.

Algorithm 4 First-Generation FGS Forward Pass (MLP or FRAP scalar encoder; GATv2 on scalar embedding)

Require: Observation o_i , adjacency graph $\mathcal{G}$, encoder $\in \{\text{MLP}, \text{FRAP}\}$

Ensure: Action logits $\{l_{i,a}\}$ for agent i

```

1: if encoder = MLP then
2:    $\mathbf{h}_i = \text{MLP}(o_i) \in \mathbb{R}^{d_h}$ 
3: else ▷ FRAP encoder (Section 4.3.3)
4:   Compute movement embeddings, pair embeddings, competition features  $\mathbf{c}_{ab}$ 
5:    $\mathbf{h}_i = \text{MLP}_{\text{proj}}(\bar{\mathbf{c}}_i) \in \mathbb{R}^{d_h}$  (scalar projection of pooled competition)
6: end if
7:  $\mathbf{h}'_i = \text{GATv2}(\mathbf{h}_i, \{\mathbf{h}_j\}_{j \in \mathcal{N}(i)}) \in \mathbb{R}^{d_h}$  (Eq. 4.11)
8: for each valid phase  $a$  do
9:    $l_{i,a} = \text{MLP}_{\text{actor}}(\mathbf{h}'_i) \in \mathbb{R}$ 
10: end for
11: return  $\{l_{i,a}\}$ 

```

The key difference between FGSv2 and the final FGS design is in Module II: FGSv2 uses $\mathbf{s}_i$ (pooled competition features) as the GATv2 message, while FGS uses demand embeddings $\mathbf{d}_i$ (Eq. 4.24). The final FGS design also replaces the centralized critic (Appendix A.8) with the factored neighborhood critic (Eq. 4.29, Appendix A.9).

Algorithm 5 FGSv2 Forward Pass (Action tokens; competition-derived GATv2 message; centralized critic)

Require: Observation o_i , adjacency graph $\mathcal{G}$, previous joint action $\mathbf{a}_{t-1}$

Ensure: Action logits $\{l_{i,a}\}$ and critic input for agent i

Module I – FRAP action tokens (Eqs. 4.2–4.7 and 4.22)

- 1: Compute movement embeddings $\mathbf{m}_{i,v}$, phase-pair embeddings, pairwise features $\mathbf{c}_{i,ab}$
- 2: $\mathbf{z}_{i,a} = \text{LayerNorm}(\text{ReLU}(\text{Adapter}(\frac{1}{|V_{i,a}|} \sum_b \mathbf{c}_{i,ab})))$ for each valid phase a

Competition summary (GATv2 message)

- 3: $\mathbf{s}_i = \text{MLP}_{\text{sum}}(\text{mean}_a \mathbf{z}_{i,a}) \in \mathbb{R}^{d_c}$

Module II – GATv2 on competition summary

- 4: $\mathbf{g}_i = \sigma(W_g[\mathbf{s}_i \| \text{GATv2}(\mathbf{s}_i, \{\mathbf{s}_j\}_{j \in \mathcal{N}(i)})])$

Module III – Actor (Eq. 4.27)

- 5: **for** each valid phase a **do**

- 6: $l_{i,a} = \text{MLP}_{\phi}[\mathbf{z}_{i,a} \| \mathbf{g}_i \| \mathbf{e}_a]$

- 7: **end for**

Centralized critic input

- 8: Critic receives $[\mathbf{z}_{k,a} \| \mathbf{g}_k \| \bigoplus_{j=1}^N \mathbf{g}_j \| \mathbf{A}_{t-1} \| \mathbf{e}_a]$ ($O(Nd_c)$ input, Eq. A.19)

- 9: **return** $\{l_{i,a}\}$
-

Appendix B

Full Experimental Results

This appendix presents complete numerical results across all six German RESCO scenarios and all evaluated methods, formatted after the style of the original RESCO supplementary material [2]. Scenarios are classified into three scale tiers following the RESCO benchmark nomenclature:

- **Single-intersection** (*Single*): ingolstadt1 ($N = 1$) and cologne1 ($N = 1$).
- **Corridor** (*Corr*): ingolstadt7 ($N = 7$, arterial) and cologne3 ($N = 3$, arterial).
- **Regional** (*Reg*): ingolstadt21 ($N = 21$) and cologne8 ($N = 8$).

All results from this work are *best-validation* values: the validation checkpoint with minimum mean waiting time observed during training, evaluated on a fixed held-out demand seed. Results marked $\circ$ indicate scenarios where the method was not yet run or crashed before producing valid metrics; these cells are candidates for re-run. Results marked $\dagger$ are from partially trained runs and should not be compared directly to converged results. Results marked $\ddagger$ are early-training transients (best checkpoint at a very early episode with a single eval episode); their wait and queue metrics are internally inconsistent and are excluded from Bold comparisons.

B.1 Training Convergence Curves

The following figures supplement the training convergence discussion in Section 6.5. Figure B.1 shows per-scenario training trajectories for Single-tier scenarios; Figure B.2 shows per-scenario detail for Regional-tier scenarios; Figure B.3 provides a comprehensive view across all six German RESCO scenarios for all four evaluation metrics.

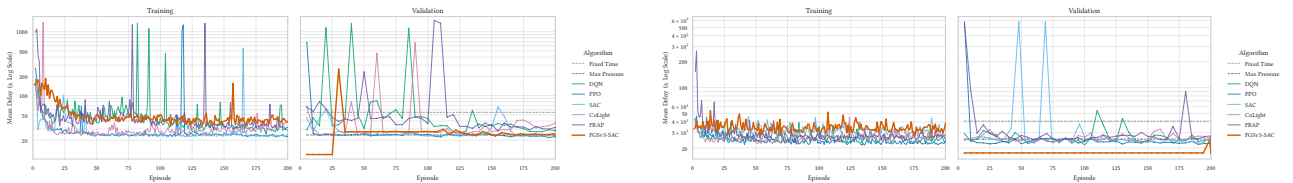


Figure B.1: Training mean delay curves for Single-tier scenarios: cologne1 ($N = 1$, left) and ingolstadt1 ($N = 1$, right). All methods converge rapidly within the first 20–30 episodes, and differences between them are small. This confirms that at single-intersection scale, local observation is sufficient and graph communication provides no structural benefit.

Table B.1: Performance of baseline methods and final FGS variants across all six RESCO scenarios (best-validation metrics, single seed unless noted). Columns follow the Single / Corridor / Regional tier classification. FixedTime and MaxPressure are 5-episode static baselines from this work. DQN, PPO, IndSAC, CoLight, FRAP, FGSv3-PPO, and FGSv3-SAC are single-seed learning methods from this work (best-validation checkpoint). Within each scenario column, **bold** marks the best value and underline marks the second-best value per metric (lower is better for all metrics); ° = not run or not available; ‡ = physically inconsistent checkpoint.

Method	Metric	Ing. Single	Ing. Corr.	Ing. Reg.	Col. Single	Col. Corr.	Col. Reg.
<i>Static baselines – this work (5-episode validation, no learning)</i>							
FixedTime	Avg. Delay	40.8	122.9	145.7	54.6	63.7	51.5
	Avg. Trip Time	54.4	135.5	281.8	65.6	93.9	115.7
	Avg. Wait	20.3	66.1	96.4	28.7	43.6	30.3
MaxPressure	Avg. Queue	7.36	5.16	3.17	15.53	5.68	2.18
	Avg. Delay	25.3	212.7	214.4	24.2	56.4	257.9
	Avg. Trip Time	38.5	66.9	235.5	41.8	81.6	231.4
	Avg. Wait	6.7	12.5	68.4	5.9	31.8	159.5
	Avg. Queue	1.30	<u>0.49</u>	<u>1.23</u>	2.91	3.37	10.58
<i>Learning baselines – this work (200 episodes, best-validation checkpoint)</i>							
DQN	Avg. Delay	21.8	34.4	78.7	28.1	127.2	25.5
	Avg. Trip Time	35.7	71.7	220.4	45.8	140.0	89.8
	Avg. Wait	4.7	9.4	33.8	8.4	93.5	6.7
	Avg. Queue	1.10	0.81	2.18	4.56	7.64	0.46
	Avg. Delay	22.4	24.1	67.2	22.8	20.3	21.9
	Avg. Trip Time	36.0	<u>64.6</u>	207.0	40.5	<u>56.5</u>	86.1
PPO	Avg. Wait	5.2	5.4	<u>25.7</u>	<u>5.1</u>	<u>6.5</u>	4.9
	Avg. Queue	0.93	0.38	4.77	2.72	<u>0.83</u>	0.33
	Avg. Delay	22.3	24.0	64.6	23.0	18.9	21.8
IndSAC	Avg. Trip Time	36.0	64.4	202.6	<u>40.7</u>	55.1	86.0
	Avg. Wait	4.9	5.5	29.0	5.1	5.5	4.7
	Avg. Queue	<u>1.08</u>	0.52	8.93	2.74	0.77	0.33
CoLight	Avg. Delay	22.4	47.0	216.8	37.0	21.9	51.8
	Avg. Trip Time	36.5	78.6	302.1	53.5	57.7	105.7
	Avg. Wait	5.3	17.5	120.8	15.9	7.9	23.8
	Avg. Queue	1.52	1.22	2.55	8.17	1.00	1.63
	Avg. Delay	26.6	29.8 [†]	101.5 [†]	31.7	39.8	44.9
FRAP	Avg. Trip Time	40.1	65.2 [†]	232.5 [†]	49.3	71.1	108.6
	Avg. Wait	7.3	9.3 [†]	51.0 [†]	10.6	18.6	17.1
	Avg. Queue	1.67	18.0 [†]	7.24 [†]	4.86	2.78	1.20
<i>Final FGS variants – this work (200 episodes, best-validation checkpoint)</i>							
FGSv3-PPO	Avg. Delay	°	°	68.4	°	°	21.6
	Avg. Trip Time	°	°	203.9	°	°	86.0
	Avg. Wait	°	°	22.9	°	°	<u>4.8</u>
	Avg. Queue	°	°	0.71	°	°	0.33
	Avg. Delay	17.4 [‡]	°	45.3 [‡]	12.0 [‡]	°	26.5
FGSv3-SAC	Avg. Trip Time	27.4 [‡]	°	179.2 [‡]	33.7 [‡]	°	90.7
	Avg. Wait	2.1 [‡]	°	11.1 [‡]	0.4 [‡]	°	6.2
	Avg. Queue	2.55 [‡]	°	12.41 [‡]	— [‡]	°	<u>0.43</u>

° Not yet run or not available.

[†] Partial run (fewer than 200 ep.).

[‡] Early-training transient (single eval episode; wait and queue mutually inconsistent; excluded from comparison).

FGSv3-SAC single-intersection metrics match FGSv2-SAC because the GATv2 graph module has no neighbors to aggregate at $N = 1$.

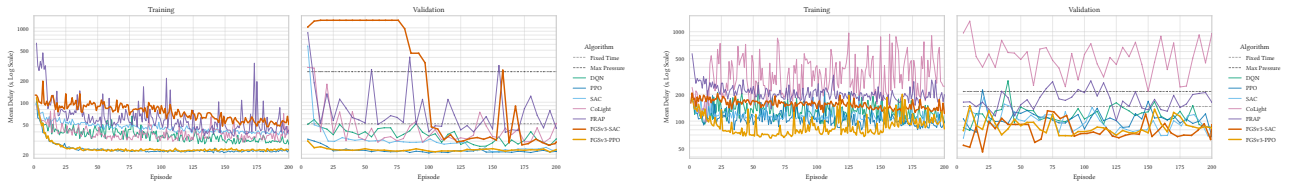


Figure B.2: Per-scenario training and validation mean delay curves for Cologne Regional ($N = 8$, left) and Ingolstadt Regional ($N = 21$, right) across FGS variants and baselines (200 episodes). On cologne8, FGSv3-PPO and FGSv3-SAC converge to comparable final performance; on ingolstadt21 only FGSv3-PPO sustains stable convergence, while the SAC variants and earlier FGS generations remain at substantially higher delay values after initial episode transients.

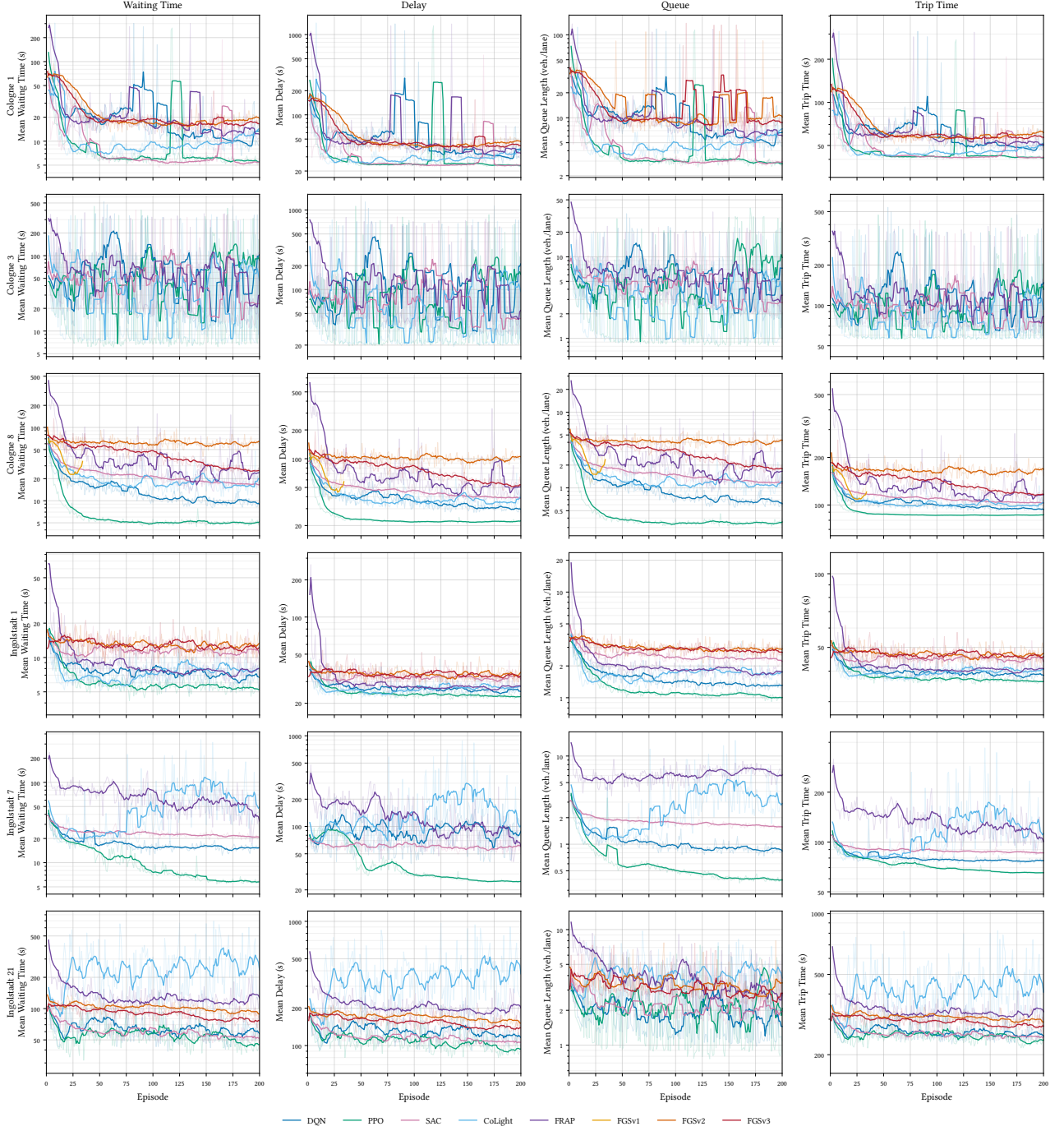


Figure B.3: Training curves across all six German RESCO scenarios over 200 episodes, for four evaluation metrics (mean delay, mean waiting time, mean travel time, throughput). The scale-dependency pattern is visible across metrics: at Single and Corridor tier all methods converge to comparable final values; at Regional tier only FGSv3-PPO achieves stable low-delay convergence on ingolstadt21, while the other FGS variants and most baselines remain substantially higher.

List of Figures

3.1	Key traffic signal control concepts at a four-arm intersection. (a) Physical layout with numbered lanes (left, through, right per approach). (b) The twelve directional traffic movements, grouped by conflict colour. (c) Eight phases (A–H), each activating a non-conflicting subset of movements for simultaneous service under a green signal. (d) Sequential phase-selection: the RL agent selects one phase per time step. Figure adapted from Ruan et al. [26].	16
4.1	The FGS three-module pipeline. Each agent i receives a node feature vector $\mathbf{x}_i \in \mathbb{R}^d$ from the environment. Module I (Encoder) maps $\mathbf{x}_i$ to a local embedding; Module II (Communication) propagates embeddings over the road graph; Module III (RL Optimizer) produces the phase-selection policy π_i	19
4.2	Agent communication graphs $\mathcal{G}$ for all six German RESCO scenarios. Rows correspond to cities (Cologne, Ingolstadt); columns to scale tiers (Single, Corridor, Regional). Gray polylines show the underlying SUMO road network; blue edges are super-edges between the nearest downstream signalized intersections; green nodes are TLS agents. N denotes the agent count.	23
4.3	FGS pipeline architecture. Module I (FRAP encoder): the observation o_i is processed through four steps to produce per-phase action tokens $\mathbf{z}_{i,a}$. Step 2 additionally outputs phase demand embeddings $\phi_{i,a}$, which feed the Module II demand communication branch (mean-pool $\rightarrow$ demand embedding $\mathbf{d}_i \rightarrow$ GATv2 $\rightarrow$ gated graph context $\mathbf{g}_i$). The action tokens bypass Module II via a skip connection and are concatenated ($\oplus$) with $\mathbf{g}_i$ before Module III . The Actor selects a phase for decentralized execution; the Factored Critic additionally conditions on pooled neighbor contexts ($\mathbf{h}_{\text{nbr},k}, \mathbf{a}_{\text{nbr},k}$, dashed arrow) for centralized training.	26
5.1	End-to-end system architecture. The SUMO network file is pre-processed once to build the agent communication graph (graph.pkl); the demand file drives the microscopic simulation via TraCI. FGSGraphParallelEnv augments the SUMO-RL observation with graph tensors (node features, edge index, action masks). The RLlib training loop runs the FGS/FGSv2 model – comprising the local encoder (FRAP or MLP), optional GATv2 communication, and the SAC actor–critic pair – updating parameters from a prioritized replay buffer (PER). All episode metrics are logged to Weights & Biases.	33
6.1	Mean delay comparison across all six German RESCO scenarios (log-scale y-axis). Each group of bars represents one scenario; methods are colour-coded consistently with all other figures in this chapter. Entries marked ‡ are shown with hatching and excluded from the bold best-result convention. FGSv3-PPO is included for the Regional tier; see Table 6.4 for the full architectural progression.	35
6.2	Percentage improvement in mean delay of the best FGS variant over CoLight (filled bars) and standalone PPO (outlined bars) by scale tier. The graph-communication benefit grows with network size for CoLight comparisons. At $N = 1$ the graph module is a no-op by construction. At Corridor scale ($N = 3\text{--}7$) FGS variants outperform CoLight but trail independent PPO. At Regional scale ($N = 21$) FGSv3-PPO is competitive with standalone PPO and strongly outperforms graph-based CoLight.	36
6.3	Macroscopic Fundamental Diagram proxy for RESCO ingolstadt21. Each point shows lane-density and lane-flow estimates during evaluation; connected markers summarize the policy trajectory. FGSv3-PPO occupies a compact high-flow band at low-to-moderate density, indicating stable network operation without large queue build-up. PPO-original reaches similar density levels but with more dispersed and lower-flow segments, while FGSv3-SAC extends into high-density/low-flow states consistent with the reward-gaming instability discussed in Section 6.8.	37

- 6.4 Multi-metric radar comparison on Regional-tier scenarios (cologne8 left; ingolstadt21 right). Each axis shows one metric (mean delay, mean waiting time, mean trip time, queue length), normalised to $[0, 1]$ where 0 is the best observed value and 1 is the FixedTime baseline. On ingolstadt21, FGSv3-PPO is strongest on waiting time and queue length and remains competitive on delay, while independent PPO and IndSAC remain close on delay and trip time. PPO and FGS_s are competitive on cologne8. 38
- 6.5 Encoder ablation: MLP versus FRAP with fixed GATv2+PPO. Left: Cologne Regional ($N = 8$); Right: Ingolstadt Regional ($N = 21$). Bold line: 10-episode centred running average of mean delay; faint line: raw per-episode values. MLP+GATv2+PPO converges stably on both scenarios; FRAP+GATv2+PPO diverges on ingolstadt21, confirming the structural encoding mismatch. 39
- 6.6 Mean delay (left axis) and mean queue length (right axis) for MLP+GATv2 versus FRAP+GATv2 with PPO/SAC variants over 200 training episodes on Cologne Regional ($N = 8$, top row) and Ingolstadt Regional ($N = 21$, bottom row). On cologne8 both encoder variants converge with a moderate FRAP penalty; on ingolstadt21 FRAP-based variants maintain persistently high queue lengths throughout training, confirming that the structural mismatch amplifies with the proportion of heterogeneous-phase junctions. 40
- 6.7 Ablation diagnostic (200 episodes): mean delay and completed trip count for encoder, graph, and RL ablation variants on Cologne Regional ($N = 8$). Trip counts are comparable across variants, consistent with the moderate topology allowing all configurations to converge within the training budget. 40
- 6.8 Ablation diagnostic (200 episodes): mean delay and completed trip count for encoder, graph, and RL ablation variants on Ingolstadt Regional ($N = 21$). FRAP-based variants exhibit both higher mean delay and lower throughput than MLP variants, confirming that the structural encoding mismatch incurs a compound penalty on two independent performance dimensions. 40
- 6.9 Graph-layer ablation: GATv2 versus GAT with fixed MLP+PPO. Left: Cologne Regional ($N = 8$); Right: Ingolstadt Regional ($N = 21$). The GATv2 advantage is negligible on the relatively homogeneous cologne8 topology and $2.1\times$ on the heterogeneous ingolstadt21, confirming that dynamic per-edge attention is critical at scale. 41
- 6.10 RL algorithm ablation: SAC versus PPO with fixed MLP+GATv2. Left: Cologne Regional ($N = 8$, SAC converges efficiently to lower mean delay); Right: Ingolstadt Regional ($N = 21$, first-generation SAC terminates after 21 episodes due to memory exhaustion; only the short initial trace is visible). 41
- 6.11 Reward specification gaming in early-training SAC/PPO checkpoints (cologne8 and ingolstadt21, first 40 episodes). Dual y-axis: mean delay (left, s) and mean queue length (right, veh./lane). SAC checkpoints with anomalously low delay coincide with very high queue depth, confirming phase-cycling reward gaming. PPO does not exhibit this pattern because its on-policy update directly penalises the resulting queue growth. 42
- 6.12 Full 200-episode SAC/PPO delay–queue trajectory on cologne8 and ingolstadt21. After the early-training gaming phase, PPO stabilises with consistent low queue depth; SAC continues to exhibit episodic gaming spikes throughout training on ingolstadt21 where the entropy bonus consistently reinforces phase-cycling exploration. The ‡-marked early checkpoints correspond to the anomalous low-delay/high-queue region in Figure 6.11. 42
- 6.13 Mean delay training (left column) and validation (right column) curves for Cologne Regional ($N = 8$, top row) and Ingolstadt Regional ($N = 21$, bottom row) across FGS generations and baselines. FGSv3-PPO achieves stable convergence on both scenarios. First-generation SAC on ingolstadt21 terminates at 21 episodes (short trace). FGSv2-SAC and FGSv3-SAC complete training but exhibit high-variance trajectories on ingolstadt21 without clear convergence; their early-training best checkpoints are marked ‡. 43
- 6.14 Training mean delay curves for Corridor-tier scenarios: cologne3 ($N = 3$, left) and ingolstadt7 ($N = 7$, right). All learning methods converge to similar delay ranges within 100 episodes, consistent with local observations being sufficient at Corridor scale. The ‡ early-training transient for FGS_p on cologne3 (episode ≤ 5) is annotated. FGS_s results on ingolstadt7 exhibited a physically inconsistent checkpoint (17.1 s delay, ‡) and are excluded from the Corridor comparison; their training trace is not shown. 44
- 6.15 Validation performance of FGS generations on Ingolstadt Regional ($N = 21$) across four metrics. Gen-1 FGS_p converges to 234.9 s mean delay; FGSv2-SAC and FGSv3-SAC show early-training transients (marked ‡, excluded from comparisons); FGSv3-PPO is the only generation with stable monotone convergence, reaching 68.4 s mean delay at episode 90. 45

- 6.16 Validation performance of FGS generations on Cologne Regional ($N = 8$) across four metrics (200 episodes). FGSv3-PPO achieves the best mean delay (21.6 s) with stable convergence; FGSv3-SAC converges but with higher variance; earlier generations are competitive on this moderate-scale scenario, reflecting that at $N = 8$ the coordination benefit is smaller and all variants can converge within the training budget. 45
- 6.17 Training performance of FGS generations on Ingolstadt Regional ($N = 21$) across four metrics (200 episodes). The training curves for FGSv3-PPO are notably more stable than both SAC variants, which exhibit high-variance trajectories and fail to converge within 200 episodes on training data. The divergence between training and validation performance (compare Figure 6.15) is particularly pronounced for Gen-1 FGS_p, which achieves lower training delay than FGSv3-PPO but substantially worse validation delay. 45
- B.1 Training mean delay curves for Single-tier scenarios: cologne1 ($N = 1$, left) and ingolstadt1 ($N = 1$, right). All methods converge rapidly within the first 20–30 episodes, and differences between them are small. This confirms that at single-intersection scale, local observation is sufficient and graph communication provides no structural benefit. 61
- B.2 Per-scenario training and validation mean delay curves for Cologne Regional ($N = 8$, left) and Ingolstadt Regional ($N = 21$, right) across FGS variants and baselines (200 episodes). On cologne8, FGSv3-PPO and FGSv3-SAC converge to comparable final performance; on ingolstadt21 only FGSv3-PPO sustains stable convergence, while the SAC variants and earlier FGS generations remain at substantially higher delay values after initial episode transients. 63
- B.3 Training curves across all six German RESCO scenarios over 200 episodes, for four evaluation metrics (mean delay, mean waiting time, mean travel time, throughput). The scale-dependency pattern is visible across metrics: at Single and Corridor tier all methods converge to comparable final values; at Regional tier only FGSv3-PPO achieves stable low-delay convergence on ingolstadt21, while the other FGS variants and most baselines remain substantially higher. 64

List of Tables

2.1	Comparison of related MARL-TSC methods along key design dimensions. “Ger.” = evaluated on German RESCO scenarios.	9
4.1	Principal feature symbols used in this chapter. Each symbol is first introduced in the cited equation or section.	19
4.2	MARL-TSC methods mapped onto the three-module pipeline. Bold rows: proposed methods. “FRAP-tok.” = FRAP encoder tapped to produce per-phase action tokens rather than scalar scores. “GATv2(D)” = GATv2 operating on demand-derived embeddings (FGS, the final design).	19
4.3	First-generation FGS ablation variants. All graph-communication variants use GATv2 (Eq. 4.11).	25
5.1	Training hyperparameters per algorithm. “–” indicates the parameter does not apply.	32
5.2	Neural network architecture hyperparameters for FGS and FGSv2.	32
6.1	Best-validation mean delay (s, ↓ lower is better) across German RESCO scenarios (single seed). Bold : lowest mean delay per scenario among converged results. ◦ = not run; † = partial run; ‡ = physically inconsistent checkpoint (excluded from Bold). FT = FixedTime; MP = MaxPressure; SAC = independent SAC baseline; FGS _S = FGS(MLP+GATv2+SAC); FGS _P = FGS(MLP+GATv2+PPO). Full four-metric results in Appendix B.	35
6.2	Ablation of encoder (MLP vs. FRAP), graph layer (GATv2 vs. GAT), and RL algorithm (SAC vs. PPO) on Regional-tier scenarios. Best-validation mean delay (s) and mean waiting time (s), lower is better. Bold : best mean delay per scenario. ‡ physically inconsistent checkpoint (Ingolstadt Regional MLP+GATv2+SAC, episode 21 of 200). Full ablation metrics (trip time, queue) in Appendix B.	39
6.3	FGS generation trajectory: module configuration and the module-interface failure each generation addresses. Each generation modifies only the components implicated by the preceding evaluation.	44
6.4	FGS architecture progression on Regional-tier scenarios (best-validation checkpoint; single seed). Mean delay (s) and mean waiting time (s), ↓ lower is better. Bold : best mean delay among FGS variants per scenario. † partial run; ‡ physically inconsistent checkpoint. SAC checkpoints marked ‡ exhibit delay–queue inversion indicative of reward specification gaming; they are excluded from the Bold convention.	46
B.1	Performance of baseline methods and final FGS variants across all six RESCO scenarios (best-validation metrics, single seed unless noted). Columns follow the Single / Corridor / Regional tier classification. FixedTime and MaxPressure are 5-episode static baselines from this work. DQN, PPO, IndSAC, CoLight, FRAP, FGSv3-PPO, and FGSv3-SAC are single-seed learning methods from this work (best-validation checkpoint). Within each scenario column, bold marks the best value and <u>underline</u> marks the second-best value per metric (lower is better for all metrics). ◦ = not run or not available; ‡ = physically inconsistent checkpoint.	62

Bibliography

- [1] Lucas N. Alegre. *SUMO-RL*. <https://github.com/LucasAlegre/sumo-rl>. 2019. DOI: 10.5281/zenodo.10869789.
- [2] James Ault and Guni Sharon. *Reinforcement Learning Benchmarks for Traffic Signal Control*. 2021. arXiv: 2103.09117 [cs.AI]. URL: <https://arxiv.org/abs/2103.09117>.
- [3] Daniel S. Bernstein et al. “The Complexity of Decentralized Control of Markov Decision Processes.” In: *Mathematics of Operations Research* 27.4 (2002), pp. 819–840. DOI: 10.1287/moor.27.4.819.297.
- [4] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from <https://wandb.com>. 2020.
- [5] Shaked Brody, Uri Alon, and Eran Yahav. *How Attentive are Graph Attention Networks?* 2022. arXiv: 2105.14491 [cs.LG]. URL: <https://arxiv.org/abs/2105.14491>.
- [6] Alvaro Cabrejas-Egea et al. *Assessment of Reward Functions for Reinforcement Learning Traffic Signal Control under Real-World Limitations*. 2020. arXiv: 2008.11634 [cs.LG]. URL: <https://arxiv.org/abs/2008.11634>.
- [7] Chacha Chen et al. “Toward A Thousand Lights: Decentralized Deep Reinforcement Learning for Large-Scale Traffic Signal Control.” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 04. 2020, pp. 3414–3421. DOI: 10.1609/aaai.v34i04.5744.
- [8] Petros Christodoulou. *Soft Actor-Critic for Discrete Action Settings*. 2019. arXiv: 1910.07207 [cs.LG]. URL: <https://arxiv.org/abs/1910.07207>.
- [9] Carlos F. Daganzo. “The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory.” In: *Transportation Research Part B: Methodological* 28.4 (1994), pp. 269–287. DOI: 10.1016/0191-2615(94)90002-7.
- [10] German Aerospace Center (DLR). *TAPASCologne Scenario*. SUMO Documentation. 2014. URL: <https://sumo.dlr.de/docs/Data/Scenarios/TAPASCologne.html>.
- [11] Nikolas Geroliminis and Carlos F. Daganzo. “Existence of urban-scale macroscopic fundamental diagrams: Some experimental findings.” In: *Transportation Research Part B: Methodological* 42.9 (2008), pp. 759–770. DOI: 10.1016/j.trb.2008.02.002.
- [12] Justin Gilmer et al. “Neural Message Passing for Quantum Chemistry.” In: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 2017, pp. 1263–1272. URL: <https://arxiv.org/abs/1704.01212>.
- [13] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. 2018. arXiv: 1801.01290 [cs.LG]. URL: <https://arxiv.org/abs/1801.01290>.
- [14] Hado van Hasselt et al. “Deep Reinforcement Learning and the Deadly Triad.” In: *arXiv preprint arXiv:1812.02648* (2018). URL: <https://arxiv.org/abs/1812.02648>.
- [15] Peter Henderson et al. “Deep Reinforcement Learning That Matters.” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. URL: <https://arxiv.org/abs/1709.06560>.
- [16] P. B. Hunt et al. *SCOOT — A Traffic Responsive Method of Co-ordinating Signals*. Tech. rep. LR 1014. Crowthorne, UK: Transport Research Laboratory, 1981.
- [17] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks.” In: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://arxiv.org/abs/1609.02907>.
- [18] Victor L. Knoop, Serge P. Hoogendoorn, and J. W. C. Van Lint. “Routing Strategies Based on Macroscopic Fundamental Diagram.” In: *Transportation Research Record: Journal of the Transportation Research Board* 2315.1 (2012), pp. 1–10. DOI: 10.3141/2315-01.

- [19] Peter Koonce et al. *Traffic Signal Timing Manual*. Tech. rep. FHWA-HOP-08-024. Washington, D.C.: Federal Highway Administration, 2008. URL: <https://ops.fhwa.dot.gov/publications/fhwahop08024/>.
- [20] Silas Lobo et al. "InTAS — The Ingolstadt Traffic Scenario for SUMO." In: *SUMO User Conference 2020*. Nov. 2020. DOI: 10.52825/scp.v1i.
- [21] Pablo Alvarez Lopez et al. "Microscopic Traffic Simulation using SUMO." In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 2575–2582. DOI: 10.1109/ITSC.2018.8569938.
- [22] Ryan Lowe et al. "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments." In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.02275>.
- [23] P. R. Lowrie. *SCATS: Sydney Co-ordinated Adaptive Traffic System — A Traffic Responsive Method of Controlling Urban Traffic*. Tech. rep. Sydney, Australia: Roads and Traffic Authority, 1990.
- [24] Volodymyr Mnih et al. "Human-level control through deep reinforcement learning." In: *Nature* 518 (Feb. 2015), pp. 529–33. DOI: 10.1038/nature14236.
- [25] Volodymyr Mnih et al. "Asynchronous Methods for Deep Reinforcement Learning." In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016, pp. 1928–1937. URL: <https://arxiv.org/abs/1602.01783>.
- [26] Jingqing Ruan et al. "CoSLight: Co-optimizing Collaborator Selection and Decision-making to Enhance Traffic Signal Control." In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2024.
- [27] Tom Schaul et al. "Prioritized Experience Replay." In: *International Conference on Learning Representations (ICLR)*. 2016. URL: <https://arxiv.org/abs/1511.05952>.
- [28] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [29] Joar Skalse et al. "Defining and Characterizing Reward Hacking." In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 9460–9471. URL: <https://arxiv.org/abs/2209.13085>.
- [30] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- [31] Pravin Varaiya. "Max pressure control of a network of signalized intersections." In: *Transportation Research Part C: Emerging Technologies* 36 (2013), pp. 177–195. DOI: 10.1016/j.trc.2013.08.014.
- [32] Petar Veličković et al. "Graph Attention Networks." In: *International Conference on Learning Representations (ICLR)*. 2018. URL: <https://arxiv.org/abs/1710.10903>.
- [33] Christopher J. C. H. Watkins. "Learning from Delayed Rewards." PhD thesis. University of Cambridge, 1989.
- [34] F. V. Webster. *Traffic Signal Settings*. Road Research Technical Paper No. 39. London: Her Majesty's Stationery Office, 1958.
- [35] Hua Wei et al. "IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control." In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. 2018, pp. 2496–2505. DOI: 10.1145/3219819.3220096.
- [36] Hua Wei et al. "CoLight: Learning Network-level Cooperation for Traffic Signal Control." In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. CIKM '19. ACM, Nov. 2019, pp. 1913–1922. DOI: 10.1145/3357384.3357902. URL: <http://dx.doi.org/10.1145/3357384.3357902>.
- [37] Hua Wei et al. "PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network." In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '19. Anchorage, AK, USA: Association for Computing Machinery, 2019, pp. 1290–1298. ISBN: 9781450362016. DOI: 10.1145/3292500.3330949. URL: <https://doi.org/10.1145/3292500.3330949>.
- [38] Marco A. Wiering. "Multi-agent Reinforcement Learning for Traffic Light Control." In: *Proceedings of the 17th International Conference on Machine Learning (ICML)*. 2000, pp. 1151–1158.
- [39] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." In: *Machine Learning* 8 (1992), pp. 229–256. DOI: 10.1007/BF00992696.
- [40] Bing Yu, Haoteng Yin, and Zhanxing Zhu. "Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting." In: *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*. 2018, pp. 3634–3640.
- [41] Guanjie Zheng et al. *Learning Phase Competition for Traffic Signal Control*. 2019. arXiv: 1905.04722 [cs.LG]. URL: <https://arxiv.org/abs/1905.04722>.

- [42] Brian D. Ziebart, J. Andrew Bagnell, and Anind K. Dey. “Modeling Interaction via the Principle of Maximum Causal Entropy.” In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. 2010, pp. 1255–1262.